FACE IT 2026 · Hands-on Mini Project

ECG Signal Processing, Heart-Rate Analysis and Arrhythmia Classification

PhysioNet AHA Database — record 0001, second channel, fs = 250 Hz, 1-minute recording of a recruitment-medical-examination candidate

Student (GitHub)	**EaindrayMFU26**
Notebook	`EaindrayMFU26_ECG_Mini_Project.ipynb`
Working environment	JupyterLab 4.6.4 · Python 3.13 (Linux) · NumPy, SciPy, pandas, Matplotlib, scikit-learn, wfdb
Date	15 October 2026

Heart rate (1 min)
76 bpm
Normal (60–100 bpm)

R-peaks detected
76
13 in the first 10 s

Ectopic (PVC-like) beats
3
isolated, with compensatory pause

SVM test accuracy
100 %
36 / 36 test beats correct

Results at a glance

Step	Task	Result
1	Data acquisition (first 10 s)	Record 0001, channel 2, 2 500 samples; files verified against PhysioNet SHA-256 sums
2	FIR band-stop filtering	281-tap Kaiser FIR, 48–52 Hz; 49–51 Hz power −59 dB; visible change negligible (little 50-Hz interference)
3	R-peak detection (first 10 s)	13 R-peaks (Pan–Tompkins); beat 5 is ectopic
4	Heart rhythm (1 min)	**76 R-peaks → Heart Rate = 76 bpm → Normal** (75.6 bpm from mean RR)
5	Feature extraction (6 × 10 s)	Mean RR 0.782–0.806 s; mean QRS 0.061–0.068 s
6	ML classification (SVM)	Confusion matrix [[21, 0], [0, 15]]; accuracy, sensitivity, specificity, precision = 100 %

Contents

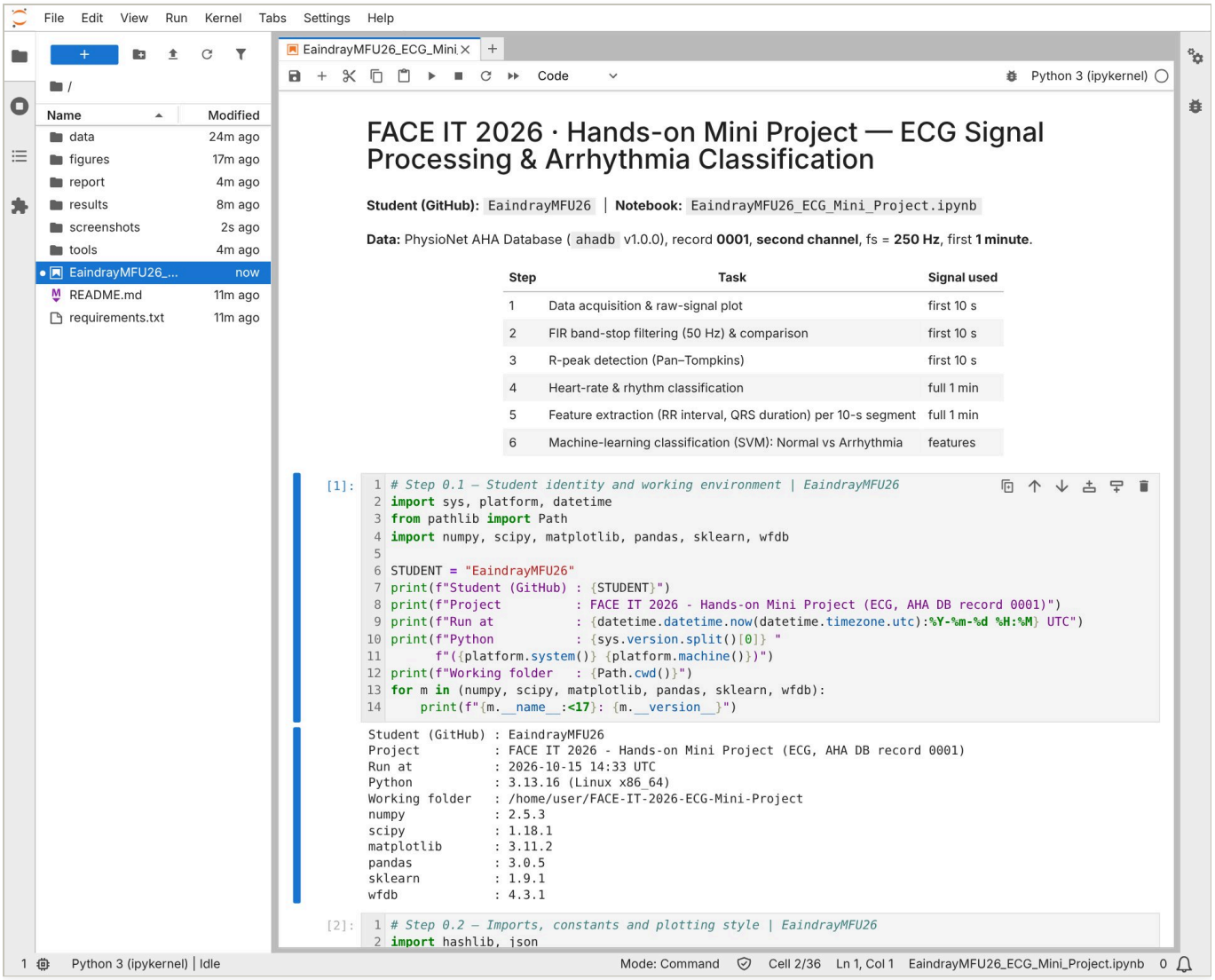
Case context, data and working environment 3
Step 1 — Data acquisition and visualization 6
Step 2 — ECG filtering (FIR band-stop) 9
Step 3 — R-peak detection 18
Step 4 — Heart-rhythm analysis 27
Step 5 — ECG feature extraction 34
Step 6 — Machine-learning classification 41
Interpretation, conclusion and limitations 54
Appendix A — Beat-level feature table 55
Appendix B — Machine-learning data set 56
Appendix C — Reproducibility 58
References 58

Each step contains (a) the method, (b) the code used, (c) screenshots of the code and its output taken in JupyterLab, and (d) the plots, tables and interpretation. Every screenshot shows the JupyterLab menu bar, the notebook tab `EaindrayMFU26_ECG_Mini_Project.ipynb`, the toolbar with the active kernel and the status bar. All numbers in this report were produced by the notebook and read from its saved result files.

Case context, data and working environment

As part of a recruitment medical examination, a candidate's ECG was recorded for one minute. The signal is taken from the **PhysioNet AHA Database (ahadb v1.0.0), record 0001**. The record contains two ECG channels sampled at **250 Hz** (12-bit, 400 adu/mV, 3 h long); as required, the **second channel** is analysed, using the first 10 s (Steps 1–3) and the first 60 s (Steps 4–5). The original PhysioNet files (`0001.hea` , `0001.dat` , `0001.atr`) were obtained from PhysioNet's official open-data mirror and all three match PhysioNet's published SHA-256 checksums (Step 1.1; see its output in the Step 1 screenshot).

All processing was done in a Jupyter notebook running in **JupyterLab**. Screenshot 1 shows the working environment: the project folder in the file browser, the notebook header, and the first cell, which prints the student identity, run time, Python version, working folder and package versions.



Screenshot 1. JupyterLab working environment: project folder, notebook `EaindrayMFU26_ECG_Mini_Project.ipynb` and the identity / environment cell (Step 0.1). [ss00_overview_environment.jpg]

Code 1 · Step 0.1 — Student identity and working environment

```
# Step 0.1 — Student identity and working environment | EaindrayMFU26
import sys, platform, datetime
from pathlib import Path
import numpy, scipy, matplotlib, pandas, sklearn, wfdb

STUDENT = "EaindrayMFU26"
print(f"Student (GitHub) : {STUDENT}")
print(f"Project          : FACE IT 2026 - Hands-on Mini Project (ECG, AHA DB record 0001)")
print(f"Run at           : {datetime.datetime.now(datetime.timezone.utc):%Y-%m-%d %H:%M} UTC")
```

```
print(f"Python           : {sys.version.split()[0]} "
      f"({platform.system()} {platform.machine()})")
print(f"Working folder   : {Path.cwd()}")
for m in (numpy, scipy, matplotlib, pandas, sklearn, wfdb):
    print(f"{m.__name__:<17}: {m.__version__}")
```

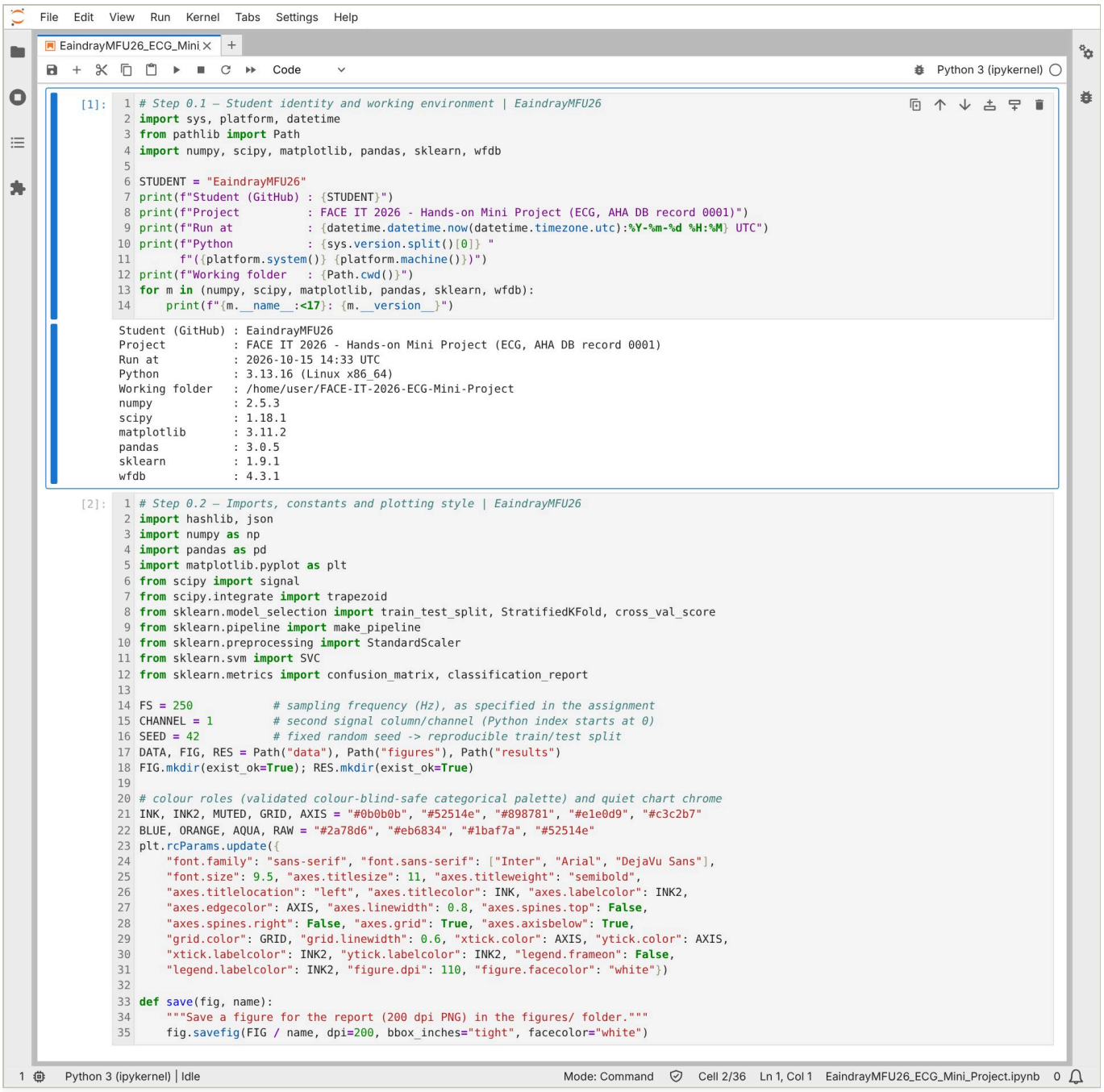
Code 2 · Step 0.2 — Imports, constants and plotting style

```
# Step 0.2 — Imports, constants and plotting style | EaindrayMFU26
import hashlib, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import signal
from scipy.integrate import trapezoid
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, classification_report

FS = 250            # sampling frequency (Hz), as specified in the assignment
CHANNEL = 1         # second signal column/channel (Python index starts at 0)
SEED = 42           # fixed random seed -> reproducible train/test split
DATA, FIG, RES = Path("data"), Path("figures"), Path("results")
FIG.mkdir(exist_ok=True); RES.mkdir(exist_ok=True)

# colour roles (validated colour-blind-safe categorical palette) and quiet chart chrome
INK, INK2, MUTED, GRID, AXIS = "#0b0b0b", "#52514e", "#898781", "#e1e0d9", "#c3c2b7"
BLUE, ORANGE, AQUA, RAW = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
plt.rcParams.update({
    "font.family": "sans-serif", "font.sans-serif": ["Inter", "Arial", "DejaVu Sans"],
    "font.size": 9.5, "axes.titlesize": 11, "axes.titleweight": "semibold",
    "axes.titlelocation": "left", "axes.titlecolor": INK, "axes.labelcolor": INK2,
    "axes.edgecolor": AXIS, "axes.linewidth": 0.8, "axes.spines.top": False,
    "axes.spines.right": False, "axes.grid": True, "axes.axisbelow": True,
    "grid.color": GRID, "grid.linewidth": 0.6, "xtick.color": AXIS, "ytick.color": AXIS,
    "xtick.labelcolor": INK2, "ytick.labelcolor": INK2, "legend.frameon": False,
    "legend.labelcolor": INK2, "figure.dpi": 110, "figure.facecolor": "white"})

def save(fig, name):
    """Save a figure for the report (200 dpi PNG) in the figures/ folder."""
    fig.savefig(FIG / name, dpi=200, bbox_inches="tight", facecolor="white")
```



Screenshot 2. Notebook cells 0.1, 0.2: identity, environment, imports, constants and plotting style. [ss01_step0.1.jpg]

Step 1 — Data acquisition and visualization (first 10 s)

Method. The record is read with `wfdb.rdrecord` (first 60 s only), the second signal column (`p_signal[:, 1]`, in mV) is selected, and a time axis $t = n / f_s$ with f_s = 250 Hz is built. The first 10 s (2 500 samples) are plotted as amplitude versus time.

Code 3 · Step 1.1 — Import AHA Database record 0001 from PhysioNet

```
# Step 1.1 — Import AHA Database record 0001 from PhysioNet | EaindrayMFU26
rec_dir = DATA / "ahadb"
sha_lines = (rec_dir / "SHA256SUMS.txt").read_text().splitlines()
sums = {name: digest for digest, name in (line.split() for line in sha_lines if line.strip())}
for f in ("0001.hea", "0001.dat", "0001.atr"):
    ok = hashlib.sha256((rec_dir / f).read_bytes()).hexdigest() == sums[f]
    print(f"{f:<9} SHA-256 matches PhysioNet: {ok}")

header = wfdb.rdheader(str(rec_dir / "0001"))
record = wfdb.rdrecord(str(rec_dir / "0001"), sampto=60 * FS)     # first 1 minute
# online alternative: wfdb.rdrecord("0001", pn_dir="ahadb", sampto=60 * FS)
assert record.fs == FS
print(f"\nRecord {header.record_name}: {header.n_sig} signals, fs = {header.fs} Hz, "
      f"full length = {header.sig_len / header.fs / 3600:.1f} h")
print("Signals:", header.sig_name, "| units:", header.units, "| ADC gain:", header.adc_gain)
```

Code 4 · Step 1.2 — Second channel: 1-minute and 10-second signals

```
# Step 1.2 — Second channel: 1-minute and 10-second signals | EaindrayMFU26
ecg60 = record.p_signal[:, CHANNEL]                # 1-minute ECG (mV), 15 000 samples
t60 = np.arange(ecg60.size) / FS                   # time axis (s)
ecg10, t10 = ecg60[:10 * FS], t60[:10 * FS]        # first 10 s, 2 500 samples
print(f"Channel used : column {CHANNEL + 1} of {record.n_sig} "
      f"({record.sig_name[CHANNEL]}, {record.units[CHANNEL]})")
print(f"Samples      : 1 min = {ecg60.size}, 10 s = {ecg10.size}")
pd.DataFrame({"first 10 s": pd.Series(ecg10).describe(),
              "full 60 s": pd.Series(ecg60).describe()}).round(3).T
```

Code 5 · Step 1.3 — Plot the raw ECG (first 10 s)

```
# Step 1.3 — Plot the raw ECG (first 10 s) | EaindrayMFU26
fig, ax = plt.subplots(figsize=(12, 3.6))
ax.plot(t10, ecg10, color=RAW, lw=1.0)
ax.set(title="Raw ECG — AHA DB record 0001, channel 2, first 10 s (fs = 250 Hz)",
       xlabel="Time (s)", ylabel="Amplitude (mV)", xlim=(0, 10))
ax.set_xticks(np.arange(0, 10.1, 1))
save(fig, "fig01_raw_ecg_10s.png"); plt.show()
```

File Edit View Run Kernel Tabs Settings Help

EaindrayMFU26_ECG_Mini

Code Python 3 (ipykernel)

Step 1 — Data acquisition and visualization (first 10 s)

Record 0001 of the PhysioNet AHA Database is read with the `wfdb` package. The files in `data/ahadb/` are the original PhysioNet files (`0001.hea`, `0001.dat`, `0001.atr`), obtained from PhysioNet's official open-data mirror, and their integrity is checked against PhysioNet's published `SHA256SUMS.txt`.

[3]:
```
# Step 1.1 — Import AHA Database record 0001 from PhysioNet | EaindrayMFU26
rec_dir = DATA / "ahadb"
sha_lines = (rec_dir / "SHA256SUMS.txt").read_text().splitlines()
sums = {name: digest for digest, name in (line.split() for line in sha_lines if line.strip())}
for f in ("0001.hea", "0001.dat", "0001.atr"):
    ok = hashlib.sha256((rec_dir / f).read_bytes()).hexdigest() == sums[f]
    print(f"{f:<9} SHA-256 matches PhysioNet: {ok}")

header = wfdb.rdheader(str(rec_dir / "0001"))
record = wfdb.rdrecord(str(rec_dir / "0001"), sampto=60 * FS)     # first 1 minute
# online alternative: wfdb.rdrecord("0001", pn_dir="ahadb", sampto=60 * FS)
assert record.fs == FS
print(f"\nRecord {header.record_name}: {header.n_sig} signals, fs = {header.fs} Hz, "
      f"full length = {header.sig_len / header.fs / 3600:.1f} h")
print("Signals:", header.sig_name, "| units:", header.units, "| ADC gain:", header.adc_gain)
```

```
0001.hea  SHA-256 matches PhysioNet: True
0001.dat  SHA-256 matches PhysioNet: True
0001.atr  SHA-256 matches PhysioNet: True

Record 0001: 2 signals, fs = 250 Hz, full length = 3.0 h
Signals: ['ECG', 'ECG'] | units: ['mV', 'mV'] | ADC gain: [400.0, 400.0]
```

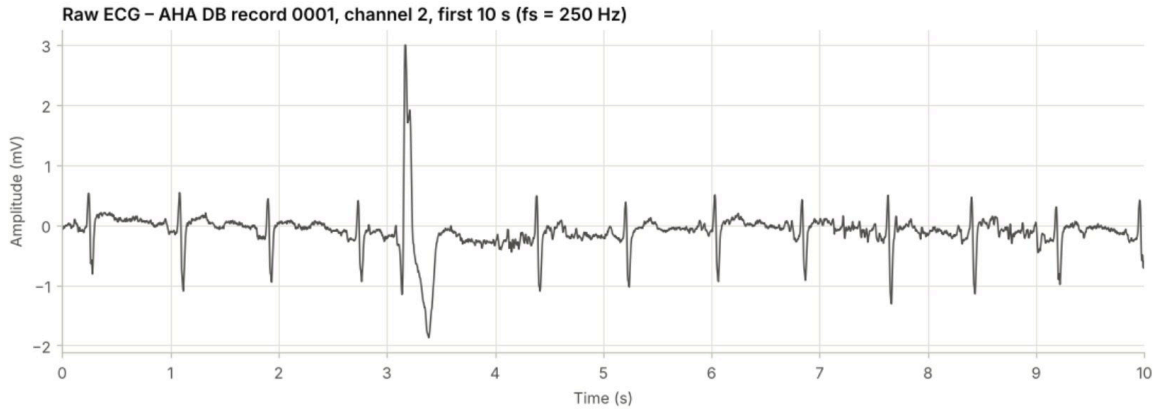
[4]:
```
# Step 1.2 — Second channel: 1-minute and 10-second signals | EaindrayMFU26
ecg60 = record.p_signal[:, CHANNEL]                # 1-minute ECG (mV), 15 000 samples
t60 = np.arange(ecg60.size) / FS                   # time axis (s)
ecg10, t10 = ecg60[:10 * FS], t60[:10 * FS]        # first 10 s, 2 500 samples
print(f"Channel used : column {CHANNEL + 1} of {record.n_sig} "
      f"({record.sig_name[CHANNEL]}, {record.units[CHANNEL]})")
print(f"Samples      : 1 min = {ecg60.size}, 10 s = {ecg10.size}")
pd.DataFrame({"first 10 s": pd.Series(ecg10).describe(),
              "full 60 s": pd.Series(ecg60).describe()}).round(3).T
```

```
Channel used : column 2 of 2 (ECG, mV)
Samples      : 1 min = 15000, 10 s = 2500
```

[4]:

	count	mean	std	min	25%	50%	75%	max
first 10 s	2500.0	-0.100	0.318	-1.872	-0.180	-0.068	0.021	2.998
full 60 s	15000.0	-0.147	0.271	-1.948	-0.208	-0.128	-0.048	3.188

[5]:
```
# Step 1.3 — Plot the raw ECG (first 10 s) | EaindrayMFU26
fig, ax = plt.subplots(figsize=(12, 3.6))
ax.plot(t10, ecg10, color=RAW, lw=1.0)
ax.set(title="Raw ECG — AHA DB record 0001, channel 2, first 10 s (fs = 250 Hz)",
       xlabel="Time (s)", ylabel="Amplitude (mV)", xlim=(0, 10))
ax.set_xticks(np.arange(0, 10.1, 1))
save(fig, "fig01_raw_ecg_10s.png"); plt.show()
```



1 Python 3 (ipykernel) | Idle Mode: Command Cell 5/36 Ln 1, Col 1 EaindrayMFU26_ECG_Mini_Project.ipynb 0

Screenshot 3. Step 1 in JupyterLab — notebook cells markdown, 1.1, 1.2, 1.3 with their outputs. [ss02_step1.1.jpg]

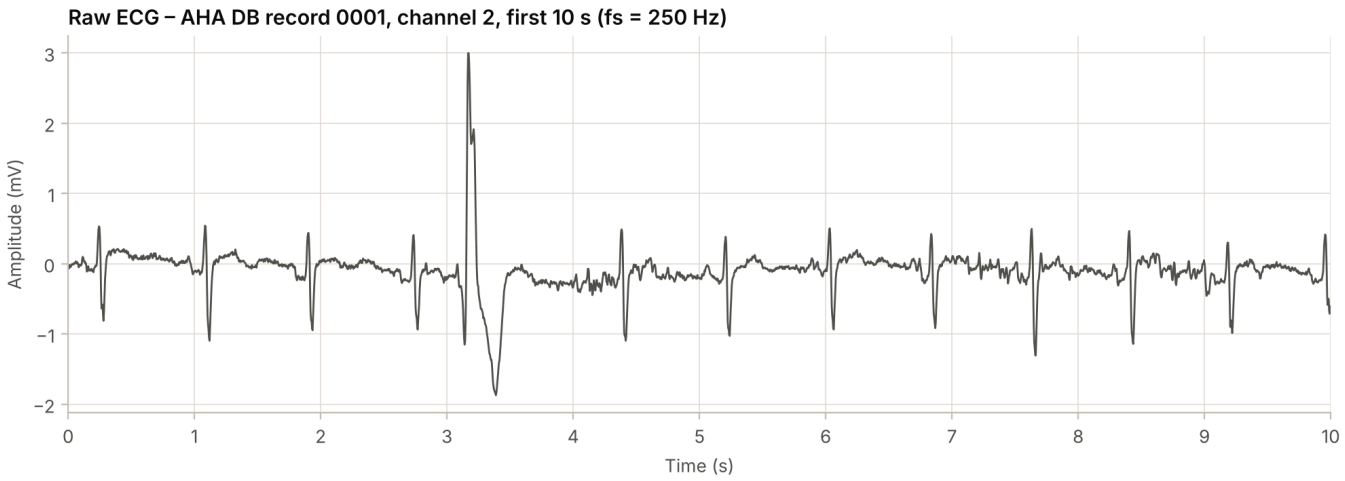


Figure 1. Raw ECG, AHA DB record 0001, channel 2, first 10 s (fs = 250 Hz).

Observation. Thirteen heartbeats are visible. In this lead the normal QRS complex is an *rS* pattern: a small positive R wave (≈ 0.4–0.55 mV) followed by a deep S wave (≈ −1 mV). At ≈ 3.2 s there is one much larger (≈ 3 mV), wider complex that occurs early and is followed by a long pause — an ectopic (PVC-like) beat, analysed in Steps 3–6. Small high-frequency ripples and a slow baseline drift are also visible.

Step 2 — ECG filtering with an FIR band-stop filter (first 10 s)

Design. Power-line interference is a narrow-band sinusoid, so a band-stop FIR filter centred on 50 Hz is used. It is designed with the window method (Kaiser window; order chosen by `scipy.signal.kaiserord` for ≥ 40 dB attenuation with a 2-Hz transition band) and applied forward–backward with `filtfilt`. A symmetric FIR filter has linear phase; forward–backward filtering cancels its 0.56-s group delay, so the P-QRS-T waves are not shifted.

Table 1. FIR band-stop filter specification.

Parameter	Value
Filter type	FIR band-stop (window method, Kaiser window)
Cut-off frequencies	48 Hz and 52 Hz (centre 50 Hz)
Number of taps (order)	281 taps (order 280), Kaiser β = 3.395
Attenuation at 50 Hz	38.2 dB per pass, 76.4 dB with filtfilt
Pass-band ripple (≤ 46 Hz, ≥ 54 Hz)	< 0.07 dB
Phase	zero phase (forward–backward filtering)

Code 6 · Step 2.1 — Design the FIR band-stop filter (50 Hz)

```
# Step 2.1 — Design the FIR band-stop filter (50 Hz) | EaindrayMFU26
F0, HALF_BW = 50.0, 2.0              # stop band centred on 50 Hz -> cut-offs 48 and 52 Hz
ATTEN_DB, TRANS_HZ = 40.0, 2.0       # Kaiser design: >= 40 dB attenuation, 2-Hz transition
numtaps, beta = signal.kaiserord(ATTEN_DB, TRANS_HZ / (FS / 2))
numtaps += numtaps % 2 == 0          # band-stop FIR needs an odd length (type-I linear phase)
h_bs = signal.firwin(numtaps, [F0 - HALF_BW, F0 + HALF_BW], window=("kaiser", beta),
                     pass_zero="bandstop", fs=FS)

f_h, H = signal.freqz(h_bs, worN=16384, fs=FS)
H_db = 20 * np.log10(np.abs(H) + 1e-12)
att50 = -H_db[np.argmin(np.abs(f_h - 50))]
ripple = np.abs(H_db[(f_h <= 46) | (f_h >= 54)]).max()
print(f"FIR band-stop: {numtaps} taps, Kaiser beta = {beta:.2f}, cut-offs 48 / 52 Hz")
print(f"Attenuation at 50 Hz: {att50:.1f} dB per pass -> {2 * att50:.1f} dB with filtfilt")
print(f"Max pass-band ripple (<= 46 Hz and >= 54 Hz): {ripple:.3f} dB")
print(f"Group delay {(numtaps - 1) / 2 / FS:.2f} s -> cancelled by filtfilt (zero phase)")

fig, (a1, a2) = plt.subplots(1, 2, figsize=(12, 3.6), gridspec_kw={"width_ratios": [2, 1]})
for ax, (lo, hi) in ((a1, (0, FS / 2)), (a2, (40, 60))):
    ax.axvspan(48, 52, color=GRID, alpha=0.6, lw=0)
    ax.plot(f_h, H_db, color=BLUE, lw=1.6)
    ax.set(xlim=(lo, hi), ylim=(-60, 5), xlabel="Frequency (Hz)")
a1.set(title="FIR band-stop filter — magnitude response", ylabel="Magnitude (dB)")
a2.set(title="Zoom 40–60 Hz")
a1.annotate("stop band 48–52 Hz", (52, -45), xytext=(62, -45), color=INK2,
            arrowprops=dict(arrowstyle="-", color=MUTED, lw=0.8), va="center")
fig.tight_layout(); save(fig, "fig02_fir_bandstop_response.png"); plt.show()
```

Code 7 · Step 2.2 — Apply the filter to the first 10 s and compare with the raw ECG

```
# Step 2.2 — Apply the filter to the first 10 s and compare with the raw ECG | EaindrayMFU26
def bandstop(x):
    """Zero-phase FIR band-stop filtering (forward-backward, no phase distortion)."""
    return signal.filtfilt(h_bs, 1.0, x)

ecg10_f = bandstop(ecg10)
removed = ecg10 - ecg10_f                       # what the filter took out

fig, axes = plt.subplots(3, 1, figsize=(12, 8), sharex=True,
                         gridspec_kw={"height_ratios": [1, 1, 0.7]})
axes[0].plot(t10, ecg10, color=RAW, lw=0.9)
axes[0].set(title="(a) Raw ECG", ylabel="mV")
axes[1].plot(t10, ecg10_f, color=BLUE, lw=0.9)
axes[1].set(title="(b) Filtered ECG — FIR band-stop 48–52 Hz", ylabel="mV")
axes[1].set_ylim(axes[0].get_ylim())
axes[2].plot(t10, removed * 1000, color=MUTED, lw=0.7)
```

```
axes[2].set(title="(c) Removed component = raw − filtered (note µV scale)",
            ylabel="µV", xlabel="Time (s)", xlim=(0, 10))
fig.tight_layout(); save(fig, "fig03_raw_vs_filtered_10s.png"); plt.show()

z = (t10 >= 0.9) & (t10 <= 1.6)                 # one heartbeat, overlaid
fig, ax = plt.subplots(figsize=(12, 3.4))
ax.plot(t10[z], ecg10[z], color=RAW, lw=1.6, label="Raw")
ax.plot(t10[z], ecg10_f[z], color=BLUE, lw=1.6, label="Filtered")
ax.set(title="Raw vs filtered ECG — one heartbeat (0.9–1.6 s)", xlabel="Time (s)",
       ylabel="Amplitude (mV)", xlim=(0.9, 1.6))
ax.legend(loc="lower right", ncol=2)
save(fig, "fig03b_raw_vs_filtered_zoom.png"); plt.show()
```

Code 8 · Step 2.3 — Quantify the effect of the filter (spectrum, statistics)

```
# Step 2.3 — Quantify the effect of the filter (spectrum, statistics) | EaindrayMFU26
f_p, P_raw = signal.welch(ecg10 - ecg10.mean(), fs=FS, nperseg=1024)
_, P_flt = signal.welch(ecg10_f - ecg10_f.mean(), fs=FS, nperseg=1024)
band = (f_p >= 49) & (f_p <= 51)
side = ((f_p > 44) & (f_p < 47)) | ((f_p > 53) & (f_p < 56))
rms = lambda v: np.sqrt(np.mean((v - v.mean()) ** 2))
pw_raw = trapezoid(P_raw[band], f_p[band]) * 1e6          # µV²
pw_flt = trapezoid(P_flt[band], f_p[band]) * 1e6
filter_stats = pd.Series({
    "50-Hz peak above neighbouring noise floor, raw (dB)":
        10 * np.log10(P_raw[band].max() / np.median(P_raw[side])),
    "Power 49-51 Hz, raw (µV²)": pw_raw,
    "Power 49-51 Hz, filtered (µV²)": pw_flt,
    "Power reduction 49-51 Hz (dB)": 10 * np.log10(pw_raw / pw_flt),
    "RMS raw ECG (µV)": rms(ecg10) * 1000,
    "RMS removed component (µV)": rms(removed) * 1000,
    "Removed / raw RMS (%)": 100 * rms(removed) / rms(ecg10),
    "Correlation raw vs filtered": np.corrcoef(ecg10, ecg10_f)[0, 1]})
filter_stats.to_csv(RES / "step2_filter_effect.csv", header=["value"])
print(filter_stats.map(lambda v: f"{v:.4g}").to_string())

fig, ax = plt.subplots(figsize=(12, 3.6))
ax.axvspan(48, 52, color=GRID, alpha=0.6, lw=0)
ax.plot(f_p, 10 * np.log10(P_raw), color=RAW, lw=1.2, label="Raw")
ax.plot(f_p, 10 * np.log10(P_flt), color=BLUE, lw=1.2, label="Filtered")
ax.annotate(f"49–51 Hz power reduced by {filter_stats.iloc[3]:.0f} dB", (52, -100),
            xytext=(57, -100), color=INK2, va="center",
            arrowprops=dict(arrowstyle="-", color=MUTED, lw=0.8))
ax.set(title="Power spectral density (Welch) — first 10 s", xlabel="Frequency (Hz)",
       ylabel="PSD (dB re 1 mV²/Hz)", xlim=(0, FS / 2))
ax.legend(loc="upper right", ncol=2)
save(fig, "fig04_psd_raw_vs_filtered.png"); plt.show()
```

File Edit View Run Kernel Tabs Settings Help

EaindrayMFU26_ECG_Mini

Code Python 3 (ipykernel)

Step 2 — ECG filtering with an FIR band-stop filter (first 10 s)

Design. Power-line interference is a narrow-band sinusoid, so a **band-stop (notch-type) FIR filter** centred on **50 Hz** is used: cut-off frequencies 48 Hz and 52 Hz, Kaiser window designed for ≥ 40 dB attenuation with a 2-Hz transition band (`scipy.signal.kaiserord` + `firwin`). An FIR filter with symmetric coefficients has **linear phase**, and it is applied forward–backward (`filtfilt`), so the output has **zero phase shift** – the P-QRS-T waves are not displaced in time.

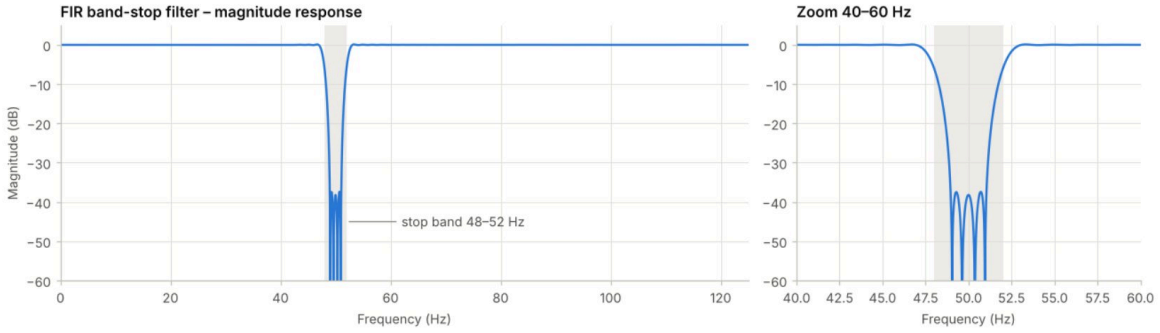
[6]:

```
# Step 2.1 — Design the FIR band-stop filter (50 Hz) | EaindrayMFU26
F0, HALF_BW = 50.0, 2.0              # stop band centred on 50 Hz -> cut-offs 48 and 52 Hz
ATTEN_DB, TRANS_HZ = 40.0, 2.0       # Kaiser design: >= 40 dB attenuation, 2-Hz transition
numtaps, beta = signal.kaiserord(ATTEN_DB, TRANS_HZ / (FS / 2))
numtaps += numtaps % 2 == 0          # band-stop FIR needs an odd length (type-I linear phase)
h_bs = signal.firwin(numtaps, [F0 - HALF_BW, F0 + HALF_BW], window=("kaiser", beta),
                     pass_zero="bandstop", fs=FS)

f_h, H = signal.freqz(h_bs, worN=16384, fs=FS)
H_db = 20 * np.log10(np.abs(H) + 1e-12)
att50 = -H_db[np.argmin(np.abs(f_h - 50))]
ripple = np.abs(H_db[(f_h <= 46) | (f_h >= 54)]).max()
print(f"FIR band-stop: {numtaps} taps, Kaiser beta = {beta:.2f}, cut-offs 48 / 52 Hz")
print(f"Attenuation at 50 Hz: {att50:.1f} dB per pass -> {2 * att50:.1f} dB with filtfilt")
print(f"Max pass-band ripple (<= 46 Hz and >= 54 Hz): {ripple:.3f} dB")
print(f"Group delay {(numtaps - 1) / 2 / FS:.2f} s -> cancelled by filtfilt (zero phase)")

fig, (a1, a2) = plt.subplots(1, 2, figsize=(12, 3.6), gridspec_kw={"width_ratios": [2, 1]})
for ax, (lo, hi) in ((a1, (0, FS / 2)), (a2, (40, 60))):
    ax.axvspan(48, 52, color=GRID, alpha=0.6, lw=0)
    ax.plot(f_h, H_db, color=BLUE, lw=1.6)
    ax.set(xlim=(lo, hi), ylim=(-60, 5), xlabel="Frequency (Hz)")
a1.set(title="FIR band-stop filter — magnitude response", ylabel="Magnitude (dB)")
a2.set(title="Zoom 40–60 Hz")
a1.annotate("stop band 48–52 Hz", (52, -45), xytext=(62, -45), color=INK2,
            arrowprops=dict(arrowstyle="-", color=MUTED, lw=0.8), va="center")
fig.tight_layout(); save(fig, "fig02_fir_bandstop_response.png"); plt.show()
```

```
FIR band-stop: 281 taps, Kaiser beta = 3.40, cut-offs 48 / 52 Hz
Attenuation at 50 Hz: 38.2 dB per pass -> 76.4 dB with filtfilt
Max pass-band ripple (<= 46 Hz and >= 54 Hz): 0.062 dB
Group delay 0.56 s -> cancelled by filtfilt (zero phase)
```



1 Python 3 (ipykernel) | Idle Mode: Command Cell 9/36 Ln 1, Col 1 EaindrayMFU26_ECG_Mini_Project.ipynb 0

Screenshot 4. Step 2 in JupyterLab — notebook cells markdown, 2.1 with their outputs. [ss03_step2.1.jpg]

File Edit View Run Kernel Tabs Settings Help

EaindrayMFU26_ECG_Mini

Code Python 3 (ipykernel)

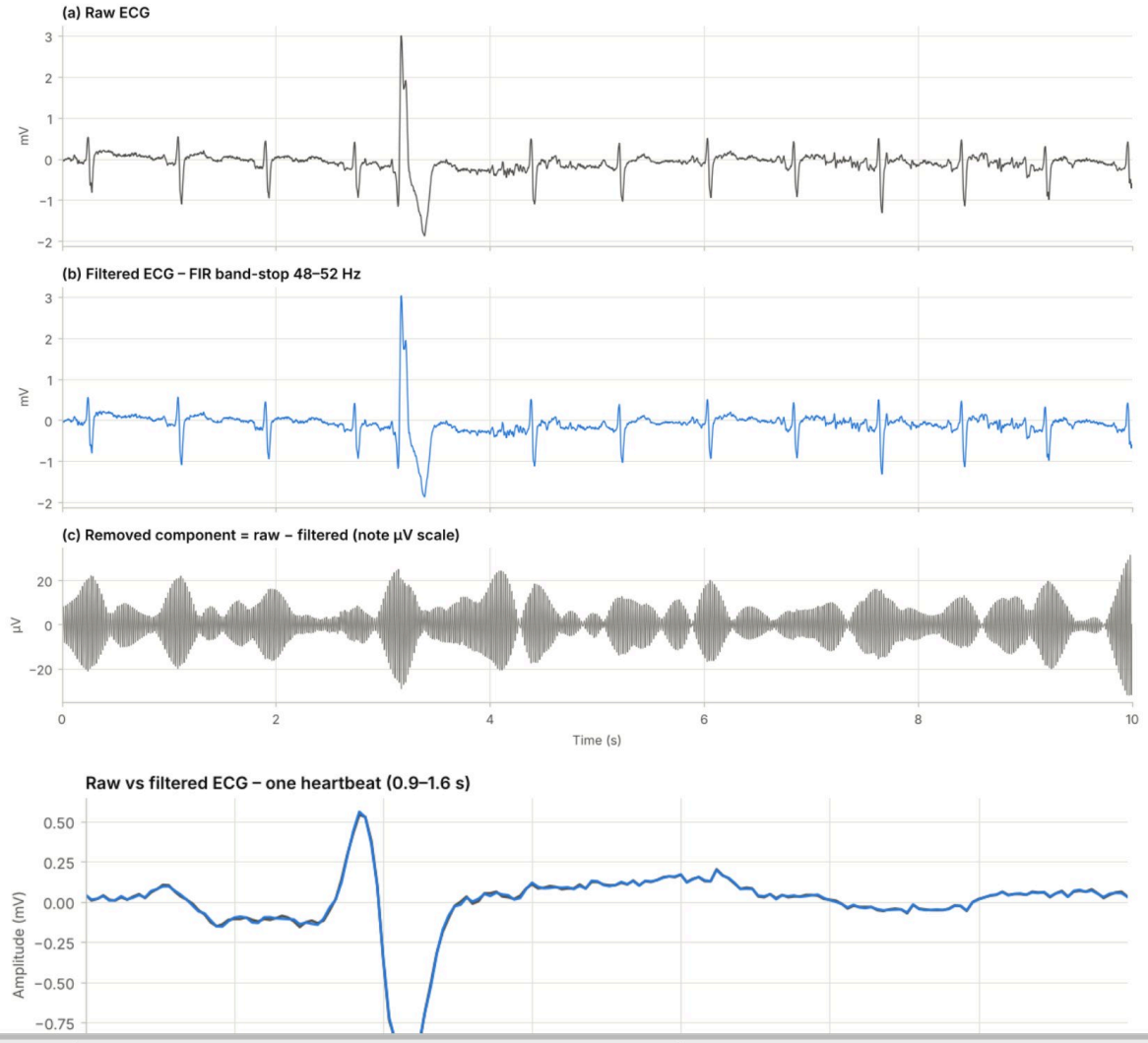
[7]:

```
# Step 2.2 — Apply the filter to the first 10 s and compare with the raw ECG | EaindrayMFU26
def bandstop(x):
    """Zero-phase FIR band-stop filtering (forward-backward, no phase distortion)."""
    return signal.filtfilt(h_bs, 1.0, x)

ecg10_f = bandstop(ecg10)
removed = ecg10 - ecg10_f                       # what the filter took out

fig, axes = plt.subplots(3, 1, figsize=(12, 8), sharex=True,
                         gridspec_kw={"height_ratios": [1, 1, 0.7]})
axes[0].plot(t10, ecg10, color=RAW, lw=0.9)
axes[0].set(title="(a) Raw ECG", ylabel="mV")
axes[1].plot(t10, ecg10_f, color=BLUE, lw=0.9)
axes[1].set(title="(b) Filtered ECG — FIR band-stop 48–52 Hz", ylabel="mV")
axes[1].set_ylim(axes[0].get_ylim())
axes[2].plot(t10, removed * 1000, color=MUTED, lw=0.7)
axes[2].set(title="(c) Removed component = raw − filtered (note µV scale)",
            ylabel="µV", xlabel="Time (s)", xlim=(0, 10))
fig.tight_layout(); save(fig, "fig03_raw_vs_filtered_10s.png"); plt.show()

z = (t10 >= 0.9) & (t10 <= 1.6)                 # one heartbeat, overlaid
fig, ax = plt.subplots(figsize=(12, 3.4))
ax.plot(t10[z], ecg10[z], color=RAW, lw=1.6, label="Raw")
ax.plot(t10[z], ecg10_f[z], color=BLUE, lw=1.6, label="Filtered")
ax.set(title="Raw vs filtered ECG — one heartbeat (0.9–1.6 s)", xlabel="Time (s)",
       ylabel="Amplitude (mV)", xlim=(0.9, 1.6))
ax.legend(loc="lower right", ncol=2)
save(fig, "fig03b_raw_vs_filtered_zoom.png"); plt.show()
```



1 Python 3 (ipykernel) | Idle Mode: Command Cell 10/36 Ln 1, Col 1 EaindrayMFU26_ECG_Mini_Project.ipynb 0

Screenshot 5 (part 1 of 2). Step 2 in JupyterLab — notebook cells 2.2 with their outputs. [ss04_step2.2_part1.jpg]



Screenshot 6 (part 2 of 2). Step 2 in JupyterLab — notebook cells 2.2 with their outputs. [ss05_step2.2_part2.jpg]

File Edit View Run Kernel Tabs Settings Help

EaindrayMFU26_ECG_Mini

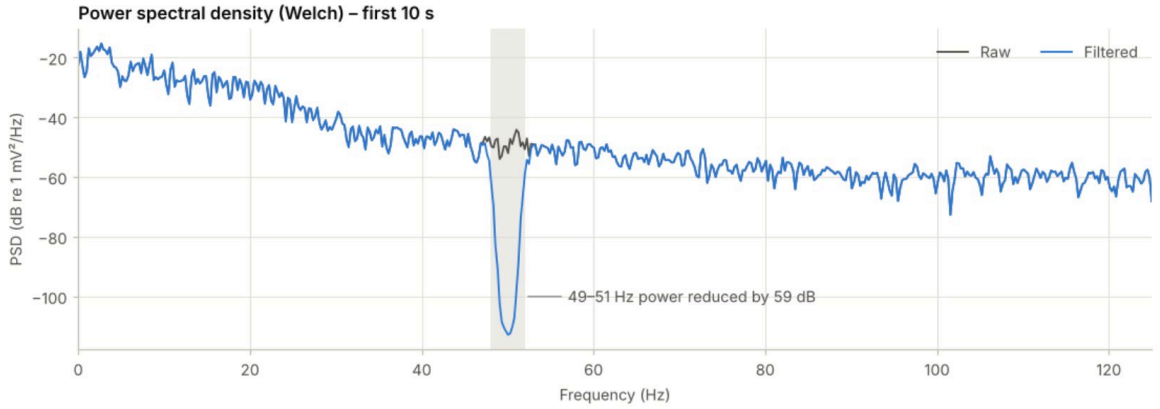
Code Python 3 (ipykernel)

[8]:

```
# Step 2.3 — Quantify the effect of the filter (spectrum, statistics) | EaindrayMFU26
f_p, P_raw = signal.welch(ecg10 - ecg10.mean(), fs=FS, nperseg=1024)
_, P_flt = signal.welch(ecg10_f - ecg10_f.mean(), fs=FS, nperseg=1024)
band = (f_p >= 49) & (f_p <= 51)
side = ((f_p > 44) & (f_p < 47)) | ((f_p > 53) & (f_p < 56))
rms = lambda v: np.sqrt(np.mean((v - v.mean()) ** 2))
pw_raw = trapezoid(P_raw[band], f_p[band]) * 1e6          # µV²
pw_flt = trapezoid(P_flt[band], f_p[band]) * 1e6
filter_stats = pd.Series({
    "50-Hz peak above neighbouring noise floor, raw (dB)":
        10 * np.log10(P_raw[band].max() / np.median(P_raw[side])),
    "Power 49-51 Hz, raw (µV²)": pw_raw,
    "Power 49-51 Hz, filtered (µV²)": pw_flt,
    "Power reduction 49-51 Hz (dB)": 10 * np.log10(pw_raw / pw_flt),
    "RMS raw ECG (µV)": rms(ecg10) * 1000,
    "RMS removed component (µV)": rms(removed) * 1000,
    "Removed / raw RMS (%)": 100 * rms(removed) / rms(ecg10),
    "Correlation raw vs filtered": np.corrcoef(ecg10, ecg10_f)[0, 1]})
filter_stats.to_csv(RES / "step2_filter_effect.csv", header=["value"])
print(filter_stats.map(lambda v: f"{v:.4g}").to_string())

fig, ax = plt.subplots(figsize=(12, 3.6))
ax.axvspan(48, 52, color=GRID, alpha=0.6, lw=0)
ax.plot(f_p, 10 * np.log10(P_raw), color=RAW, lw=1.2, label="Raw")
ax.plot(f_p, 10 * np.log10(P_flt), color=BLUE, lw=1.2, label="Filtered")
ax.annotate(f"49–51 Hz power reduced by {filter_stats.iloc[3]:.0f} dB", (52, -100),
            xytext=(57, -100), color=INK2, va="center",
            arrowprops=dict(arrowstyle="-", color=MUTED, lw=0.8))
ax.set(title="Power spectral density (Welch) — first 10 s", xlabel="Frequency (Hz)",
       ylabel="PSD (dB re 1 mV²/Hz)", xlim=(0, FS / 2))
ax.legend(loc="upper right", ncol=2)
save(fig, "fig04_psd_raw_vs_filtered.png"); plt.show()
```

```
50-Hz peak above neighbouring noise floor, raw (dB)        3.479
Power 49-51 Hz, raw (µV²)                                  18.97
Power 49-51 Hz, filtered (µV²)                          2.186e-05
Power reduction 49-51 Hz (dB)                              59.39
RMS raw ECG (µV)                                           318.2
RMS removed component (µV)                                 8.144
Removed / raw RMS (%)                                       2.56
Correlation raw vs filtered                               0.9997
```



Does the filtering visibly improve the signal? *Only marginally – because there is very little 50-Hz interference to remove.* In the raw spectrum the 50-Hz peak is only ≈ 3.5 dB above the neighbouring noise floor. The filter works exactly as designed – the power in the 49–51 Hz band drops by ≈ 59 dB (deep notch in the PSD) – but the removed component is tiny (RMS ≈ 8 µV, ≈ 2.6 % of the ECG RMS of ≈ 318 µV), so the raw and filtered traces are practically identical (correlation 0.9997). Importantly, the P-QRS-T morphology and the timing of the waves are preserved (zero-phase filtering, no distortion). The disturbances that are *visible* in the raw trace – slow baseline wander and broadband muscle noise – lie outside the 48–52 Hz stop band; removing them would need a high-pass (≈ 0.5 Hz) and a low-pass (≈ 40 Hz) filter. (The AHA recordings come from the USA, where the mains frequency is 60 Hz; setting `F0 = 60` re-targets the same design, and the spectrum shows no dominant 60-Hz line either.)

1 Python 3 (ipykernel) | Idle Mode: Command Cell 11/36 Ln 1, Col 1 EaindrayMFU26_ECG_Mini_Project.ipynb 0

Screenshot 7. Step 2 in JupyterLab — notebook cells 2.3, markdown with their outputs. [ss06_step2.3.jpg]

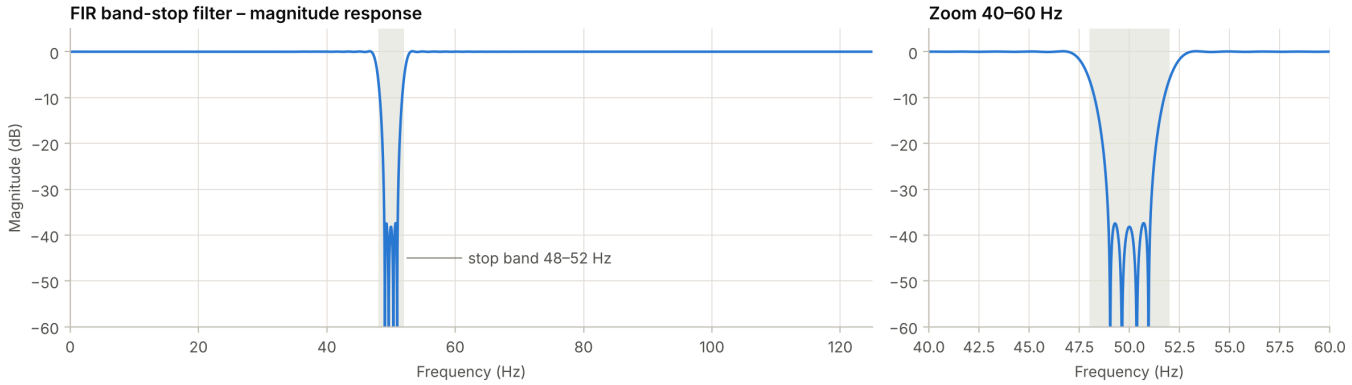


Figure 2. Magnitude response of the 281-tap FIR band-stop filter (single pass); grey band = stop band 48–52 Hz.

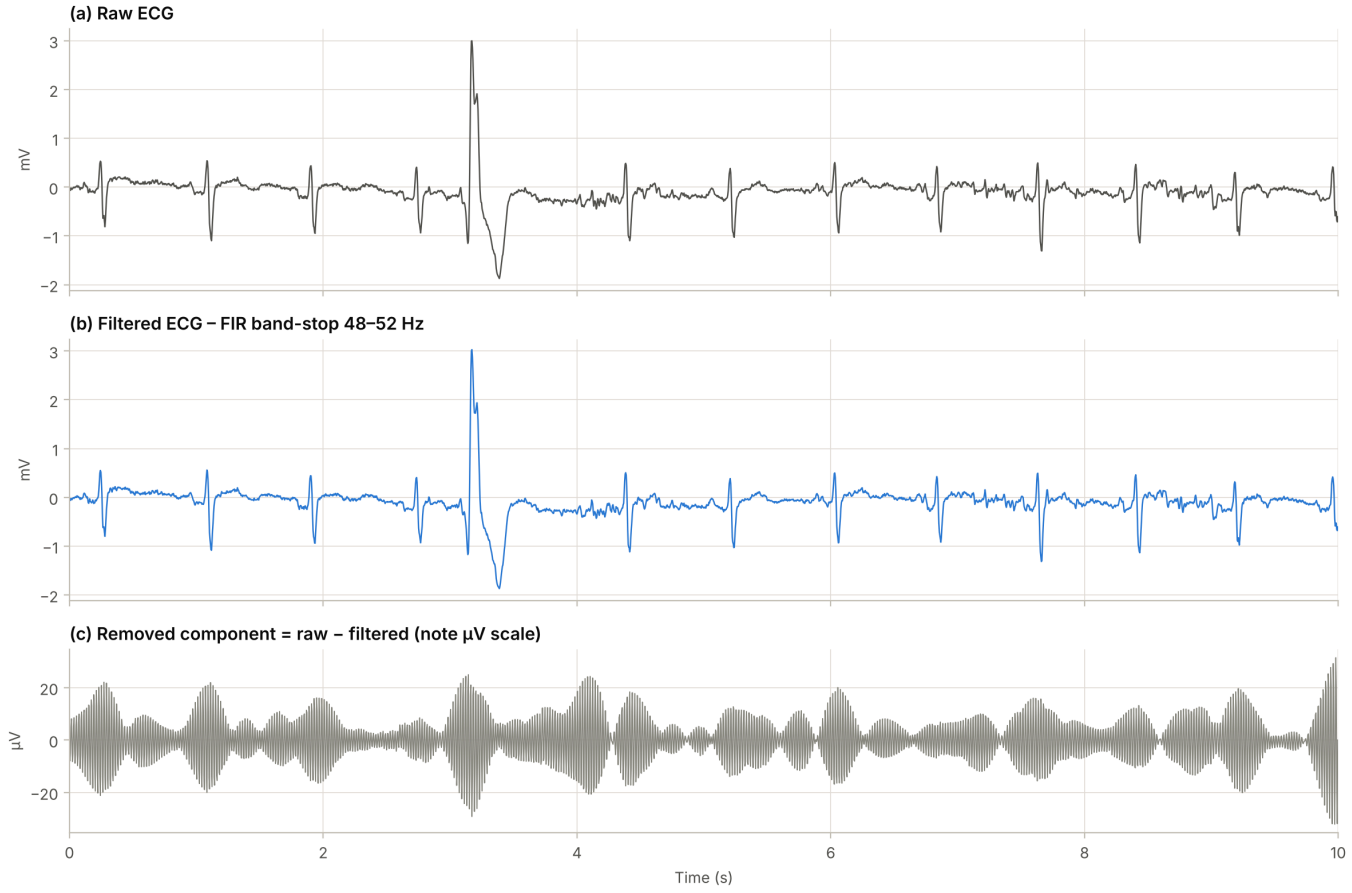


Figure 3. Comparison of (a) the raw ECG and (b) the filtered ECG over the first 10 s; (c) the component removed by the filter (raw − filtered), in µV.

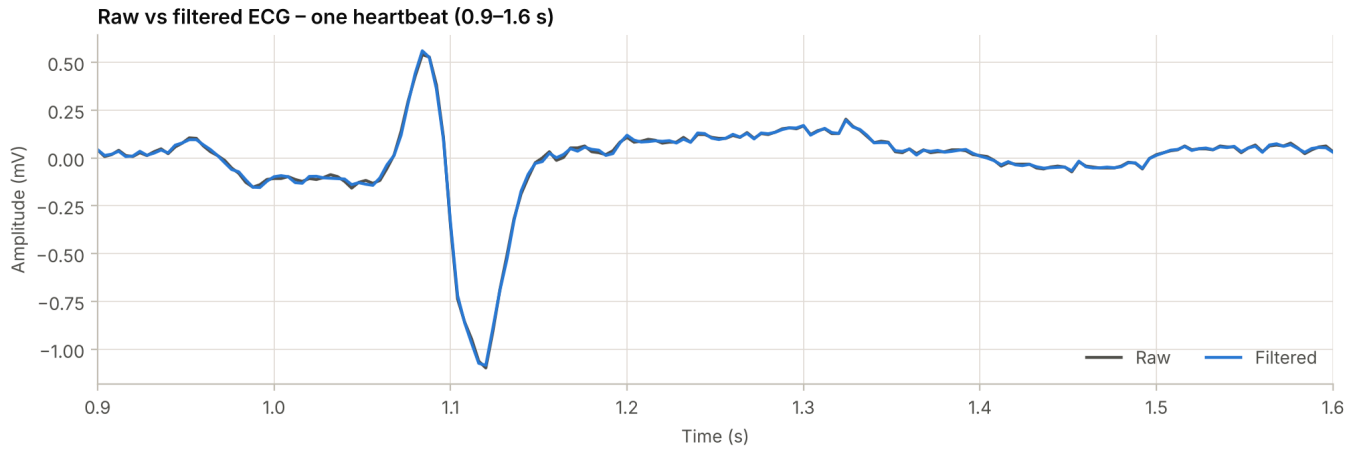


Figure 4. One heartbeat (0.9–1.6 s): raw and filtered ECG overlaid — the two traces are practically identical.

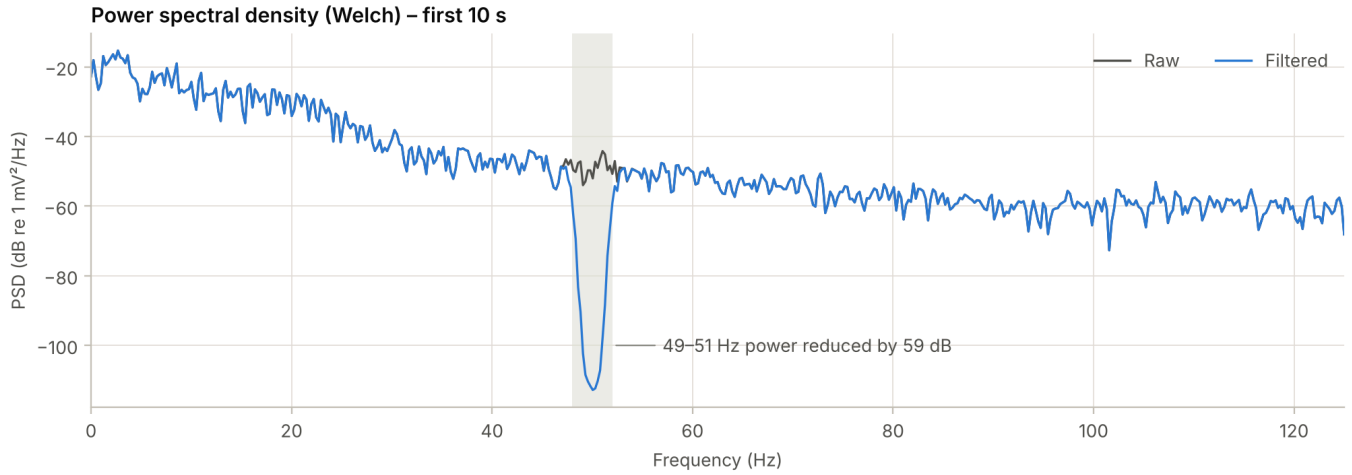


Figure 5. Power spectral density (Welch) of the raw and filtered ECG: the filter cuts a deep notch at 48–52 Hz and leaves the rest of the spectrum unchanged.

Table 2. Quantitative effect of the filter (first 10 s).

Quantity	Value
50-Hz peak above neighbouring noise floor, raw (dB)	3.48
Power 49-51 Hz, raw (µV²)	18.97
Power 49-51 Hz, filtered (µV²)	2.19e-05
Power reduction 49-51 Hz (dB)	59.39
RMS raw ECG (µV)	318.15
RMS removed component (µV)	8.14
Removed / raw RMS (%)	2.56
Correlation raw vs filtered	0.9997

Does the filtering visibly improve the signal quality? Only marginally, because this recording contains very little 50-Hz interference: the 50-Hz peak in the raw spectrum is only ≈ 3.5 dB above the neighbouring noise floor. The filter works as designed — the power in 49–51 Hz falls by ≈ 59 dB — but the removed component is tiny (RMS ≈ 8.1 µV, 2.6 % of the ECG RMS), so the raw and filtered traces are almost identical (correlation 0.9997) and the P-QRS-T morphology is preserved without any time shift. The disturbances that are *visible* — slow baseline wander and broadband muscle noise — lie outside the 48–52 Hz stop band and would need a high-pass (≈ 0.5 Hz) and low-pass (≈ 40 Hz) filter. (The AHA recordings come from the USA, where mains is 60 Hz; setting `F0 = 60` re-targets the same design, and the spectrum shows no dominant 60-Hz line either.)

Step 3 — R-peak detection (first 10 s)

Method — Pan–Tompkins algorithm on the band-stop-filtered ECG: (1) band-pass 5–15 Hz (2nd-order Butterworth, zero phase) to keep the QRS energy; (2) five-point derivative to emphasise the steep QRS slopes; (3) squaring; (4) 150-ms moving-window integration (MWI); (5) adaptive thresholds on the MWI peaks (signal and noise levels updated after every peak, 200-ms refractory period, T-wave rejection within 360 ms, search-back when a beat seems missed). Each R-peak is then located as the maximum of the filtered ECG near the detected QRS.

Two simple plausibility checks make the detector robust to a muscle-noise burst later in the recording (Step 4): (a) a detection that splits one normal RR interval into two is removed (the one that least resembles the median beat), and (b) a beat that matches the median beat template poorly is moved to the best match within ±150 ms. Each beat is labelled *Normal*, *Ectopic (PVC-like)* — premature (RR < 80 % of the median), followed by a compensatory pause (> 120 %), abnormal shape (template correlation < 0.5) and > 2× larger — or *Low-confidence (noise)*.

Code 9 · Step 3.1 — R-peak detector: Pan-Tompkins + plausibility checks

```python
# Step 3.1 — R-peak detector: Pan-Tompkins + plausibility checks | EaindrayMFU26
def pan_tompkins(ecg, fs, pad_s=0.5):
    """Pan & Tompkins (1985) QRS detector -> MWI peak indices and intermediate signals."""
    pad = int(pad_s * fs)
    x = np.pad(ecg, pad, mode="reflect")                        # mirror padding (edge beats)
    b, a = signal.butter(2, [5, 15], btype="bandpass", fs=fs)
    bp = signal.filtfilt(b, a, x)                               # 1) band-pass 5-15 Hz
    der = np.convolve(bp, np.array([1, 2, 0, -2, -1]) * fs / 8, mode="same")  # 2) derivative
    sq = der ** 2                                               # 3) squaring
    w = int(round(0.150 * fs))
    mwi = np.convolve(sq, np.ones(w) / w, mode="same")          # 4) 150-ms integration
    cand, _ = signal.find_peaks(mwi, distance=int(0.200 * fs))  # 200-ms refractory period
    spki, npki = 0.25 * mwi[:2 * fs].max(), 0.5 * mwi[:2 * fs].mean()   # 2-s learning phase
    thr, qrs, noise, thr_hist = npki + 0.25 * (spki - npki), [], [], []
    slope = lambda i: np.abs(der[max(0, i - int(0.075 * fs)):i + int(0.075 * fs)]).max()
    for c in cand:                                              # 5) adaptive thresholds
        t_wave = bool(qrs) and c - qrs[-1] < 0.36 * fs and slope(c) < 0.5 * slope(qrs[-1])
        if mwi[c] > thr and not t_wave:
            if len(qrs) >= 2 and c - qrs[-1] > 1.66 * np.median(np.diff(qrs[-9:])):
                missed = [n for n in noise if qrs[-1] + 0.2 * fs < n < c - 0.2 * fs
                          and mwi[n] > 0.5 * thr]               # search-back
                if missed:
                    qrs.append(max(missed, key=lambda n: mwi[n]))
            qrs.append(c); spki = 0.125 * mwi[c] + 0.875 * spki
        else:
            noise.append(c); npki = 0.125 * mwi[c] + 0.875 * npki
        thr = npki + 0.25 * (spki - npki); thr_hist.append((c - pad, thr))
    qrs = np.array(sorted(qrs)) - pad
    qrs = qrs[(qrs > -0.08 * fs) & (qrs < len(ecg) + 0.10 * fs)]
    stages = {k: v[pad:pad + len(ecg)]
              for k, v in dict(bp=bp, der=der, sq=sq, mwi=mwi).items()}
    stages["thr"] = np.array(thr_hist)
    return qrs, stages

def locate_r(ecg, qrs, fs):
    """R-peak = maximum of the filtered ECG from 100 ms before to 80 ms after each QRS."""
    r = []
    for c in qrs:
        lo, hi = max(0, c - int(0.10 * fs)), min(len(ecg), c + int(0.08 * fs))
        i = lo + int(np.argmax(ecg[lo:hi]))
        if not r or i - r[-1] > int(0.02 * fs):          # skip duplicates at the edges
            r.append(i)
    return np.array(r)

PRE, POST = int(0.10 * FS), int(0.15 * FS)                      # beat window around R

def detect_r_peaks(ecg, fs):
    """Pan-Tompkins -> R location -> plausibility checks -> beat labels."""
    qrs, stages = pan_tompkins(ecg, fs)
    r = list(locate_r(ecg, qrs, fs))
    seg = lambda i: ecg[i - PRE:i + POST]
    inside = lambda i: i - PRE >= 0 and i + POST <= len(ecg)
    tpl = np.median([seg(i) - np.median(seg(i)) for i in r if inside(i)], axis=0)
    corr = lambda i: np.corrcoef(seg(i), tpl)[0, 1] if inside(i) else np.nan
```

```
    while len(r) > 2:          # (a) extra detection splitting one normal RR interval
        med = np.median(np.diff(r))
        extra = [k for k in range(1, len(r) - 1) if r[k + 1] - r[k - 1] < 1.5 * med]
        if not extra:
            break
        r.pop(min(extra, key=lambda k: np.nan_to_num(corr(r[k]), nan=1.0)))
    r = np.array(r); moved = np.zeros(len(r), bool)
    for k, i in enumerate(r):  # (b) re-locate a poorly matching beat (best match +-150 ms)
        c0 = corr(i)
        if np.isnan(c0) or c0 >= 0.6:
            continue
        js = np.arange(max(PRE, i - int(0.15 * fs)), min(len(ecg) - POST, i + int(0.15 * fs)))
        cs = np.array([corr(j) for j in js])
        if cs.max() >= 0.6 and cs.max() - c0 >= 0.1:
            r[k], moved[k] = js[cs.argmax()], True
    cc = np.array([corr(i) for i in r])                          # (c) beat labels
    rr_pre, rr_post = np.r_[np.nan, np.diff(r)] / fs, np.r_[np.diff(r), np.nan] / fs
    amp = np.array([np.ptp(ecg[max(0, i - PRE):i + POST]) for i in r])
    med_rr = np.median(np.diff(r)) / fs
    ectopic = ((rr_pre < 0.8 * med_rr) & (rr_post > 1.2 * med_rr) & (cc < 0.5)
               & (amp / np.median(amp) > 2.0))
    label = np.where(ectopic, "Ectopic (PVC-like)",
                     np.where(cc < 0.6, "Low-confidence (noise)", "Normal"))
    beats = pd.DataFrame({"t_R (s)": r / fs, "R amplitude (mV)": ecg[r],
                          "template corr": cc.round(2), "re-located": moved,
                          "beat type": label})
    return r, beats, stages
```

Code 10 · Step 3.2 — Detect R-peaks in the first 10 s of the filtered ECG

```
# Step 3.2 — Detect R-peaks in the first 10 s of the filtered ECG | EaindrayMFU26
r10, beats10, st10 = detect_r_peaks(ecg10_f, FS)
beats10.index = pd.RangeIndex(1, len(r10) + 1, name="beat")
beats10.to_csv(RES / "step3_rpeaks_first10s.csv")
print(f"R-peaks detected in the first 10 s: {len(r10)}")
beats10.round(3)
```

Code 11 · Step 3.3 — Pan-Tompkins intermediate signals (first 10 s)

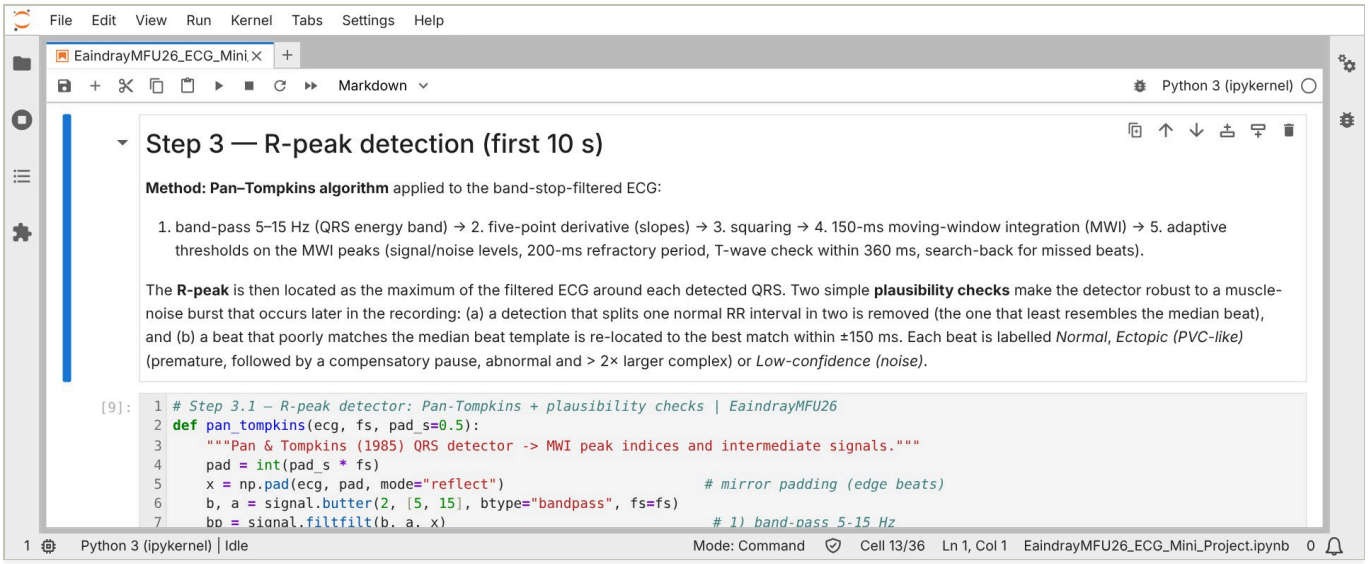
```
# Step 3.3 — Pan-Tompkins intermediate signals (first 10 s) | EaindrayMFU26
fig, axes = plt.subplots(4, 1, figsize=(12, 8.6), sharex=True)
for ax, key, title in zip(axes, ["bp", "der", "sq", "mwi"],
                          ["(1) Band-pass 5–15 Hz", "(2) Derivative", "(3) Squared",
                           "(4) Moving-window integration (150 ms) and adaptive threshold"]):
    ax.plot(t10, st10[key], color=BLUE, lw=0.9)
    ax.set_title(title, fontsize=10)
thr_t, thr_v = st10["thr"][:, 0] / FS, st10["thr"][:, 1]
axes[3].step(thr_t, thr_v, where="post", color=ORANGE, lw=1.2, label="adaptive threshold")
axes[3].legend(loc="upper right")
axes[3].set(xlabel="Time (s)", xlim=(0, 10))
fig.tight_layout(); save(fig, "fig05_pan_tompkins_stages_10s.png"); plt.show()
```

Code 12 · Step 3.4 — Overlay the detected R-peaks on the filtered ECG (first 10 s)

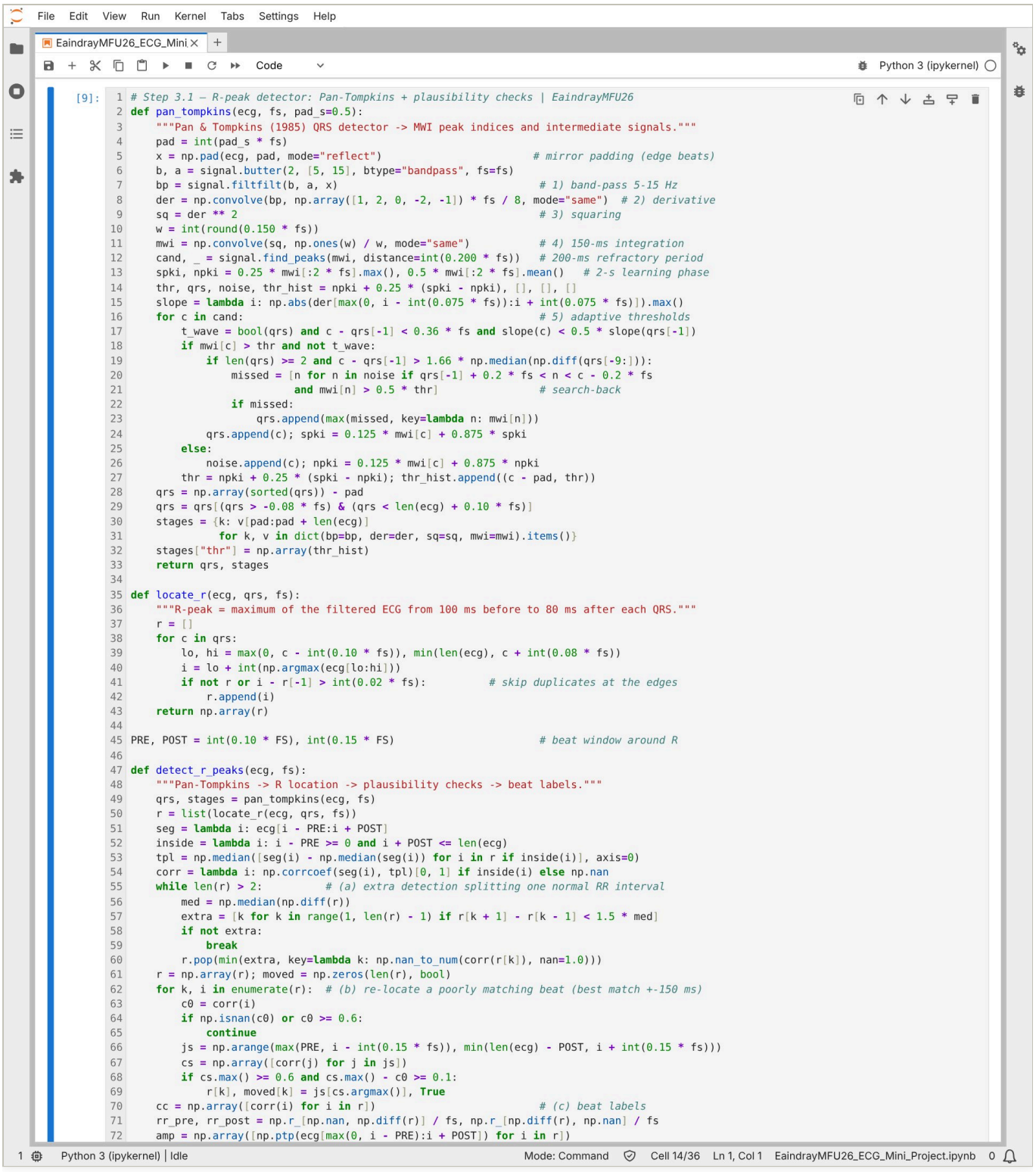
```
# Step 3.4 — Overlay the detected R-peaks on the filtered ECG (first 10 s) | EaindrayMFU26
def mark_beats(ax, t, ecg, r, labels, numbers=True):
    """R-peak markers: filled circle = normal, diamond = ectopic, ring = low-confidence."""
    styles = {"Normal": ("R-peak", dict(marker="o", mfc=ORANGE)),
              "Ectopic (PVC-like)": ("R-peak (ectopic, PVC-like)",
                                     dict(marker="D", mfc=ORANGE)),
              "Low-confidence (noise)": ("R-peak (low confidence, noise)",
                                         dict(marker="o", mfc="white"))}
    for lab, (name, sty) in styles.items():
        sel = r[labels == lab]
        if sel.size:
            ax.plot(t[sel], ecg[sel], ls="none", ms=7, mec=ORANGE, mew=1.5, label=name, **sty)
    if numbers:
        for k, i in enumerate(r, 1):
            ax.annotate(str(k), (t[i], ecg[i]), xytext=(0, 7), textcoords="offset points",
                        ha="center", fontsize=7.5, color=INK2)

fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(t10, ecg10_f, color=BLUE, lw=1.0, label="Filtered ECG")
```

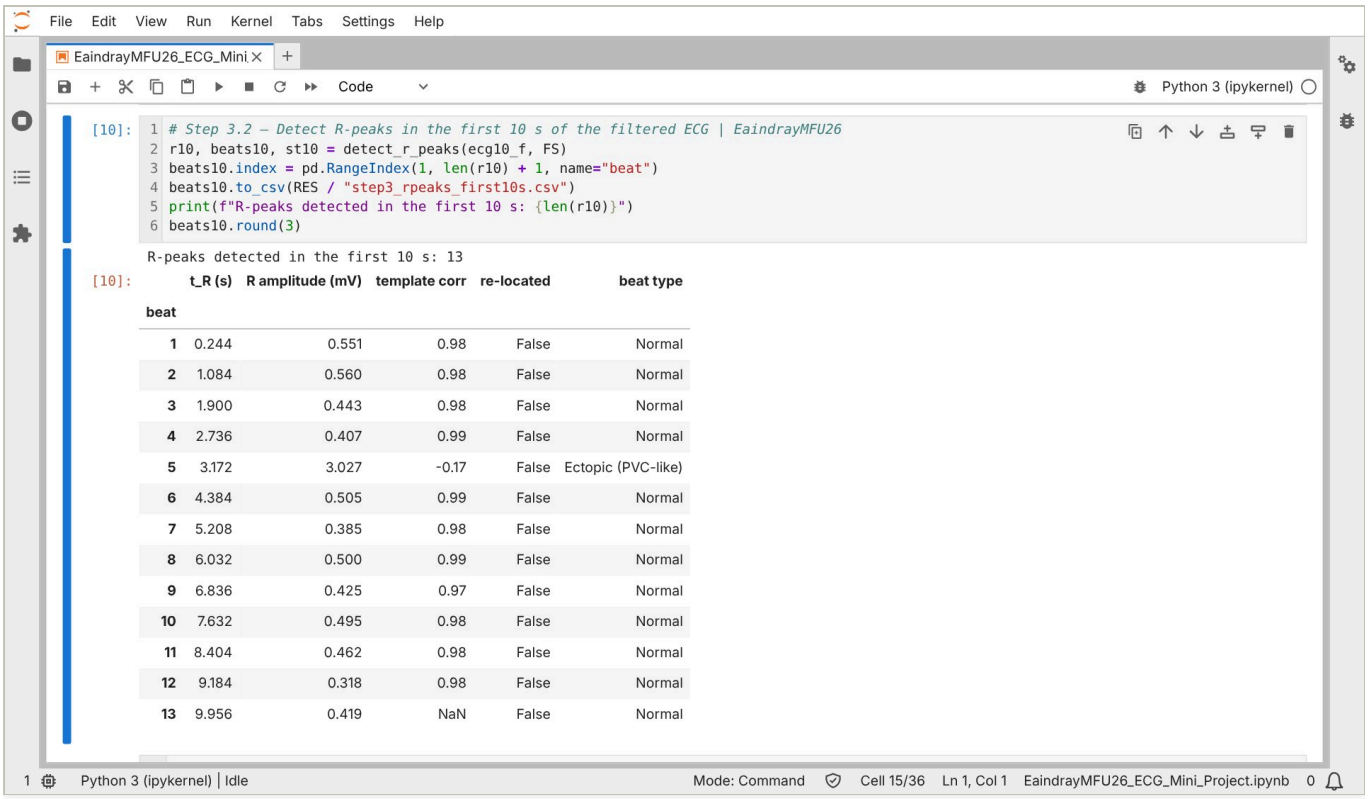
```
mark_beats(ax, t10, ecg10_f, r10, beats10["beat type"].to_numpy())
ax.set(title=f"Detected R-peaks on the filtered ECG — first 10 s ({len(r10)} beats)",
       xlabel="Time (s)", ylabel="Amplitude (mV)", xlim=(0, 10), ylim=(-2.2, 3.6))
ax.set_xticks(np.arange(0, 10.1, 1)); ax.legend(loc="upper right", ncol=3)
save(fig, "fig06_rpeaks_10s.png"); plt.show()
```



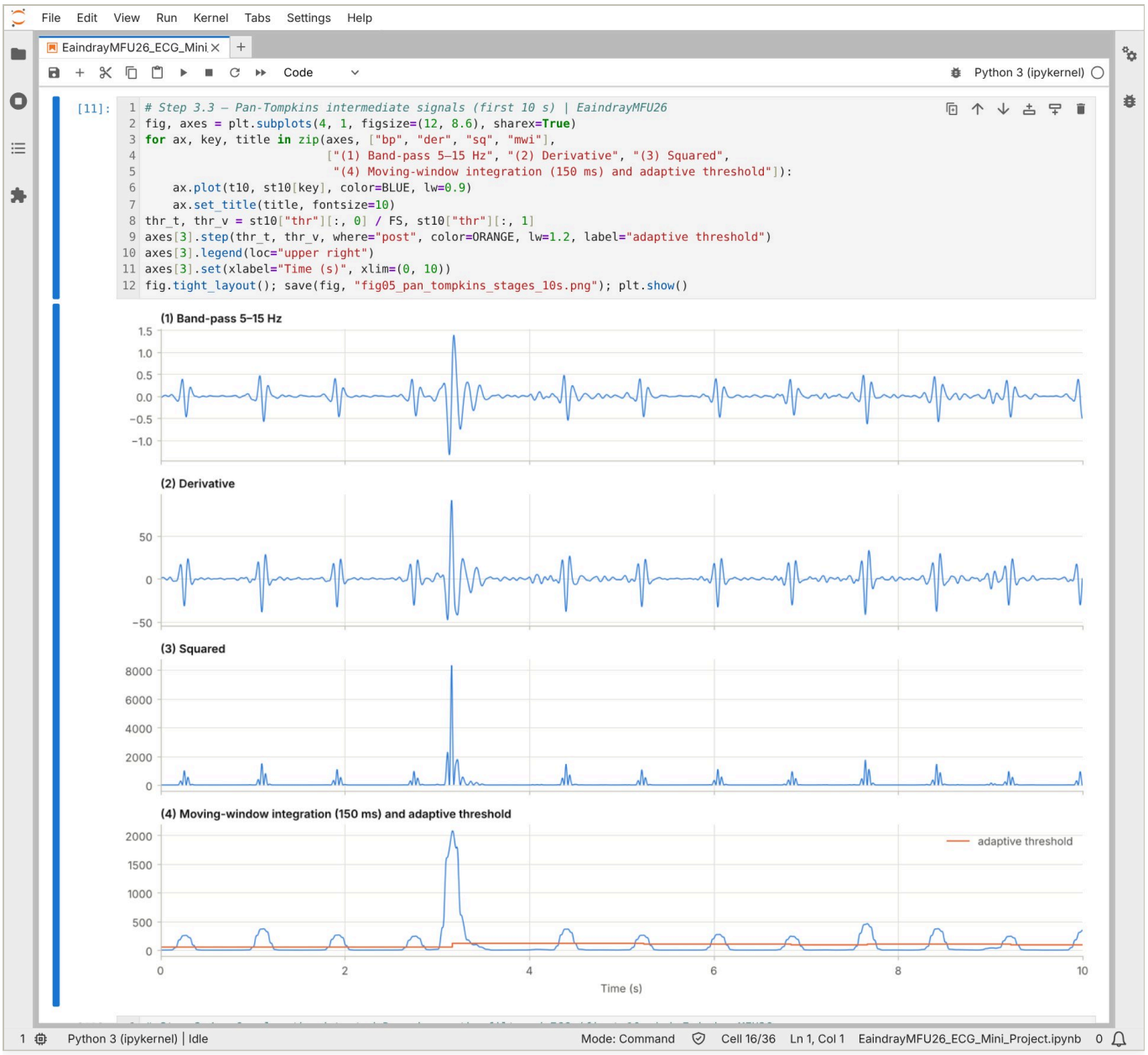
Screenshot 8. Step 3 in JupyterLab — notebook cells markdown. [ss07_stepnotes.jpg]



Screenshot 9. Step 3 in JupyterLab — notebook cells 3.1. [ss08_step3.1.jpg]



Screenshot 10. Step 3 in JupyterLab — notebook cells 3.2. [ss09_step3.2.jpg]



Screenshot 11. Step 3 in JupyterLab — notebook cells 3.3. [ss10_step3.3.jpg]

File Edit View Run Kernel Tabs Settings Help

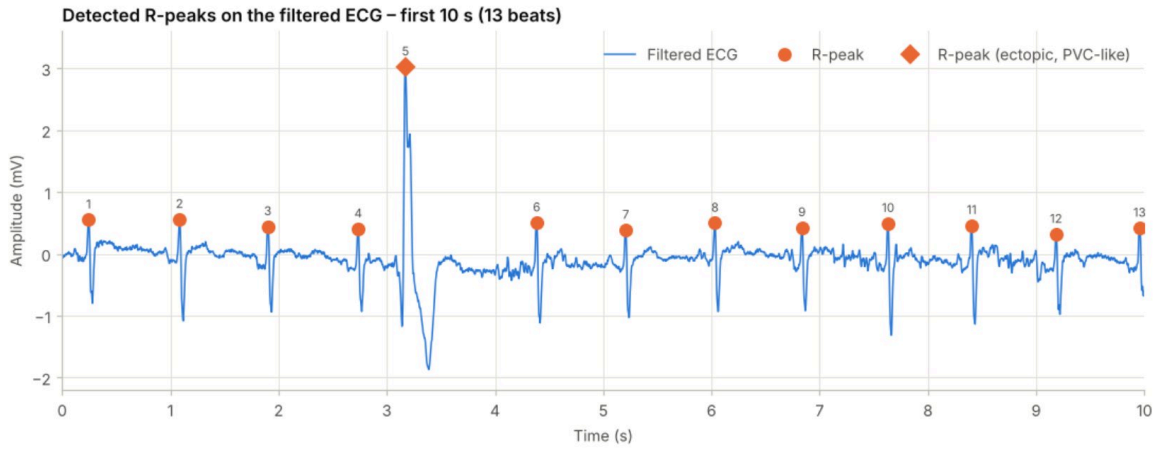
EaindrayMFU26_ECG_Mini

Code Python 3 (ipykernel)

[12]:

```
# Step 3.4 — Overlay the detected R-peaks on the filtered ECG (first 10 s) | EaindrayMFU26
def mark_beats(ax, t, ecg, r, labels, numbers=True):
    """R-peak markers: filled circle = normal, diamond = ectopic, ring = low-confidence."""
    styles = {"Normal": ("R-peak", dict(marker="o", mfc=ORANGE)),
              "Ectopic (PVC-like)": ("R-peak (ectopic, PVC-like)",
                                     dict(marker="D", mfc=ORANGE)),
              "Low-confidence (noise)": ("R-peak (low confidence, noise)",
                                         dict(marker="o", mfc="white"))}
    for lab, (name, sty) in styles.items():
        sel = r[labels == lab]
        if sel.size:
            ax.plot(t[sel], ecg[sel], ls="none", ms=7, mec=ORANGE, mew=1.5, label=name, **sty)
    if numbers:
        for k, i in enumerate(r, 1):
            ax.annotate(str(k), (t[i], ecg[i]), xytext=(0, 7), textcoords="offset points",
                        ha="center", fontsize=7.5, color=INK2)

fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(t10, ecg10_f, color=BLUE, lw=1.0, label="Filtered ECG")
mark_beats(ax, t10, ecg10_f, r10, beats10["beat type"].to_numpy())
ax.set(title=f"Detected R-peaks on the filtered ECG — first 10 s ({len(r10)} beats)",
       xlabel="Time (s)", ylabel="Amplitude (mV)", xlim=(0, 10), ylim=(-2.2, 3.6))
ax.set_xticks(np.arange(0, 10.1, 1)); ax.legend(loc="upper right", ncol=3)
save(fig, "fig06_rpeaks_10s.png"); plt.show()
```



1 Python 3 (ipykernel) | Idle Mode: Command Cell 17/36 Ln 1, Col 1 EaindrayMFU26_ECG_Mini_Project.ipynb 0

Screenshot 12. Step 3 in JupyterLab — notebook cells 3.4. [ss11_step3.4.jpg]

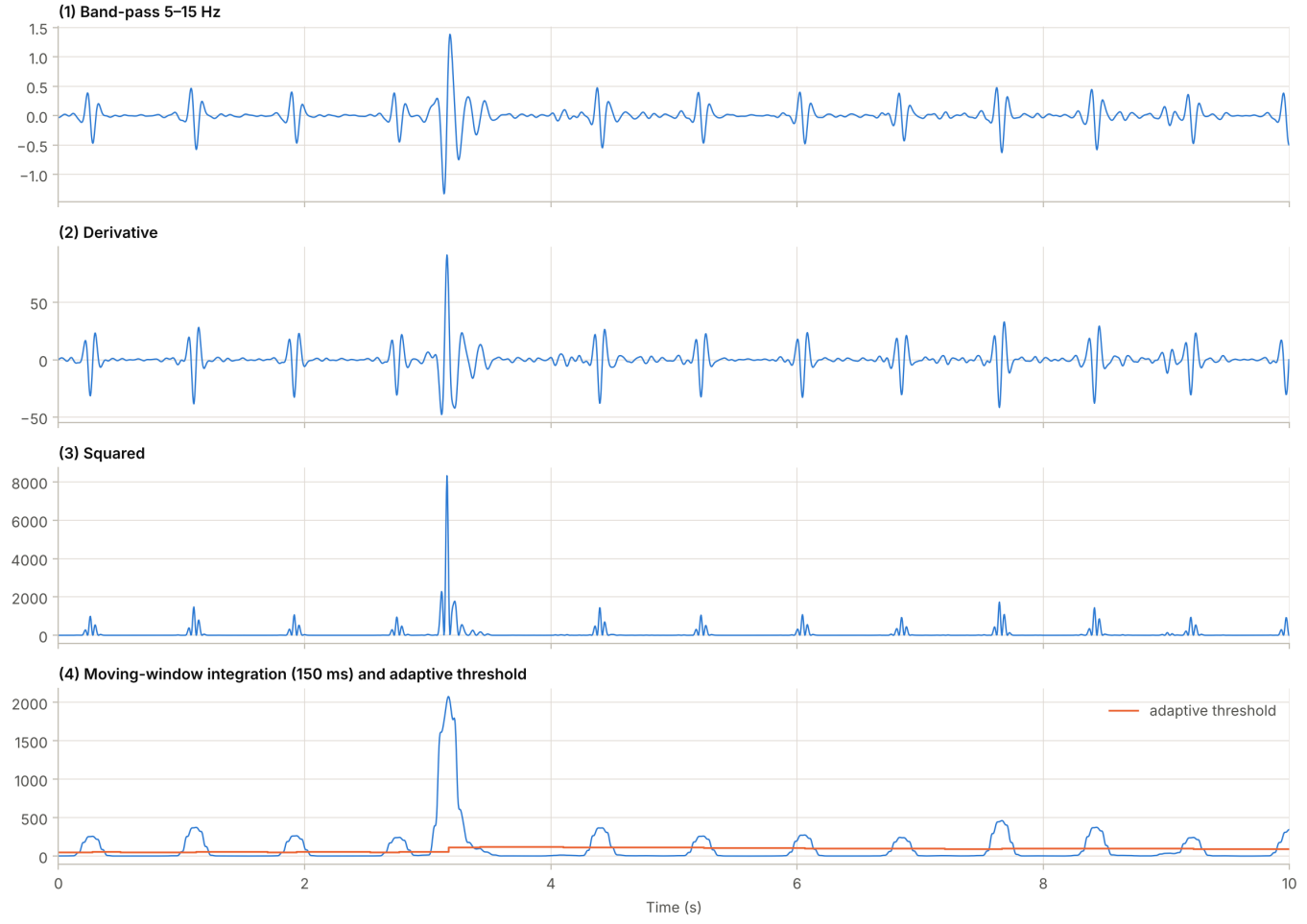


Figure 6. Pan–Tompkins intermediate signals for the first 10 s: band-pass, derivative, squared signal and moving-window integration with the adaptive threshold.

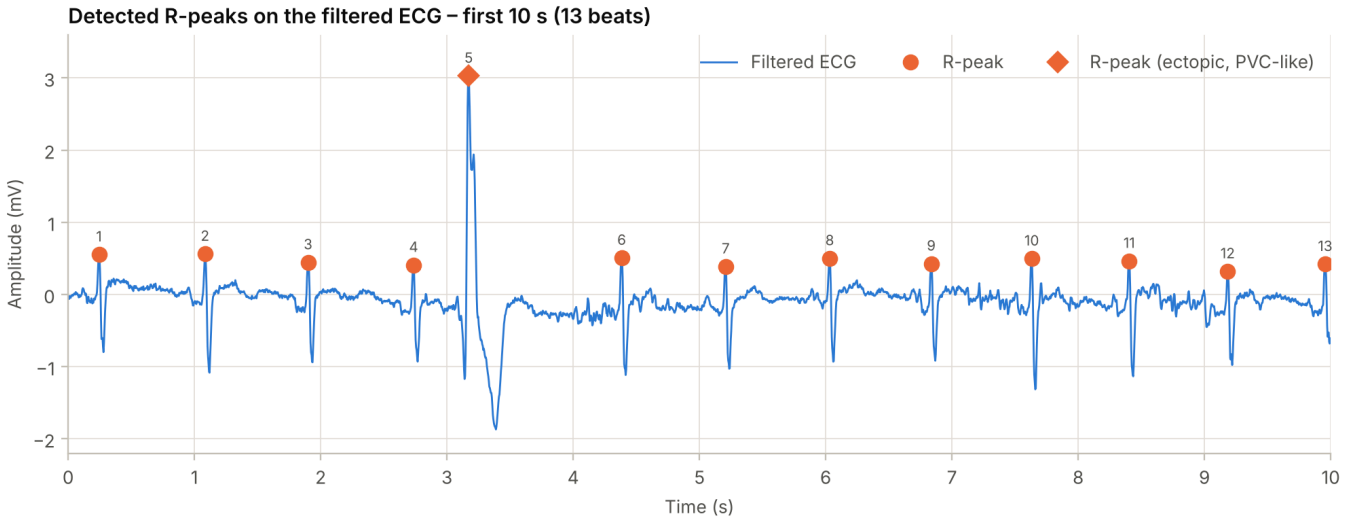


Figure 7. Filtered ECG with the 13 detected R-peaks (numbered); the diamond marks the ectopic (PVC-like) beat.

Table 3. R-peaks detected in the first 10 s (13 beats). Template correlation = similarity with the median beat (NaN for the last beat, whose window extends past 10 s).

beat	t_R (s)	R amplitude (mV)	template corr	re-located	beat type
1	0.244	0.551	0.98	False	Normal
2	1.084	0.560	0.98	False	Normal
3	1.900	0.443	0.98	False	Normal
4	2.736	0.407	0.99	False	Normal
5	3.172	3.027	-0.17	False	Ectopic (PVC-like)
6	4.384	0.505	0.99	False	Normal
7	5.208	0.385	0.98	False	Normal
8	6.032	0.500	0.99	False	Normal
9	6.836	0.425	0.97	False	Normal
10	7.632	0.495	0.98	False	Normal
11	8.404	0.462	0.98	False	Normal
12	9.184	0.318	0.98	False	Normal
13	9.956	0.419	–	False	Normal

Result. 13 R-peaks were detected in the first 10 s; every visible QRS complex is marked and there are no false detections. Beat 5 (t = 3.172 s) is the ectopic beat — its R amplitude is ≈ 3 mV and its shape is unlike the other beats (correlation −0.17).

Step 4 — Heart-rhythm analysis (full 1-minute recording)

Method. The same filter and detector are applied to the complete 60-s signal. The heart rate is computed from the R-peak count (HR = number of R-peaks / recording time in minutes) and, as a check, from the mean RR interval (HR = 60 / mean RR). Classification: < 60 bpm Bradycardia, 60–100 bpm Normal, > 100 bpm Tachycardia.

Code 13 · Step 4.1 — R-peaks, heart rate and rhythm classification (1 minute)

```python
# Step 4.1 — R-peaks, heart rate and rhythm classification (1 minute) | EaindrayMFU26
ecg60_f = bandstop(ecg60)
r60, beats60, st60 = detect_r_peaks(ecg60_f, FS)
types60 = beats60["beat type"].to_numpy()

def classify_hr(bpm):
    return "Bradycardia" if bpm < 60 else "Tachycardia" if bpm > 100 else "Normal"

duration_min = ecg60.size / FS / 60
n_peaks = len(r60)
rr_all = np.diff(r60) / FS
hr_count = n_peaks / duration_min                # beats counted in 1 minute
hr_meanrr = 60 / rr_all.mean()                   # 60 / mean RR interval
hr_summary = pd.Series({
    "Recording duration (s)": ecg60.size / FS,
    "Total R-peaks detected": n_peaks,
    "  normal beats": int((types60 == "Normal").sum()),
    "  ectopic (PVC-like) beats": int((types60 == "Ectopic (PVC-like)").sum()),
    "  low-confidence beats (noise burst)": int((types60 == "Low-confidence (noise)").sum()),
    "Heart rate from count (bpm)": round(hr_count, 1),
    "Heart rate from mean RR (bpm)": round(hr_meanrr, 1),
    "Classification": classify_hr(hr_count)})
hr_summary.to_csv(RES / "step4_heart_rate.csv", header=["value"])
print(hr_summary.to_string())
print(f"\nHeart Rate = {hr_count:.0f} bpm -> {classify_hr(hr_count)}")
```

Code 14 · Step 4.2 — The full minute with detected R-peaks (one row per 10 s)

```python
# Step 4.2 — The full minute with detected R-peaks (one row per 10 s) | EaindrayMFU26
fig, axes = plt.subplots(6, 1, figsize=(12, 13), sharey=True)
for k, ax in enumerate(axes):
    m = (t60 >= 10 * k) & (t60 < 10 * (k + 1))
    ax.plot(t60[m], ecg60_f[m], color=BLUE, lw=0.8, label="Filtered ECG")
    sel = (r60 >= 10 * k * FS) & (r60 < 10 * (k + 1) * FS)
    mark_beats(ax, t60, ecg60_f, r60[sel], types60[sel], numbers=False)
    ax.set_xlim(10 * k, 10 * (k + 1)); ax.set_ylim(-2.2, 3.6)
    ax.set_title(f"{10 * k}–{10 * (k + 1)} s  ·  {sel.sum()} R-peaks", fontsize=10)
    ax.set_ylabel("mV")
axes[-1].set_xlabel("Time (s)")
handles, labels = [], []
for ax in axes:                                  # one legend with every marker type
    for h, l in zip(*ax.get_legend_handles_labels()):
        if l not in labels:
            handles.append(h); labels.append(l)
fig.legend(handles, labels, loc="upper center", ncol=4, bbox_to_anchor=(0.5, 1.0))
fig.suptitle(f"Filtered ECG (channel 2), full minute — {n_peaks} R-peaks detected",
             x=0.01, ha="left", fontsize=12, fontweight="semibold", y=1.02)
fig.tight_layout(); save(fig, "fig07_rpeaks_full_minute.png"); plt.show()
```

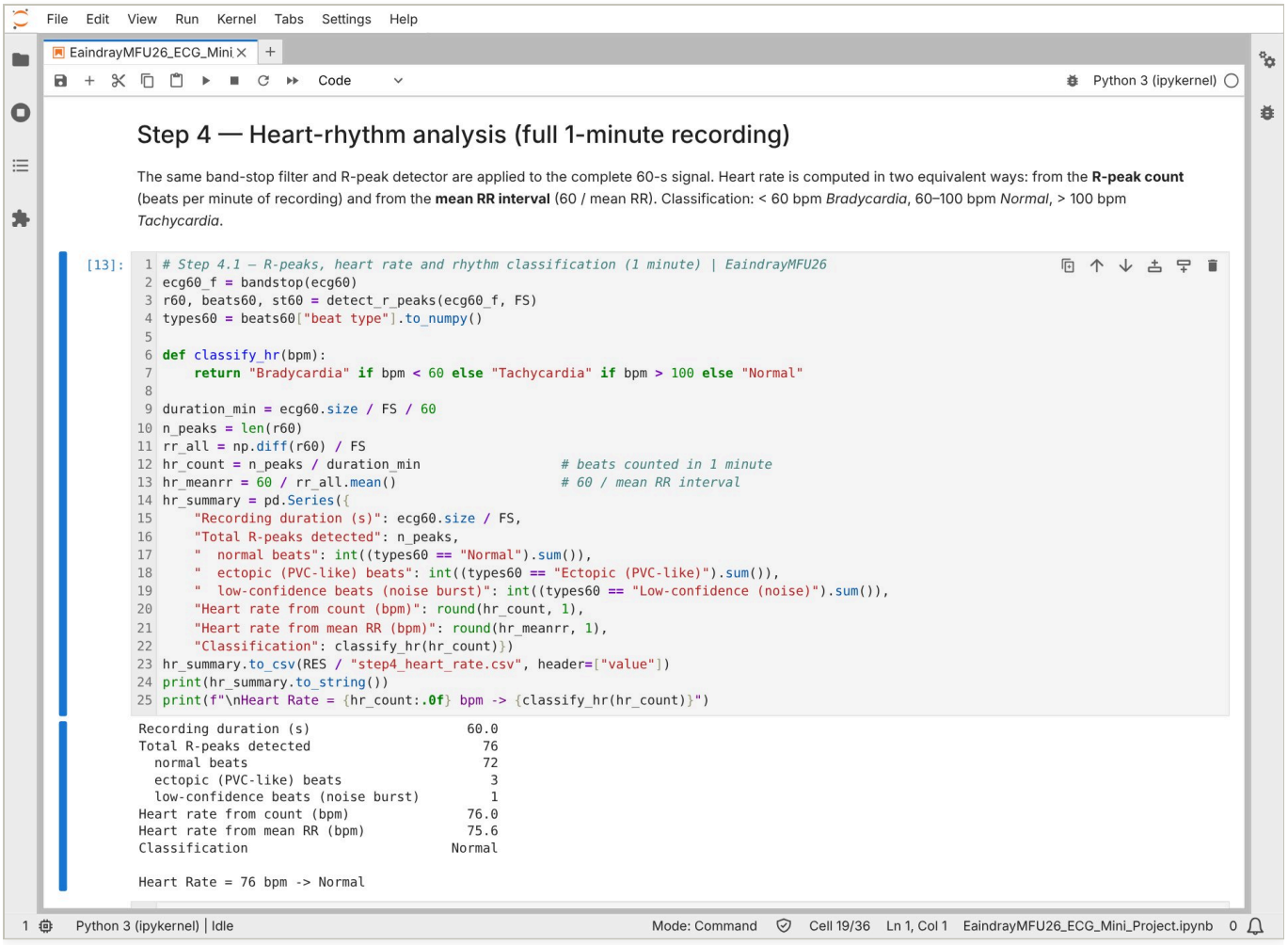
Code 15 · Step 4.3 — Instantaneous heart rate across the minute

```python
# Step 4.3 — Instantaneous heart rate across the minute | EaindrayMFU26
inst_hr = 60 / rr_all
t_hr = r60[1:] / FS
fig, ax = plt.subplots(figsize=(12, 3.8))
ax.axhspan(60, 100, color=GRID, alpha=0.45, lw=0)
ax.text(12, 97, "Normal range 60–100 bpm", color=INK2, va="top", fontsize=8.5)
ax.plot(t_hr, inst_hr, color=BLUE, lw=1.6, marker="o", ms=4, mec="white", mew=0.8)
ax.axhline(hr_count, color=INK2, lw=1.0)
ax.annotate(f"mean {hr_count:.0f} bpm", (60, hr_count), xytext=(4, 0),
            textcoords="offset points", va="center", fontsize=8.5, color=INK2)
for i in np.flatnonzero(types60 == "Ectopic (PVC-like)"):
    ax.annotate("ectopic beat", (r60[i] / FS, 60 / rr_all[i - 1]), xytext=(6, 0),
                textcoords="offset points", fontsize=8, color=INK2, va="center")
```

```
    ax.annotate("pause", (r60[i + 1] / FS, 60 / rr_all[i]), xytext=(6, -2),
                textcoords="offset points", fontsize=8, color=INK2, va="center")
k = int(np.flatnonzero(types60 == "Low-confidence (noise)")[0])
ax.annotate("noise burst:\nbeat position uncertain", (r60[k] / FS, 60 / rr_all[k - 1]),
            xytext=(-8, 6), textcoords="offset points", ha="right", fontsize=8, color=INK2)
ax.set(title="Instantaneous heart rate (60 / RR) — full minute", xlabel="Time (s)",
       ylabel="Heart rate (bpm)", xlim=(0, 60), ylim=(30, 160))
save(fig, "fig08_instantaneous_hr.png"); plt.show()
```

Code 16 · Step 4.4 — Independent check: QRS detection on channel 1 (wfdb XQRS)

```
# Step 4.4 — Independent check: QRS detection on channel 1 (wfdb XQRS) | EaindrayMFU26
# The first minute of record 0001 has no reference annotations (they start at 2.5 h),
# so the channel-2 detections are compared with an independent detector run on channel 1.
import logging; from wfdb import processing
logging.getLogger("wfdb").setLevel(logging.ERROR)
xq = processing.XQRS(sig=record.p_signal[:, 0], fs=FS); xq.detect(verbose=False)
ref = np.asarray(xq.qrs_inds)
dist = np.array([np.abs(ref - i).min() for i in r60]) / FS
print(f"Channel-1 reference beats: {ref.size} | channel-2 beats: {n_peaks}")
print(f"Channel-2 beats within 100 ms of a channel-1 beat: {(dist <= 0.1).sum()} / {n_peaks}")
print("Unmatched channel-2 beats:", [f"{t:.2f} s ({b})" for t, b, d in
      zip(r60 / FS, types60, dist) if d > 0.1])
```



Screenshot 13. Step 4 in JupyterLab — notebook cells markdown, 4.1. [ss12_step4.1.jpg]

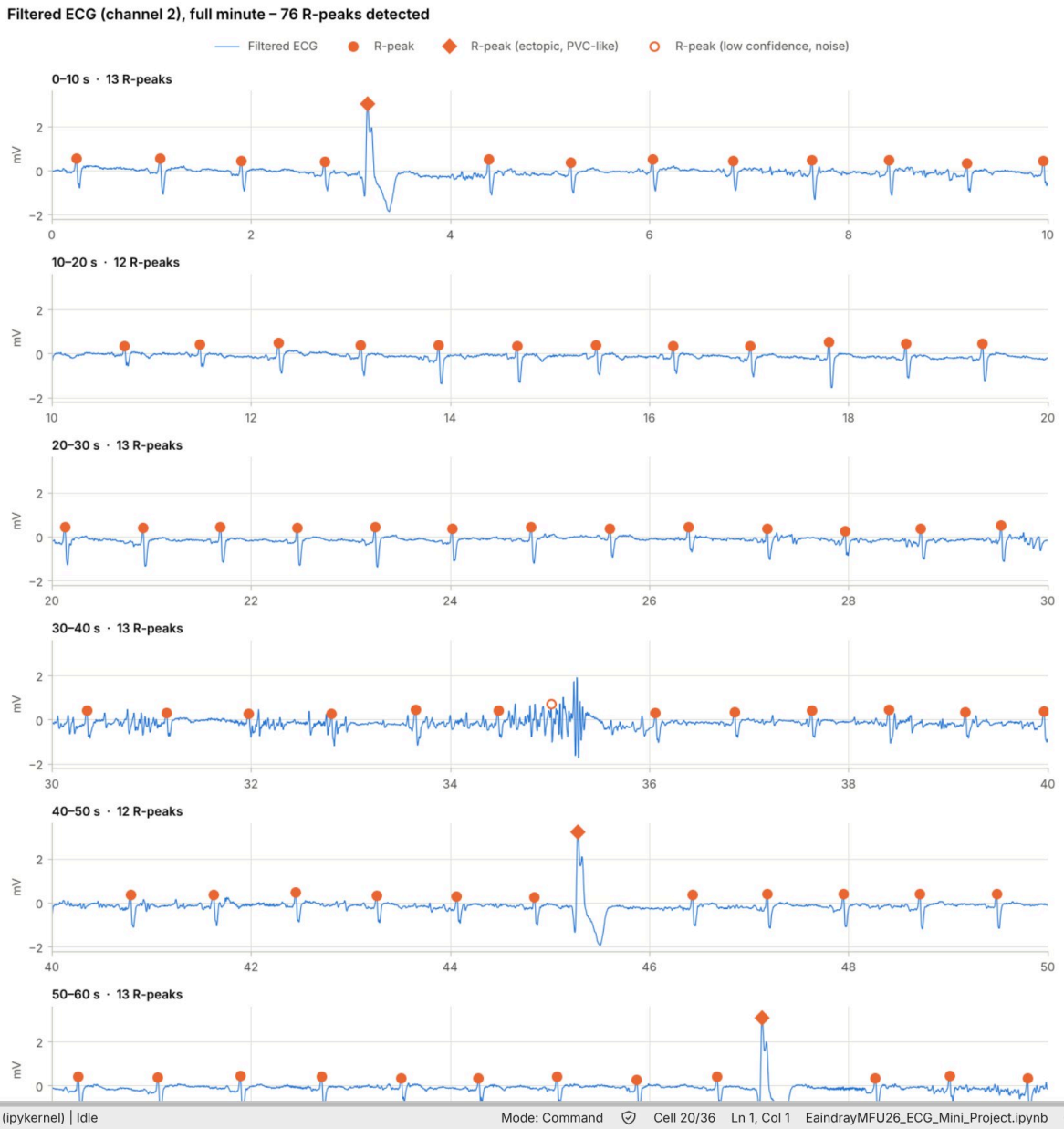
File Edit View Run Kernel Tabs Settings Help

EaindrayMFU26_ECG_Mini

Code Python 3 (ipykernel)

[14]:

```
# Step 4.2 — The full minute with detected R-peaks (one row per 10 s) | EaindrayMFU26
fig, axes = plt.subplots(6, 1, figsize=(12, 13), sharey=True)
for k, ax in enumerate(axes):
    m = (t60 >= 10 * k) & (t60 < 10 * (k + 1))
    ax.plot(t60[m], ecg60_f[m], color=BLUE, lw=0.8, label="Filtered ECG")
    sel = (r60 >= 10 * k * FS) & (r60 < 10 * (k + 1) * FS)
    mark_beats(ax, t60, ecg60_f, r60[sel], types60[sel], numbers=False)
    ax.set_xlim(10 * k, 10 * (k + 1)); ax.set_ylim(-2.2, 3.6)
    ax.set_title(f"{10 * k}–{10 * (k + 1)} s  ·  {sel.sum()} R-peaks", fontsize=10)
    ax.set_ylabel("mV")
axes[-1].set_xlabel("Time (s)")
handles, labels = [], []
for ax in axes:                                   # one legend with every marker type
    for h, l in zip(*ax.get_legend_handles_labels()):
        if l not in labels:
            handles.append(h); labels.append(l)
fig.legend(handles, labels, loc="upper center", ncol=4, bbox_to_anchor=(0.5, 1.0))
fig.suptitle(f"Filtered ECG (channel 2), full minute – {n_peaks} R-peaks detected",
             x=0.01, ha="left", fontsize=12, fontweight="semibold", y=1.02)
fig.tight_layout(); save(fig, "fig07_rpeaks_full_minute.png"); plt.show()
```



1 Python 3 (ipykernel) | Idle Mode: Command Cell 20/36 Ln 1, Col 1 EaindrayMFU26_ECG_Mini_Project.ipynb 0

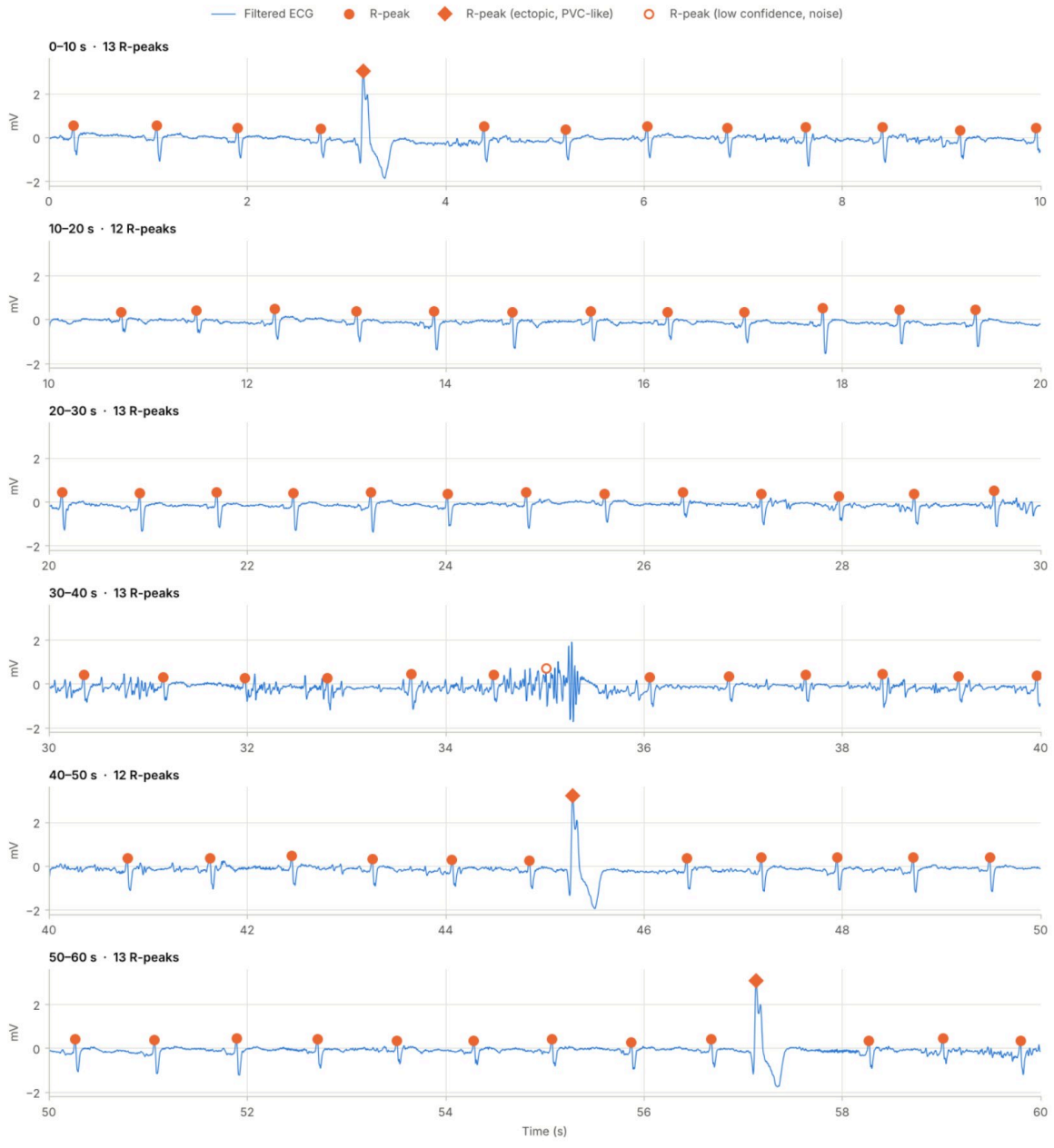
Screenshot 14 (part 1 of 2). Step 4 in JupyterLab — notebook cells 4.2. [ss13_step4.2_part1.jpg]

File Edit View Run Kernel Tabs Settings Help

EaindrayMFU26_ECG_Mini

Code Python 3 (ipykernel)

```
    sel = (r60 >= 10 * k * FS) & (r60 < 10 * (k + 1) * FS)
    mark_beats(ax, t60, ecg60_f, r60[sel], types60[sel], numbers=False)
    ax.set_xlim(10 * k, 10 * (k + 1)); ax.set_ylim(-2.2, 3.6)
    ax.set_title(f"{10 * k}–{10 * (k + 1)} s  ·  {sel.sum()} R-peaks", fontsize=10)
    ax.set_ylabel("mV")
axes[-1].set_xlabel("Time (s)")
handles, labels = [], []
for ax in axes:                                  # one legend with every marker type
    for h, l in zip(*ax.get_legend_handles_labels()):
        if l not in labels:
            handles.append(h); labels.append(l)
fig.legend(handles, labels, loc="upper center", ncol=4, bbox_to_anchor=(0.5, 1.0))
fig.suptitle(f"Filtered ECG (channel 2), full minute – {n_peaks} R-peaks detected",
             x=0.01, ha="left", fontsize=12, fontweight="semibold", y=1.02)
fig.tight_layout(); save(fig, "fig07_rpeaks_full_minute.png"); plt.show()
```



1 Python 3 (ipykernel) | Idle Mode: Command Cell 20/36 Ln 1, Col 1 EaindrayMFU26_ECG_Mini_Project.ipynb 0

Screenshot 15 (part 2 of 2). Step 4 in JupyterLab — notebook cells 4.2. [ss14_step4.2_part2.jpg]

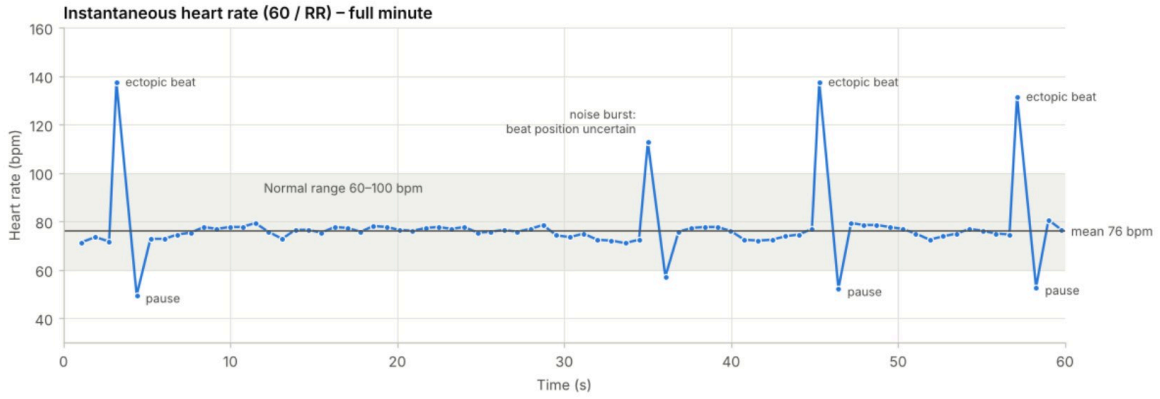
File Edit View Run Kernel Tabs Settings Help

EaindrayMFU26_ECG_Mini

Code Python 3 (ipykernel)

[15]:

```
# Step 4.3 — Instantaneous heart rate across the minute | EaindrayMFU26
inst_hr = 60 / rr_all
t_hr = r60[1:] / FS
fig, ax = plt.subplots(figsize=(12, 3.8))
ax.axhspan(60, 100, color=GRID, alpha=0.45, lw=0)
ax.text(12, 97, "Normal range 60–100 bpm", color=INK2, va="top", fontsize=8.5)
ax.plot(t_hr, inst_hr, color=BLUE, lw=1.6, marker="o", ms=4, mec="white", mew=0.8)
ax.axhline(hr_count, color=INK2, lw=1.0)
ax.annotate(f"mean {hr_count:.0f} bpm", (60, hr_count), xytext=(4, 0),
            textcoords="offset points", va="center", fontsize=8.5, color=INK2)
for i in np.flatnonzero(types60 == "Ectopic (PVC-like)"):
    ax.annotate("ectopic beat", (r60[i] / FS, 60 / rr_all[i - 1]), xytext=(6, 0),
                textcoords="offset points", fontsize=8, color=INK2, va="center")
    ax.annotate("pause", (r60[i + 1] / FS, 60 / rr_all[i]), xytext=(6, -2),
                textcoords="offset points", fontsize=8, color=INK2, va="center")
k = int(np.flatnonzero(types60 == "Low-confidence (noise)")[0])
ax.annotate("noise burst:\nbeat position uncertain", (r60[k] / FS, 60 / rr_all[k - 1]),
            xytext=(-8, 6), textcoords="offset points", ha="right", fontsize=8, color=INK2)
ax.set(title="Instantaneous heart rate (60 / RR) — full minute", xlabel="Time (s)",
       ylabel="Heart rate (bpm)", xlim=(0, 60), ylim=(30, 160))
save(fig, "fig08_instantaneous_hr.png"); plt.show()
```



[16]:

```
# Step 4.4 — Independent check: QRS detection on channel 1 (wfdb XQRS) | EaindrayMFU26
# The first minute of record 0001 has no reference annotations (they start at 2.5 h),
# so the channel-2 detections are compared with an independent detector run on channel 1.
import logging; from wfdb import processing
logging.getLogger("wfdb").setLevel(logging.ERROR)
xq = processing.XQRS(sig=record.p_signal[:, 0], fs=FS); xq.detect(verbose=False)
ref = np.asarray(xq.qrs_inds)
dist = np.array([np.abs(ref - i).min() for i in r60]) / FS
print(f"Channel-1 reference beats: {ref.size} | channel-2 beats: {n_peaks}")
print(f"Channel-2 beats within 100 ms of a channel-1 beat: {(dist <= 0.1).sum()} / {n_peaks}")
print("Unmatched channel-2 beats:", [f"{t:.2f} s ({b})" for t, b, d in
      zip(r60 / FS, types60, dist) if d > 0.1])
```

```
Channel-1 reference beats: 76 | channel-2 beats: 76
Channel-2 beats within 100 ms of a channel-1 beat: 75 / 76
Unmatched channel-2 beats: ['35.01 s (Low-confidence (noise))']
```

1 Python 3 (ipykernel) | Idle Mode: Command Cell 21/36 Ln 1, Col 1 EaindrayMFU26_ECG_Mini_Project.ipynb 0

Screenshot 16. Step 4 in JupyterLab — notebook cells 4.3, 4.4. [ss15_step4.3.jpg]

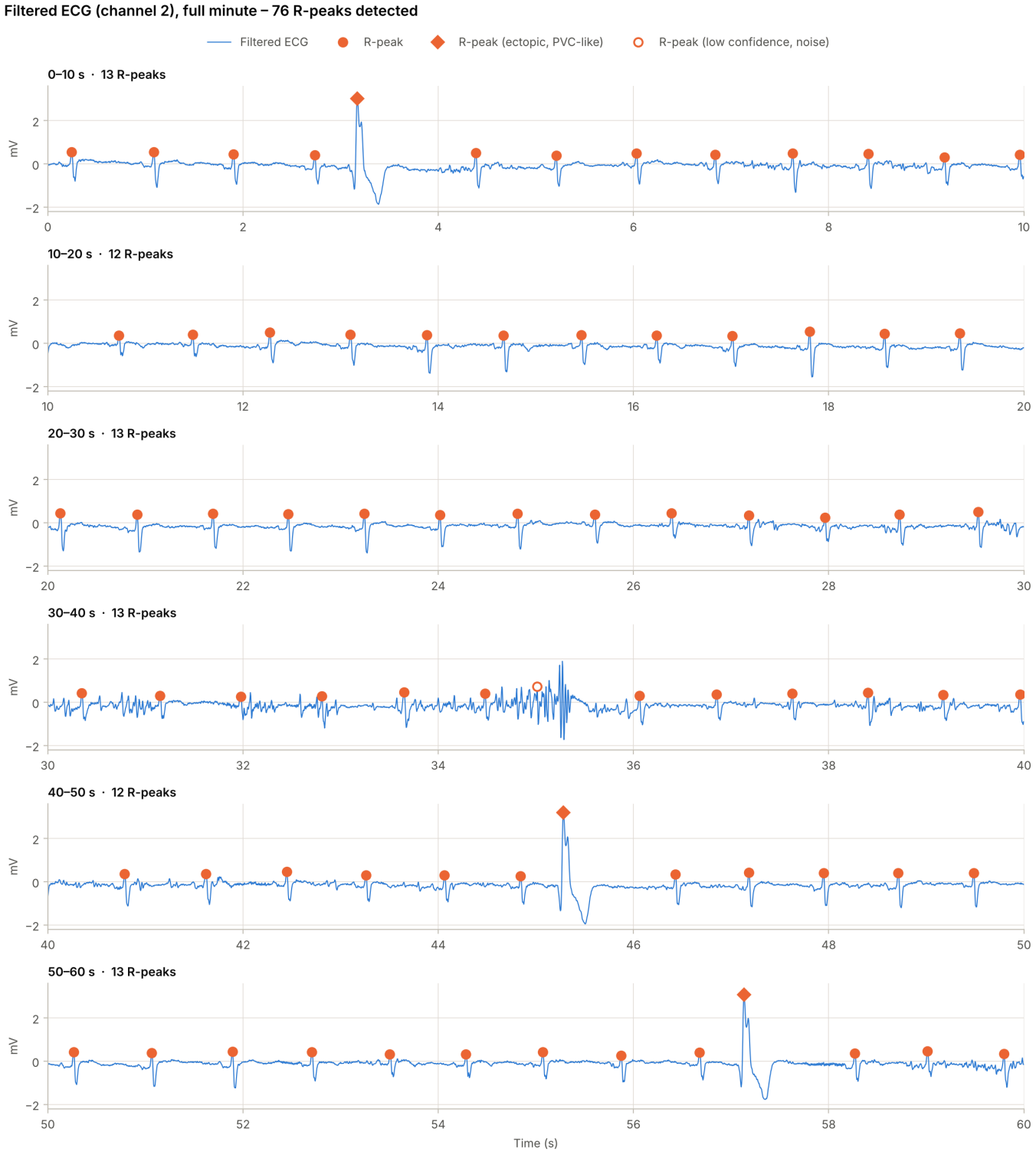


Figure 8. The full minute of filtered ECG (one row per 10 s) with all 76 detected R-peaks: filled circles = normal beats, diamonds = ectopic beats, ring = low-confidence beat inside the muscle-noise burst (30–36 s).

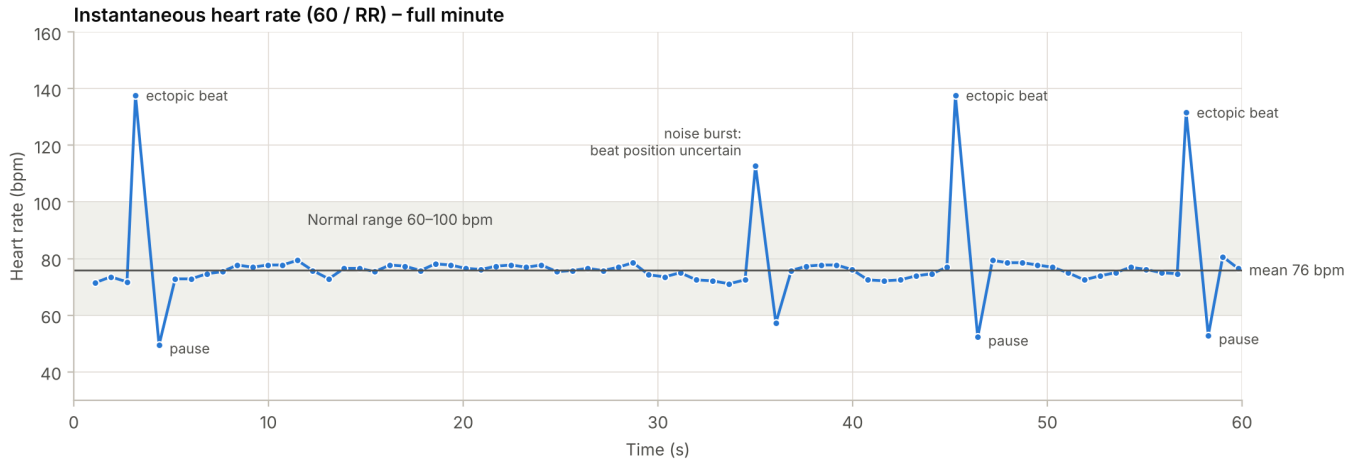


Figure 9. Instantaneous heart rate (60 / RR) over the minute; shaded band = normal range 60–100 bpm. The spikes and dips are the three ectopic beats and their compensatory pauses; the blip at ≈ 35 s comes from the uncertain beat position inside the noise burst.

Table 4. Heart-rate analysis of the 1-minute recording.

Quantity	Value
Recording duration (s)	60
Total R-peaks detected	76
normal beats	72
ectopic (PVC-like) beats	3
low-confidence beats (noise burst)	1
Heart rate from count (bpm)	76.0
Heart rate from mean RR (bpm)	75.6
Classification	Normal

> **Result.** Total number of detected R-peaks = **76** in 60 s → **Heart Rate = 76 bpm → Normal** (60–100 bpm). The mean-RR method gives 75.6 bpm, the same class. The rhythm is regular except for three isolated ectopic (PVC-like) beats at 3.2, 45.3 and 57.1 s, which do not change the classification.

Independent check. The first minute of record 0001 has no reference beat annotations (the annotations start at 2.5 h), so the channel-2 detections were compared with an independent detector (wfdb XQRS) run on channel 1, which is cleaner in this record. Channel 1 gives the same total of 76 beats, and 75 of the 76 channel-2 beats lie within 100 ms of a channel-1 beat. The only unmatched beat is the low-confidence beat at 35.0 s inside the muscle-noise burst. Its presence is confirmed, so the beat count is correct, but its exact position on channel 2 is uncertain. That beat is therefore excluded from the feature extraction.

Step 5 — ECG feature extraction (every 10 s)

Definitions used.

- **RR interval** of beat *i*: $RR_i = t(R_i) - t(R_{i-1})$, i.e. the interval that ends at that beat (the same set of intervals as $t(R_{i+1}) - t(R_i)$). A beat belongs to the 10-s segment that contains its R-peak.
- **Q point** = minimum of the filtered ECG within 40 ms before R (a Q wave lasts < 40 ms); **S point** = minimum within 80 ms after R; **QRS duration = $t_S - t_Q$**.
- **Reported value per segment = the MEAN** of the individual beats in the segment (SD and median are also given). The RR mean uses only normal-to-normal (NN) intervals and the QRS mean uses only normal beats. The ectopic beats, the compensatory-pause intervals that follow them and the low-confidence beat are excluded because they do not describe the normal rhythm (see the table of excluded beats below). All individual beat values are listed in Appendix A.

Code 17 · Step 5.1 — Q and S points, RR interval and QRS duration of every beat

```
# Step 5.1 — Q and S points, RR interval and QRS duration of every beat | EaindrayMFU26
def q_s_points(ecg, r, fs, q_win=0.040, s_win=0.080):
    """Q = minimum within q_win before R; S = minimum within s_win after R."""
    qw, sw = int(q_win * fs), int(s_win * fs)
    q = np.array([i - qw + np.argmin(ecg[i - qw:i + 1]) for i in r])
    s = np.array([i + np.argmin(ecg[i:i + sw + 1]) for i in r])
    return q, s

SEGMENTS = [f"{a}–{a + 10} s" for a in range(0, 60, 10)]
q60, s60 = q_s_points(ecg60_f, r60, FS)
feat = pd.DataFrame({
    "segment": pd.Categorical([SEGMENTS[int(i // (10 * FS))] for i in r60], SEGMENTS),
    "t_Q (s)": q60 / FS, "t_R (s)": r60 / FS, "t_S (s)": s60 / FS,
    "RR (s)": np.r_[np.nan, np.diff(r60) / FS],          # interval ending at this beat
    "QRS (s)": (s60 - q60) / FS, "beat type": types60},
    index=pd.RangeIndex(1, n_peaks + 1, name="beat"))
is_normal = feat["beat type"].eq("Normal")
feat["NN beat"] = is_normal & is_normal.shift(1, fill_value=False)   # beat & previous normal
feat.round(4).to_csv(RES / "step5_beat_features.csv")
print(f"Beats: {n_peaks} | normal: {is_normal.sum()} | NN beats used for features: "
      f"{feat['NN beat'].sum()}")
feat[~feat["NN beat"]].round(3)              # the beats that are NOT used, and why
```

Code 18 · Step 5.2 — Feature table per 10-s segment (mean ± SD of the beats)

```
# Step 5.2 — Feature table per 10-s segment (mean ± SD of the beats) | EaindrayMFU26
def summarise(g):
    nn, nq = g.loc[g["NN beat"], "RR (s)"], g.loc[g["beat type"].eq("Normal"), "QRS (s)"]
    return pd.Series({"beats detected": len(g),
                      "excluded beats": int((~g["beat type"].eq("Normal")).sum()),
                      "RR n": len(nn), "RR mean (s)": nn.mean(), "RR SD (s)": nn.std(),
                      "RR median (s)": nn.median(), "QRS n": len(nq),
                      "QRS mean (s)": nq.mean(), "QRS SD (s)": nq.std(),
                      "QRS median (s)": nq.median()})

seg_detail = feat.groupby("segment", observed=False).apply(summarise)
seg_detail.round(4).to_csv(RES / "step5_segment_features_detailed.csv")
table5 = pd.DataFrame({
    "Segment": SEGMENTS,
    "RR Interval (s)": [f"{m:.3f} ± {s:.3f}" for m, s in
                        zip(seg_detail["RR mean (s)"], seg_detail["RR SD (s)"])],
    "QRS Duration (s)": [f"{m:.3f} ± {s:.3f}" for m, s in
                         zip(seg_detail["QRS mean (s)"], seg_detail["QRS SD (s)"])]})
table5.to_csv(RES / "step5_segment_features.csv", index=False)
print("Values are MEAN ± SD of the normal beats in each segment\n")
print(table5.to_string(index=False))
seg_detail.round(4)
```

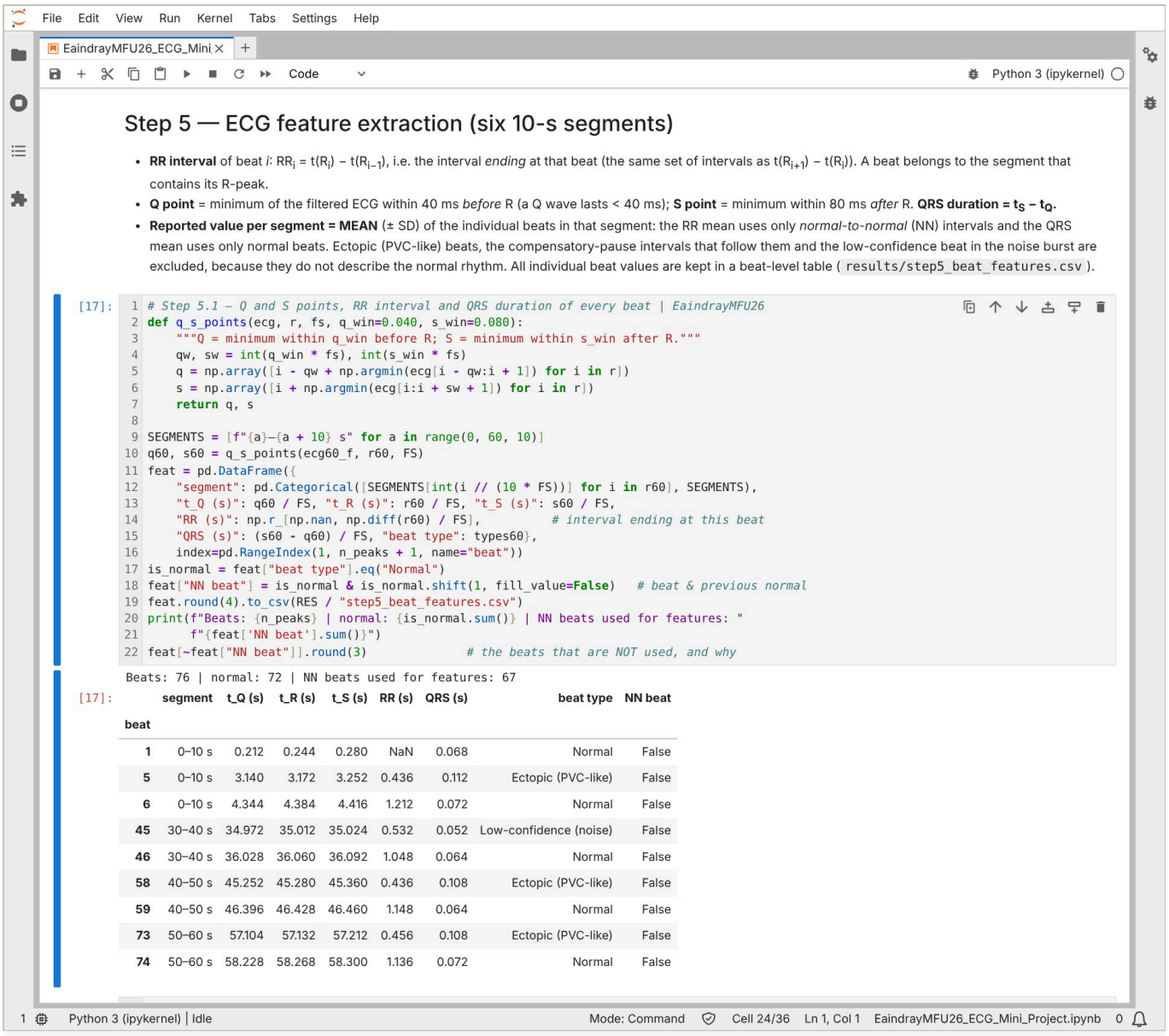
Code 19 · Step 5.3 — Q, R and S points on example beats

```
# Step 5.3 — Q, R and S points on example beats | EaindrayMFU26
examples = [(1.0, "Normal beat"), (3.17, "Ectopic (PVC-like) beat"), (13.1, "Normal beat")]
```

```
fig, axes = plt.subplots(1, 3, figsize=(12, 3.8), sharey=True)
for ax, (tc, name) in zip(axes, examples):
    k = int(np.argmin(np.abs(r60 / FS - tc)))
    m = (t60 >= r60[k] / FS - 0.30) & (t60 <= r60[k] / FS + 0.45)
    ax.axvspan(q60[k] / FS, s60[k] / FS, color=AQUA, alpha=0.12, lw=0)
    ax.plot(t60[m], ecg60_f[m], color=BLUE, lw=1.3)
    ax.plot(r60[k] / FS, ecg60_f[r60[k]], "o", color=ORANGE, ms=7, mec="white", label="R")
    ax.plot(q60[k] / FS, ecg60_f[q60[k]], ">", color=AQUA, ms=8, mec="white", label="Q")
    ax.plot(s60[k] / FS, ecg60_f[s60[k]], "<", color=AQUA, ms=8, mec="white", label="S")
    qrs_ms = (s60[k] - q60[k]) / FS * 1000
    ax.set_title(f"{name} #{k + 1}\nQRS = {qrs_ms:.0f} ms", fontsize=10)
    ax.set_xlabel("Time (s)")
axes[0].set_ylabel("Amplitude (mV)"); axes[0].legend(loc="upper left", ncol=3)
fig.tight_layout(); save(fig, "fig09_qrs_fiducials.png"); plt.show()
```

Code 20 · Step 5.4 — Beat features per segment with segment mean ± SD

```
# Step 5.4 — Beat features per segment with segment mean ± SD | EaindrayMFU26
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, col, use, title in [(axes[0], "RR (s)", feat["NN beat"], "RR interval (NN beats)"),
                            (axes[1], "QRS (s)", is_normal, "QRS duration (normal beats)")]:
    for j, segname in enumerate(SEGMENTS):
        v = feat.loc[use & feat["segment"].eq(segname), col]
        jitter = np.random.default_rng(j).uniform(-0.12, 0.12, v.size)
        ax.plot(j + jitter, v, "o", color=BLUE, ms=4, alpha=0.55, mec="none")
        ax.errorbar(j + 0.3, v.mean(), yerr=v.std(), fmt="o", color=INK, ms=6,
                    capsize=3, lw=1.2)
    ax.set_xticks(range(6), SEGMENTS)
    ax.set(title=title, ylabel=col.replace("(s)", "(s)"))
axes[0].plot([], [], "o", color=BLUE, alpha=0.55, label="individual beats")
axes[0].errorbar([], [], yerr=[], fmt="o", color=INK, label="segment mean ± SD")
axes[0].legend(loc="lower left")
fig.tight_layout(); save(fig, "fig10_segment_features.png"); plt.show()
```



Screenshot 17. Step 5 in JupyterLab — notebook cells markdown, 5.1. [ss16_step5.1.jpg]

File Edit View Run Kernel Tabs Settings Help

EaindrayMFU26_ECG_Mini

Code Python 3 (ipykernel)

[18]:

```
# Step 5.2 — Feature table per 10-s segment (mean ± SD of the beats) | EaindrayMFU26
def summarise(g):
    nn, nq = g.loc[g["NN beat"], "RR (s)"], g.loc[g["beat type"].eq("Normal"), "QRS (s)"]
    return pd.Series({"beats detected": len(g),
                      "excluded beats": int((~g["beat type"].eq("Normal")).sum()),
                      "RR n": len(nn), "RR mean (s)": nn.mean(), "RR SD (s)": nn.std(),
                      "RR median (s)": nn.median(), "QRS n": len(nq),
                      "QRS mean (s)": nq.mean(), "QRS SD (s)": nq.std(),
                      "QRS median (s)": nq.median()})

seg_detail = feat.groupby("segment", observed=False).apply(summarise)
seg_detail.round(4).to_csv(RES / "step5_segment_features_detailed.csv")
table5 = pd.DataFrame({
    "Segment": SEGMENTS,
    "RR Interval (s)": [f"{m:.3f} ± {s:.3f}" for m, s in
                        zip(seg_detail["RR mean (s)"], seg_detail["RR SD (s)"])],
    "QRS Duration (s)": [f"{m:.3f} ± {s:.3f}" for m, s in
                         zip(seg_detail["QRS mean (s)"], seg_detail["QRS SD (s)"])]})
table5.to_csv(RES / "step5_segment_features.csv", index=False)
print("Values are MEAN ± SD of the normal beats in each segment\n")
print(table5.to_string(index=False))
seg_detail.round(4)
```

```
Values are MEAN ± SD of the normal beats in each segment

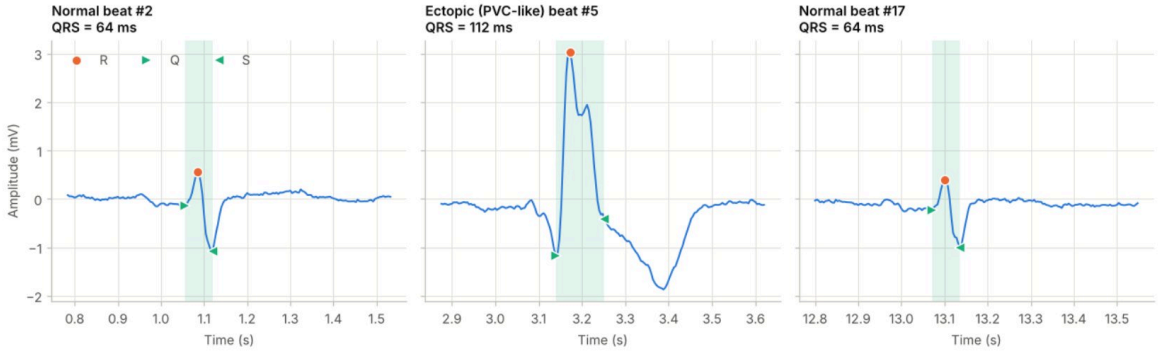
Segment RR Interval (s) QRS Duration (s)
 0–10 s   0.806 ± 0.026    0.068 ± 0.003
10–20 s   0.782 ± 0.018    0.067 ± 0.006
20–30 s   0.784 ± 0.012    0.066 ± 0.007
30–40 s   0.804 ± 0.026    0.061 ± 0.007
40–50 s   0.794 ± 0.030    0.064 ± 0.005
50–60 s   0.793 ± 0.022    0.066 ± 0.006
```

[18]:

segment	beats detected	excluded beats	RR n	RR mean (s)	RR SD (s)	RR median (s)	QRS n	QRS mean (s)	QRS SD (s)	QRS median (s)
0–10 s	13.0	1.0	10.0	0.8064	0.0256	0.810	12.0	0.0677	0.0032	0.068
10–20 s	12.0	0.0	12.0	0.7823	0.0175	0.780	12.0	0.0670	0.0057	0.068
20–30 s	13.0	0.0	13.0	0.7837	0.0117	0.784	13.0	0.0662	0.0072	0.068
30–40 s	13.0	1.0	11.0	0.8044	0.0263	0.800	12.0	0.0613	0.0071	0.064
40–50 s	12.0	1.0	10.0	0.7940	0.0300	0.792	11.0	0.0644	0.0052	0.064
50–60 s	13.0	1.0	11.0	0.7927	0.0217	0.800	12.0	0.0663	0.0060	0.068

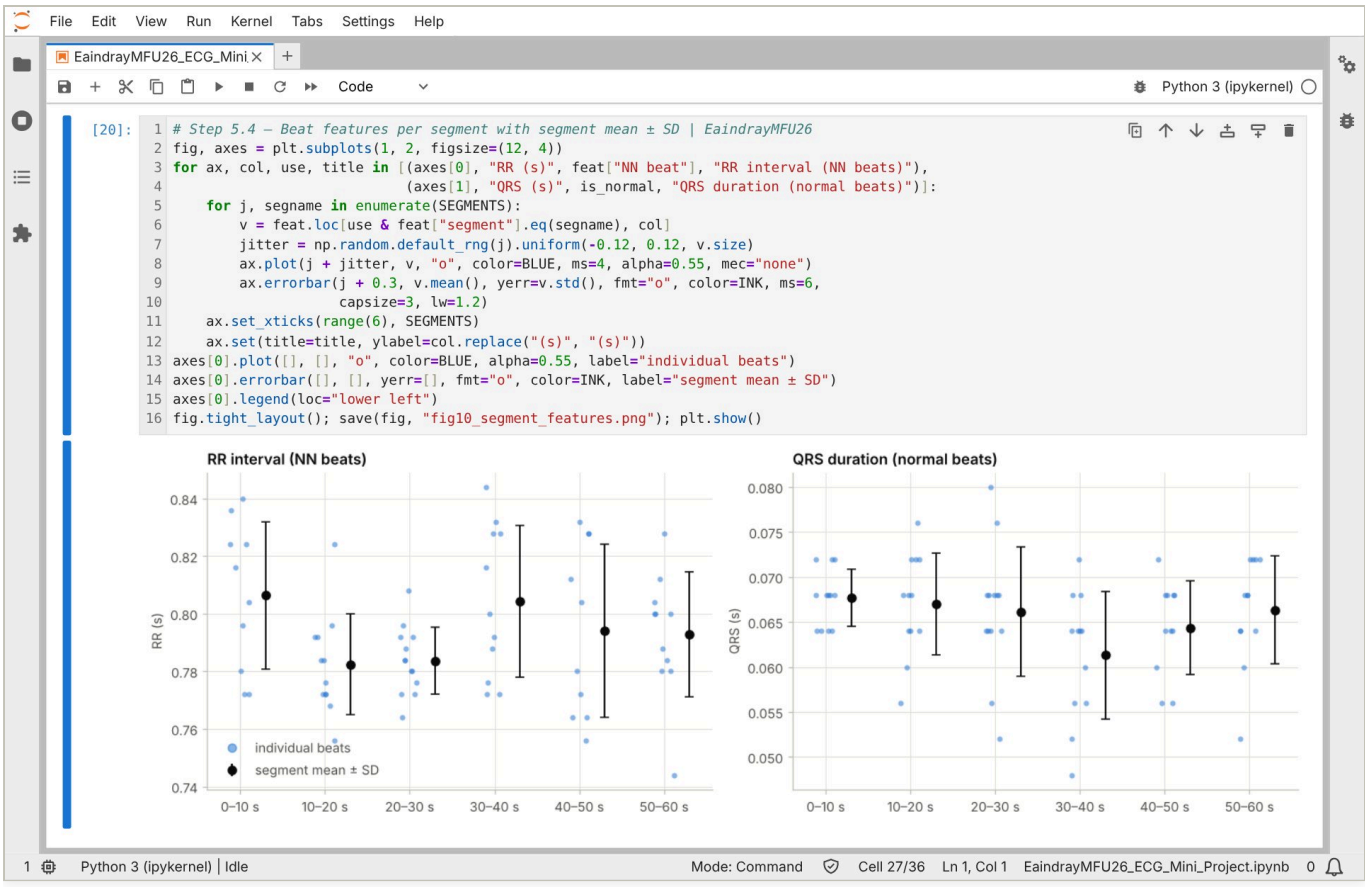
[19]:

```
# Step 5.3 — Q, R and S points on example beats | EaindrayMFU26
examples = [(1.0, "Normal beat"), (3.17, "Ectopic (PVC-like) beat"), (13.1, "Normal beat")]
fig, axes = plt.subplots(1, 3, figsize=(12, 3.8), sharey=True)
for ax, (tc, name) in zip(axes, examples):
    k = int(np.argmin(np.abs(r60 / FS - tc)))
    m = (t60 >= r60[k] / FS - 0.30) & (t60 <= r60[k] / FS + 0.45)
    ax.axvspan(q60[k] / FS, s60[k] / FS, color=AQUA, alpha=0.12, lw=0)
    ax.plot(t60[m], ecg60_f[m], color=BLUE, lw=1.3)
    ax.plot(r60[k] / FS, ecg60_f[r60[k]], "o", color=ORANGE, ms=7, mec="white", label="R")
    ax.plot(q60[k] / FS, ecg60_f[q60[k]], ">", color=AQUA, ms=8, mec="white", label="Q")
    ax.plot(s60[k] / FS, ecg60_f[s60[k]], "<", color=AQUA, ms=8, mec="white", label="S")
    qrs_ms = (s60[k] - q60[k]) / FS * 1000
    ax.set_title(f"{name} #{k + 1}\nQRS = {qrs_ms:.0f} ms", fontsize=10)
    ax.set_xlabel("Time (s)")
axes[0].set_ylabel("Amplitude (mV)"); axes[0].legend(loc="upper left", ncol=3)
fig.tight_layout(); save(fig, "fig09_qrs_fiducials.png"); plt.show()
```



1 Python 3 (ipykernel) | Idle Mode: Command Cell 25/36 Ln 1, Col 1 EaindrayMFU26_ECG_Mini_Project.ipynb 0

Screenshot 18. Step 5 in JupyterLab — notebook cells 5.2, 5.3. [ss17_step5.2.jpg]



Screenshot 19. Step 5 in JupyterLab — notebook cells 5.4. [ss18_step5.4.jpg]

Table 5. Extracted ECG features per 10-s segment (assignment format). Values are the MEAN ± SD of the normal beats in each segment.

Segment	RR Interval (s)	QRS Duration (s)
0–10 s	0.806 ± 0.026	0.068 ± 0.003
10–20 s	0.782 ± 0.018	0.067 ± 0.006
20–30 s	0.784 ± 0.012	0.066 ± 0.007
30–40 s	0.804 ± 0.026	0.061 ± 0.007
40–50 s	0.794 ± 0.030	0.064 ± 0.005
50–60 s	0.793 ± 0.022	0.066 ± 0.006

Table 6. Segment features in detail: number of beats, mean, SD and median of the RR interval (NN beats) and of the QRS duration (normal beats).

segment	beats detected	excluded beats	RR n	RR mean (s)	RR SD (s)	RR median (s)	QRS n	QRS mean (s)	QRS SD (s)	QRS median (s)
0–10 s	13	1	10	0.8064	0.0256	0.8100	12	0.0677	0.0032	0.0680
10–20 s	12	0	12	0.7823	0.0175	0.7800	12	0.0670	0.0057	0.0680
20–30 s	13	0	13	0.7837	0.0117	0.7840	13	0.0662	0.0072	0.0680
30–40 s	13	1	11	0.8044	0.0263	0.8000	12	0.0613	0.0071	0.0640
40–50 s	12	1	10	0.7940	0.0300	0.7920	11	0.0644	0.0052	0.0640
50–60 s	13	1	11	0.7927	0.0217	0.8000	12	0.0663	0.0060	0.0680

Table 7. The 9 beats whose RR interval is NOT used for the segment means or for the Normal class in Step 6, and why (their QRS is still used if the beat itself is normal).

beat	segment	t_R (s)	RR (s)	QRS (s)	beat type	reason
1	0–10 s	0.244	–	0.068	Normal	first beat – no preceding R-peak
5	0–10 s	3.172	0.436	0.112	Ectopic (PVC-like)	ectopic (PVC-like) beat
6	0–10 s	4.384	1.212	0.072	Normal	RR = compensatory pause after an ectopic beat
45	30–40 s	35.012	0.532	0.052	Low-confidence (noise)	low-confidence beat (noise burst)
46	30–40 s	36.060	1.048	0.064	Normal	RR adjacent to the low-confidence beat
58	40–50 s	45.280	0.436	0.108	Ectopic (PVC-like)	ectopic (PVC-like) beat
59	40–50 s	46.428	1.148	0.064	Normal	RR = compensatory pause after an ectopic beat
73	50–60 s	57.132	0.456	0.108	Ectopic (PVC-like)	ectopic (PVC-like) beat
74	50–60 s	58.268	1.136	0.072	Normal	RR = compensatory pause after an ectopic beat

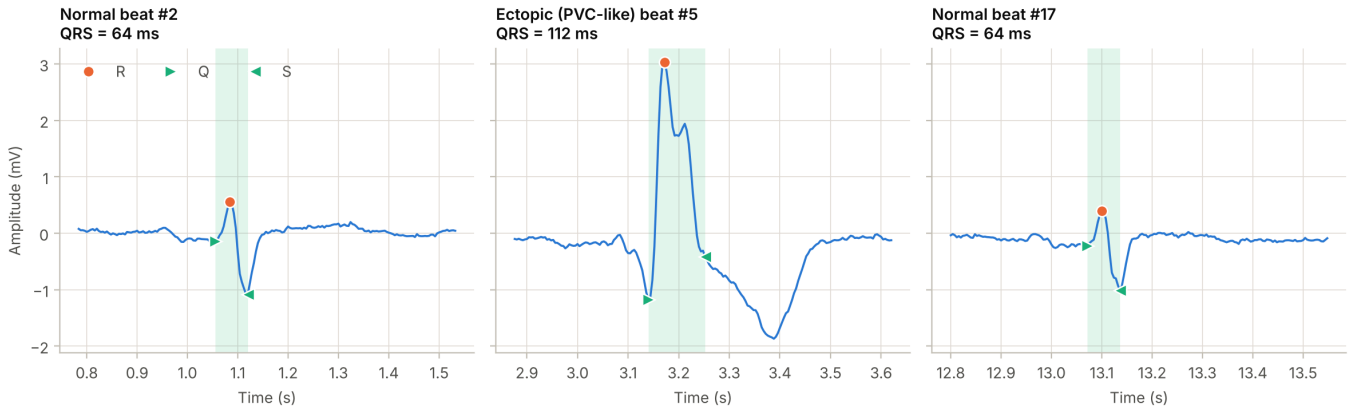


Figure 10. Q, R and S points on two normal beats and on the ectopic beat; the shaded span is the QRS duration $t_S - t_Q$.

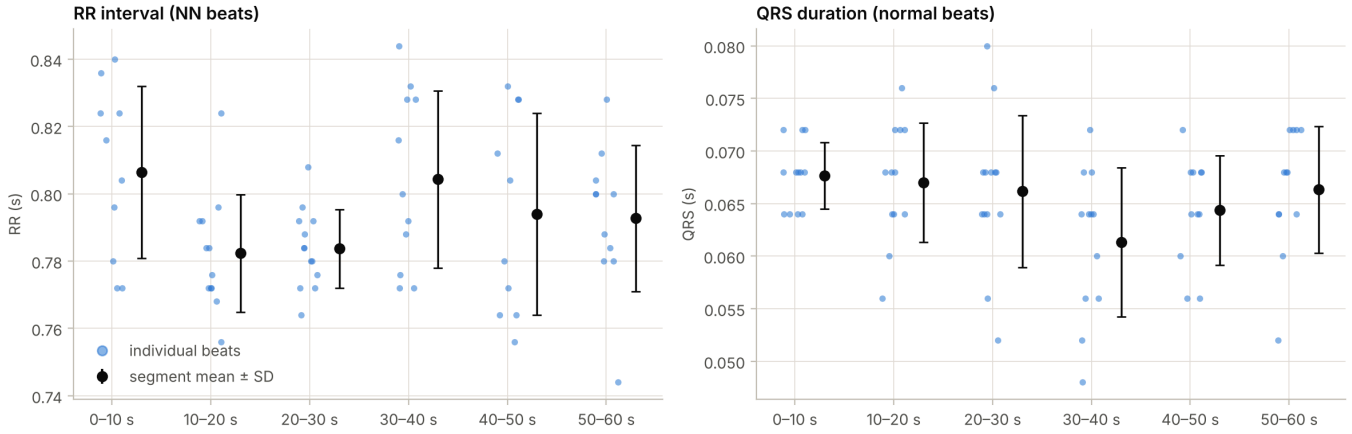


Figure 11. Individual RR intervals (NN beats) and QRS durations (normal beats) in each segment (dots) with the segment mean ± SD.

Observation. The features are stable across the six segments: mean RR 0.782–0.806 s (≈ 75–77 bpm) and mean QRS 0.061–0.068 s, far below the 0.12-s limit of a wide QRS. For the ectopic beats the QRS is 0.108–0.112 s and the RR only 0.44–0.46 s.

Step 6 — Machine-learning classification

A. Normal-class data. The RR interval and QRS duration of the 67 individual NN beats from Step 5 are labelled *Normal*. Individual beat values are used rather than only the six segment means because the arrhythmia samples are also individual beats: both classes are then described in the same unit (one sample = one beat), and the classes are reasonably balanced (67 vs 50 instead of 6 vs 50). The six-segment-mean version is tested as a robustness check (Step 6.6).
B. Arrhythmia-class data. The 50 beats from the assignment table (`data/arrhythmia_training_data.csv`), labelled *Arrhythmia*.
C. Model. X = [RR interval, QRS duration], y = [Normal, Arrhythmia]. A Support Vector Machine with an RBF kernel (C = 1, γ = 'scale') is trained after feature standardisation, using a stratified 70 % / 30 % train/test split (random_state = 42). An RBF kernel is chosen because arrhythmic beats lie on *both* sides of the normal RR range (too short and too long). A single straight boundary in RR cannot separate them; a curved boundary can.

Code 21 · Step 6.1 — Data set: Normal (Step 5) + Arrhythmia (assignment table)

```
# Step 6.1 — Data set: Normal (Step 5) + Arrhythmia (assignment table) | EaindrayMFU26
nn = feat[feat["NN beat"]]
normal_df = pd.DataFrame({"sample": [f"N{i:02d}" for i in range(1, len(nn) + 1)],
                          "RR (s)": nn["RR (s)"].to_numpy(),
                          "QRS (s)": nn["QRS (s)"].to_numpy(),
                          "label": "Normal", "source": "AHA 0001, ch 2 (Step 5)"})
arr = pd.read_csv(DATA / "arrhythmia_training_data.csv")
arr_df = pd.DataFrame({"sample": [f"A{i:02d}" for i in arr["beat_no"]],
                       "RR (s)": arr["RR_interval_s"], "QRS (s)": arr["QRS_duration_s"],
                       "label": "Arrhythmia", "source": "assignment table"})
dataset = pd.concat([normal_df, arr_df], ignore_index=True)
X = dataset[["RR (s)", "QRS (s)"]].to_numpy()        # X = [RR interval, QRS duration]
y = (dataset["label"] == "Arrhythmia").astype(int).to_numpy()   # 0 = Normal, 1 = Arrhythmia
print(dataset["label"].value_counts().to_string(), "\n")
print(dataset.groupby("label")[["RR (s)", "QRS (s)"]].describe().round(3).T.to_string())

fig, ax = plt.subplots(figsize=(10, 5))
for lab, colr in (("Normal", BLUE), ("Arrhythmia", ORANGE)):
    d = dataset[dataset["label"] == lab]
    ax.plot(d["RR (s)"], d["QRS (s)"], "o", ms=7, color=colr, mec="white", mew=1, ls="none",
            label=f"{lab} (n = {len(d)})")
ax.set(title="Combined data set — X = [RR interval, QRS duration]", xlabel="RR interval (s)",
       ylabel="QRS duration (s)", xlim=(0.1, 1.7), ylim=(0.03, 0.22))
ax.legend(loc="upper center", ncol=2)
save(fig, "fig11_dataset_scatter.png"); plt.show()
```

Code 22 · Step 6.2 — Train/test split and model training (SVM, RBF kernel)

```
# Step 6.2 — Train/test split and model training (SVM, RBF kernel) | EaindrayMFU26
idx = np.arange(len(dataset))
X_tr, X_te, y_tr, y_te, i_tr, i_te = train_test_split(
    X, y, idx, test_size=0.30, stratify=y, random_state=SEED)
model = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1.0, gamma="scale"))
model.fit(X_tr, y_tr)
dataset["split"] = np.where(np.isin(idx, i_te), "test", "train")
dataset.round(5).to_csv(RES / "step6_dataset.csv", index=False)
print(f"Training set: {len(y_tr)} samples (Normal {np.sum(y_tr == 0)}, "
      f"Arrhythmia {np.sum(y_tr == 1)})")
print(f"Test set    : {len(y_te)} samples (Normal {np.sum(y_te == 0)}, "
      f"Arrhythmia {np.sum(y_te == 1)})")
print(f"Support vectors: {model[-1].n_support_.sum()} | training accuracy: "
      f"{model.score(X_tr, y_tr):.3f}")
```

Code 23 · Step 6.3 — Predict the class of the test samples

```
# Step 6.3 — Predict the class of the test samples | EaindrayMFU26
names = np.array(["Normal", "Arrhythmia"])
y_pred = model.predict(X_te)
pred_table = dataset.iloc[i_te][["sample", "RR (s)", "QRS (s)", "label"]].copy()
pred_table["predicted"] = names[y_pred]
pred_table["decision score"] = model.decision_function(X_te).round(2)
pred_table["correct"] = pred_table["label"] == pred_table["predicted"]
```

```
pred_table = pred_table.sort_values("sample").reset_index(drop=True)
pred_table.round(4).to_csv(RES / "step6_test_predictions.csv", index=False)
print(f"Correct predictions: {pred_table['correct'].sum()} / {len(pred_table)}")
pred_table.round(4)
```

Code 24 · Step 6.4 — Evaluate: confusion matrix and performance metrics

```
# Step 6.4 — Evaluate: confusion matrix and performance metrics | EaindrayMFU26
cm = confusion_matrix(y_te, y_pred, labels=[0, 1])
tn, fp, fn, tp = cm.ravel()                 # positive class = Arrhythmia
metrics = pd.Series({"Accuracy": (tp + tn) / cm.sum(),
                     "Sensitivity / Recall": tp / (tp + fn),
                     "Specificity": tn / (tn + fp),
                     "Precision (PPV)": tp / (tp + fp),
                     "F1-score": 2 * tp / (2 * tp + fp + fn)})
cv = cross_val_score(model, X, y, cv=StratifiedKFold(5, shuffle=True, random_state=SEED))
metrics["5-fold CV accuracy (mean)"], metrics["5-fold CV accuracy (SD)"] = cv.mean(), cv.std()
metrics.round(4).to_csv(RES / "step6_metrics.csv", header=["value"])
print(pd.DataFrame(cm, index=["true Normal", "true Arrhythmia"],
                   columns=["pred Normal", "pred Arrhythmia"]), "\n")
print(metrics.round(4).to_string(), "\n")
print(classification_report(y_te, y_pred, target_names=names, digits=3))

fig, ax = plt.subplots(figsize=(4.6, 4))
ax.imshow(cm, cmap=plt.matplotlib.colors.LinearSegmentedColormap.from_list(
    "blue", ["#cde2fb", "#104281"]), vmin=0, vmax=cm.max())
for (a, b), v in np.ndenumerate(cm):
    ax.text(b, a, str(v), ha="center", va="center", fontsize=16, fontweight="semibold",
            color="white" if v > cm.max() / 2 else INK)
ax.set_xticks([0, 1], names); ax.set_yticks([0, 1], names)
ax.set(xlabel="Predicted class", ylabel="True class", title="Confusion matrix — test set")
ax.grid(False); [s.set_visible(False) for s in ax.spines.values()]
save(fig, "fig12_confusion_matrix.png"); plt.show()
```

Code 25 · Step 6.5 — Decision boundary; model applied to the excluded beats

```
# Step 6.5 — Decision boundary; model applied to the excluded beats | EaindrayMFU26
is_ect = feat["beat type"].eq("Ectopic (PVC-like)")
extra = feat[is_ect | is_ect.shift(1, fill_value=False)][["t_R (s)", "RR (s)", "QRS (s)"]]
extra.insert(0, "beat kind", np.where(is_ect[extra.index], "ectopic (PVC-like)",
                                      "sinus beat after the pause"))
Xe = extra[["RR (s)", "QRS (s)"]].to_numpy()
extra["predicted"], extra["decision score"] = names[model.predict(Xe)], \
    model.decision_function(Xe).round(2)
ect = extra[extra["beat kind"].str.startswith("ectopic")]
extra.round(3).to_csv(RES / "step6_excluded_beats_predictions.csv")
print("Beats excluded from Step 6 (never used for training or testing):")
print(extra.round(3).to_string(), "\n")

gx, gy = np.meshgrid(np.linspace(0.1, 1.7, 400), np.linspace(0.03, 0.22, 400))
Z = model.decision_function(np.c_[gx.ravel(), gy.ravel()]).reshape(gx.shape)
fig, ax = plt.subplots(figsize=(10, 5.6))
ax.contourf(gx, gy, Z > 0, levels=[-0.5, 0.5, 1.5], colors=[BLUE, ORANGE], alpha=0.10)
ax.contour(gx, gy, Z, levels=[0], colors=INK, linewidths=1.2)
for lab, colr in (("Normal", BLUE), ("Arrhythmia", ORANGE)):
    for split, face in (("train", colr), ("test", "white")):
        d = dataset[(dataset["label"] == lab) & (dataset["split"] == split)]
        ax.plot(d["RR (s)"], d["QRS (s)"], "o", ms=7, mfc=face, mec=colr, mew=1.6,
                label=f"{lab} — {split}", ls="none")
ax.plot(ect["RR (s)"], ect["QRS (s)"], "D", ms=8, mfc=AQUA, mec="white", mew=1.2, ls="none",
        label="candidate's ectopic beats (not used)")
ax.plot([], [], color=INK, lw=1.2, label="SVM decision boundary")
ax.set(title="SVM (RBF) decision regions — RR interval vs QRS duration",
       xlabel="RR interval (s)", ylabel="QRS duration (s)", xlim=(0.1, 1.7),
       ylim=(0.03, 0.22))
ax.legend(loc="upper center", ncol=3, fontsize=8.5, bbox_to_anchor=(0.5, -0.13))
save(fig, "fig13_decision_boundary.png"); plt.show()
```

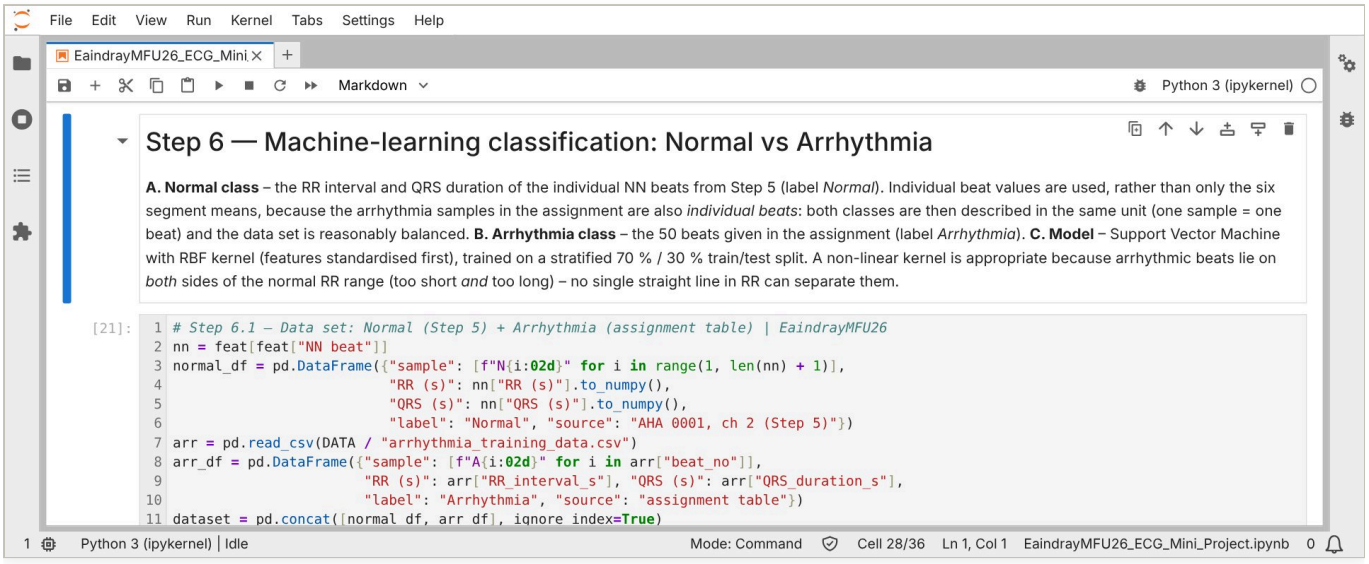
Code 26 · Step 6.6 — Robustness check: the six segment means as the Normal class

```
# Step 6.6 — Robustness check: the six segment means as the Normal class | EaindrayMFU26
X_seg = np.vstack([seg_detail[["RR mean (s)", "QRS mean (s)"]].to_numpy(),
                   arr_df[["RR (s)", "QRS (s)"]].to_numpy()])
```

```
y_seg = np.r_[np.zeros(6, int), np.ones(len(arr_df), int)]
Xs_tr, Xs_te, ys_tr, ys_te = train_test_split(X_seg, y_seg, test_size=0.30, stratify=y_seg,
                                              random_state=SEED)
m_seg = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1.0, gamma="scale"))
ys_pred = m_seg.fit(Xs_tr, ys_tr).predict(Xs_te)
print(f"Segment-mean version: {len(y_seg)} samples (6 Normal, {len(arr_df)} Arrhythmia)")
print("Confusion matrix (test):\n", confusion_matrix(ys_te, ys_pred, labels=[0, 1]))
print(f"Accuracy: {np.mean(ys_te == ys_pred):.3f}")
```

Code 27 · Step 7 — Save a summary of all key results

```
# Step 7 — Save a summary of all key results | EaindrayMFU26
summary = {
    "student": STUDENT, "record": "ahadb/0001", "channel": CHANNEL + 1, "fs_hz": FS,
    "filter": {"type": "FIR band-stop (Kaiser window)", "numtaps": int(numtaps),
               "beta": round(float(beta), 3), "cutoffs_hz": [F0 - HALF_BW, F0 + HALF_BW],
               "atten_50hz_db_single_pass": round(float(att50), 1),
               **{k: float(v) for k, v in filter_stats.items()}},
    "rpeaks_first10s": int(len(r10)),
    "heart_rate": {"n_peaks": int(n_peaks), "hr_count_bpm": round(float(hr_count), 1),
                   "hr_mean_rr_bpm": round(float(hr_meanrr), 1),
                   "n_ectopic": int((types60 == "Ectopic (PVC-like)").sum()),
                   "n_low_confidence": int((types60 == "Low-confidence (noise)").sum()),
                   "classification": classify_hr(hr_count)},
    "segment_table": table5.to_dict(orient="records"),
    "ml": {"n_normal": int((y == 0).sum()), "n_arrhythmia": int((y == 1).sum()),
           "n_train": int(len(y_tr)), "n_test": int(len(y_te)),
           "confusion_matrix": cm.tolist(), **{k: float(v) for k, v in metrics.items()},
           "ectopic_predictions": ect["predicted"].tolist(),
           "post_pause_predictions": extra.loc[~extra["beat kind"].str.startswith("ectopic"),
                                               "predicted"].tolist(),
           "segment_mean_check_accuracy": float(np.mean(ys_te == ys_pred))}}
(RES / "summary.json").write_text(json.dumps(summary, indent=2, ensure_ascii=False))
print(json.dumps(summary["heart_rate"], indent=1, ensure_ascii=False))
print(json.dumps({k: summary["ml"][k] for k in ("confusion_matrix", "Accuracy")}, indent=1))
```



Screenshot 20. Step 6 in JupyterLab — notebook cells markdown. [ss19_stepnotes.jpg]

File Edit View Run Kernel Tabs Settings Help

EaindrayMFU26_ECG_Mini

Code Python 3 (ipykernel)

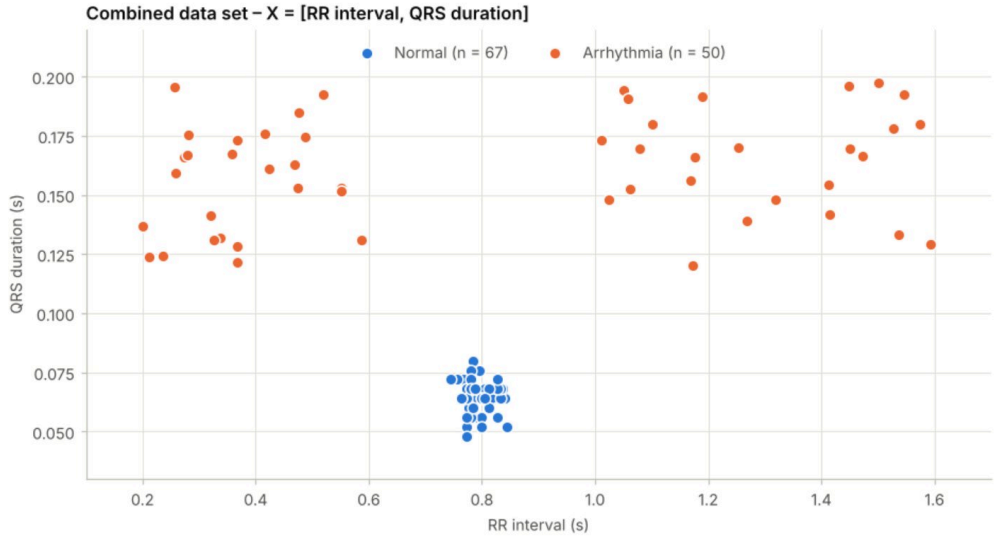
[21]:

```
# Step 6.1 — Data set: Normal (Step 5) + Arrhythmia (assignment table) | EaindrayMFU26
nn = feat[feat["NN beat"]]
normal_df = pd.DataFrame({"sample": [f"N{i:02d}" for i in range(1, len(nn) + 1)],
                          "RR (s)": nn["RR (s)"].to_numpy(),
                          "QRS (s)": nn["QRS (s)"].to_numpy(),
                          "label": "Normal", "source": "AHA 0001, ch 2 (Step 5)"})
arr = pd.read_csv(DATA / "arrhythmia_training_data.csv")
arr_df = pd.DataFrame({"sample": [f"A{i:02d}" for i in arr["beat_no"]],
                       "RR (s)": arr["RR_interval_s"], "QRS (s)": arr["QRS_duration_s"],
                       "label": "Arrhythmia", "source": "assignment table"})
dataset = pd.concat([normal_df, arr_df], ignore_index=True)
X = dataset[["RR (s)", "QRS (s)"]].to_numpy()        # X = [RR interval, QRS duration]
y = (dataset["label"] == "Arrhythmia").astype(int).to_numpy()   # 0 = Normal, 1 = Arrhythmia
print(dataset["label"].value_counts().to_string(), "\n")
print(dataset.groupby("label")[["RR (s)", "QRS (s)"]].describe().round(3).T.to_string())

fig, ax = plt.subplots(figsize=(10, 5))
for lab, colr in (("Normal", BLUE), ("Arrhythmia", ORANGE)):
    d = dataset[dataset["label"] == lab]
    ax.plot(d["RR (s)"], d["QRS (s)"], "o", ms=7, color=colr, mec="white", mew=1, ls="none",
            label=f"{lab} (n = {len(d)})")
ax.set(title="Combined data set — X = [RR interval, QRS duration]", xlabel="RR interval (s)",
       ylabel="QRS duration (s)", xlim=(0.1, 1.7), ylim=(0.03, 0.22))
ax.legend(loc="upper center", ncol=2)
save(fig, "fig11_dataset_scatter.png"); plt.show()
```

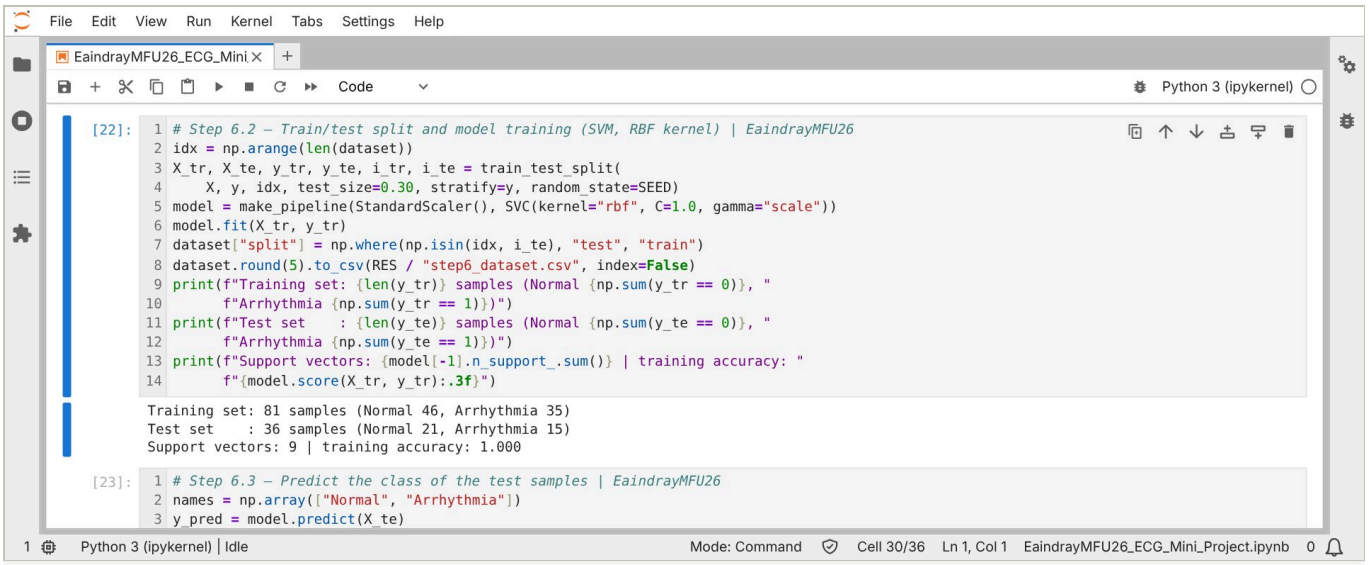
```
label
Normal        67
Arrhythmia    50

label          Arrhythmia  Normal
RR (s)  count      50.000  67.000
        mean        0.836   0.793
        std         0.492   0.023
        min         0.200   0.744
        25%         0.367   0.774
        50%         0.799   0.788
        75%         1.265   0.810
        max         1.593   0.844
QRS (s) count      50.000  67.000
        mean        0.160   0.065
        std         0.023   0.006
        min         0.120   0.048
        25%         0.141   0.064
        50%         0.164   0.068
        75%         0.176   0.068
        max         0.197   0.080
```

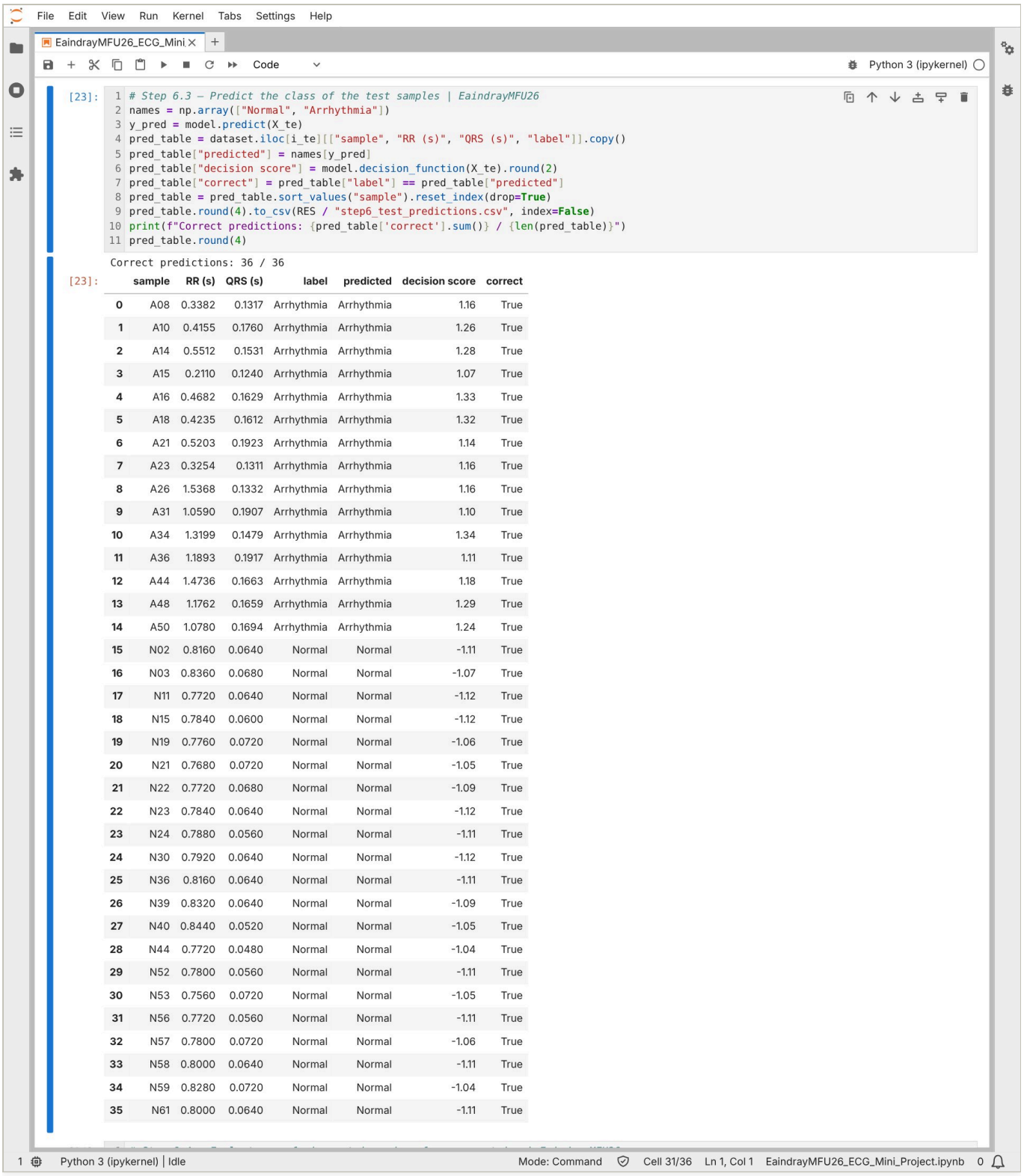


1 Python 3 (ipykernel) | Idle Mode: Command Cell 29/36 Ln 1, Col 1 EaindrayMFU26_ECG_Mini_Project.ipynb 0

Screenshot 21. Step 6 in JupyterLab — notebook cells 6.1. [ss20_step6.1.jpg]



Screenshot 22. Step 6 in JupyterLab — notebook cells 6.2. [ss21_step6.2.jpg]



	sample	RR (s)	QRS (s)	label	predicted	decision score	correct
0	A08	0.3382	0.1317	Arrhythmia	Arrhythmia	1.16	True
1	A10	0.4155	0.1760	Arrhythmia	Arrhythmia	1.26	True
2	A14	0.5512	0.1531	Arrhythmia	Arrhythmia	1.28	True
3	A15	0.2110	0.1240	Arrhythmia	Arrhythmia	1.07	True
4	A16	0.4682	0.1629	Arrhythmia	Arrhythmia	1.33	True
5	A18	0.4235	0.1612	Arrhythmia	Arrhythmia	1.32	True
6	A21	0.5203	0.1923	Arrhythmia	Arrhythmia	1.14	True
7	A23	0.3254	0.1311	Arrhythmia	Arrhythmia	1.16	True
8	A26	1.5368	0.1332	Arrhythmia	Arrhythmia	1.16	True
9	A31	1.0590	0.1907	Arrhythmia	Arrhythmia	1.10	True
10	A34	1.3199	0.1479	Arrhythmia	Arrhythmia	1.34	True
11	A36	1.1893	0.1917	Arrhythmia	Arrhythmia	1.11	True
12	A44	1.4736	0.1663	Arrhythmia	Arrhythmia	1.18	True
13	A48	1.1762	0.1659	Arrhythmia	Arrhythmia	1.29	True
14	A50	1.0780	0.1694	Arrhythmia	Arrhythmia	1.24	True
15	N02	0.8160	0.0640	Normal	Normal	-1.11	True
16	N03	0.8360	0.0680	Normal	Normal	-1.07	True
17	N11	0.7720	0.0640	Normal	Normal	-1.12	True
18	N15	0.7840	0.0600	Normal	Normal	-1.12	True
19	N19	0.7760	0.0720	Normal	Normal	-1.06	True
20	N21	0.7680	0.0720	Normal	Normal	-1.05	True
21	N22	0.7720	0.0680	Normal	Normal	-1.09	True
22	N23	0.7840	0.0640	Normal	Normal	-1.12	True
23	N24	0.7880	0.0560	Normal	Normal	-1.11	True
24	N30	0.7920	0.0640	Normal	Normal	-1.12	True
25	N36	0.8160	0.0640	Normal	Normal	-1.11	True
26	N39	0.8320	0.0640	Normal	Normal	-1.09	True
27	N40	0.8440	0.0520	Normal	Normal	-1.05	True
28	N44	0.7720	0.0480	Normal	Normal	-1.04	True
29	N52	0.7800	0.0560	Normal	Normal	-1.11	True
30	N53	0.7560	0.0720	Normal	Normal	-1.05	True
31	N56	0.7720	0.0560	Normal	Normal	-1.11	True
32	N57	0.7800	0.0720	Normal	Normal	-1.06	True
33	N58	0.8000	0.0640	Normal	Normal	-1.11	True
34	N59	0.8280	0.0720	Normal	Normal	-1.04	True
35	N61	0.8000	0.0640	Normal	Normal	-1.11	True

Screenshot 23. Step 6 in JupyterLab — notebook cells 6.3. [ss22_step6.3.jpg]

File Edit View Run Kernel Tabs Settings Help

EaindrayMFU26_ECG_Mini

Code Python 3 (ipykernel)

[24]:

```
# Step 6.4 — Evaluate: confusion matrix and performance metrics | EaindrayMFU26
cm = confusion_matrix(y_te, y_pred, labels=[0, 1])
tn, fp, fn, tp = cm.ravel()                # positive class = Arrhythmia
metrics = pd.Series({"Accuracy": (tp + tn) / cm.sum(),
                     "Sensitivity / Recall": tp / (tp + fn),
                     "Specificity": tn / (tn + fp),
                     "Precision (PPV)": tp / (tp + fp),
                     "F1-score": 2 * tp / (2 * tp + fp + fn)})
cv = cross_val_score(model, X, y, cv=StratifiedKFold(5, shuffle=True, random_state=SEED))
metrics["5-fold CV accuracy (mean)"], metrics["5-fold CV accuracy (SD)"] = cv.mean(), cv.std()
metrics.round(4).to_csv(RES / "step6_metrics.csv", header=["value"])
print(pd.DataFrame(cm, index=["true Normal", "true Arrhythmia"],
                   columns=["pred Normal", "pred Arrhythmia"]), "\n")
print(metrics.round(4).to_string(), "\n")
print(classification_report(y_te, y_pred, target_names=names, digits=3))

fig, ax = plt.subplots(figsize=(4.6, 4))
ax.imshow(cm, cmap=plt.matplotlib.colors.LinearSegmentedColormap.from_list(
    "blue", ["#cde2fb", "#104281"]), vmin=0, vmax=cm.max())
for (a, b), v in np.ndenumerate(cm):
    ax.text(b, a, str(v), ha="center", va="center", fontsize=16, fontweight="semibold",
            color="white" if v > cm.max() / 2 else INK)
ax.set_xticks([0, 1], names); ax.set_yticks([0, 1], names)
ax.set(xlabel="Predicted class", ylabel="True class", title="Confusion matrix — test set")
ax.grid(False); [s.set_visible(False) for s in ax.spines.values()]
save(fig, "fig12_confusion_matrix.png"); plt.show()
```

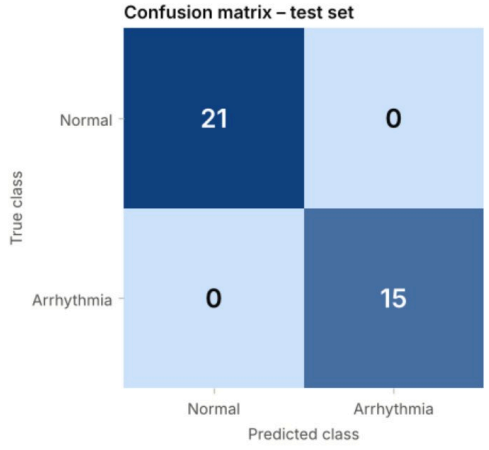
```
                pred Normal  pred Arrhythmia
true Normal              21                0
true Arrhythmia           0               15

Accuracy                    1.0
Sensitivity / Recall        1.0
Specificity                 1.0
Precision (PPV)             1.0
F1-score                    1.0
5-fold CV accuracy (mean)   1.0
5-fold CV accuracy (SD)     0.0

              precision    recall  f1-score   support

      Normal      1.000     1.000     1.000        21
  Arrhythmia      1.000     1.000     1.000        15

    accuracy                        1.000        36
   macro avg      1.000     1.000     1.000        36
weighted avg      1.000     1.000     1.000        36
```



1 Python 3 (ipykernel) | Idle Mode: Command Cell 32/36 Ln 1, Col 1 EaindrayMFU26_ECG_Mini_Project.ipynb 0

Screenshot 24. Step 6 in JupyterLab — notebook cells 6.4. [ss23_step6.4.jpg]

File Edit View Run Kernel Tabs Settings Help

EaindrayMFU26_ECG_Mini

Code Python 3 (ipykernel)

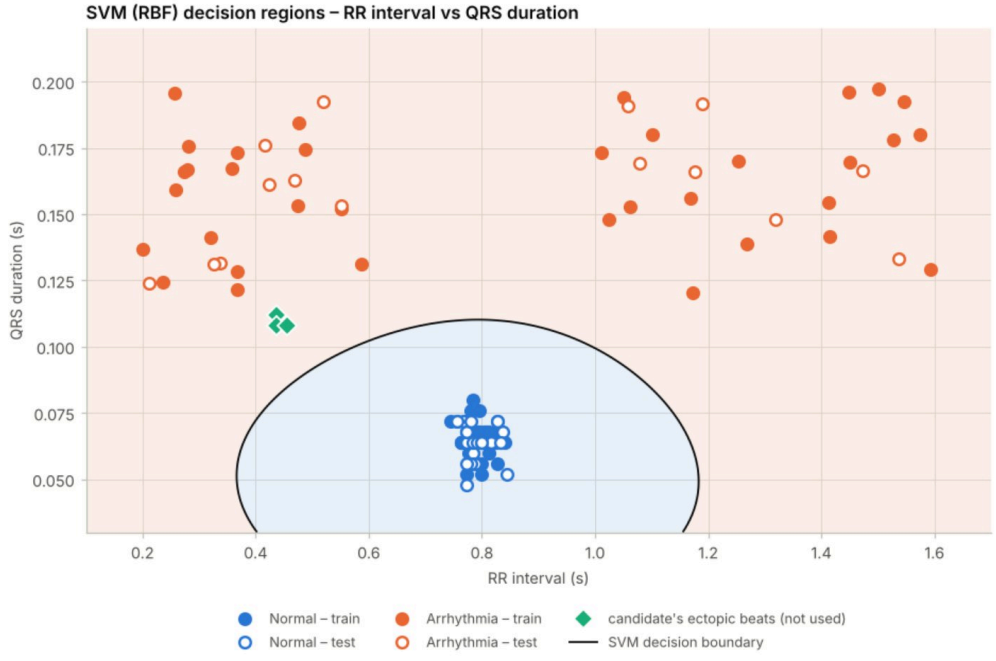
[25]:

```
# Step 6.5 — Decision boundary; model applied to the excluded beats | EaindrayMFU26
is_ect = feat["beat type"].eq("Ectopic (PVC-like)")
extra = feat[is_ect | is_ect.shift(1, fill_value=False)][["t_R (s)", "RR (s)", "QRS (s)"]]
extra.insert(0, "beat kind", np.where(is_ect[extra.index], "ectopic (PVC-like)",
                                      "sinus beat after the pause"))
Xe = extra[["RR (s)", "QRS (s)"]].to_numpy()
extra["predicted"], extra["decision score"] = names[model.predict(Xe)], \
    model.decision_function(Xe).round(2)
ect = extra[extra["beat kind"].str.startswith("ectopic")]
extra.round(3).to_csv(RES / "step6_excluded_beats_predictions.csv")
print("Beats excluded from Step 6 (never used for training or testing):")
print(extra.round(3).to_string(), "\n")

gx, gy = np.meshgrid(np.linspace(0.1, 1.7, 400), np.linspace(0.03, 0.22, 400))
Z = model.decision_function(np.c_[gx.ravel(), gy.ravel()]).reshape(gx.shape)
fig, ax = plt.subplots(figsize=(10, 5.6))
ax.contourf(gx, gy, Z > 0, levels=[-0.5, 0.5, 1.5], colors=[BLUE, ORANGE], alpha=0.10)
ax.contour(gx, gy, Z, levels=[0], colors=INK, linewidths=1.2)
for lab, colr in (("Normal", BLUE), ("Arrhythmia", ORANGE)):
    for split, face in (("train", colr), ("test", "white")):
        d = dataset[(dataset["label"] == lab) & (dataset["split"] == split)]
        ax.plot(d["RR (s)"], d["QRS (s)"], "o", ms=7, mfc=face, mec=colr, mew=1.6,
                label=f"{lab} — {split}", ls="none")
ax.plot(ect["RR (s)"], ect["QRS (s)"], "D", ms=8, mfc=AQUA, mec="white", mew=1.2, ls="none",
        label="candidate's ectopic beats (not used)")
ax.plot([], [], color=INK, lw=1.2, label="SVM decision boundary")
ax.set(title="SVM (RBF) decision regions — RR interval vs QRS duration",
       xlabel="RR interval (s)", ylabel="QRS duration (s)", xlim=(0.1, 1.7),
       ylim=(0.03, 0.22))
ax.legend(loc="upper center", ncol=3, fontsize=8.5, bbox_to_anchor=(0.5, -0.13))
save(fig, "fig13_decision_boundary.png"); plt.show()
```

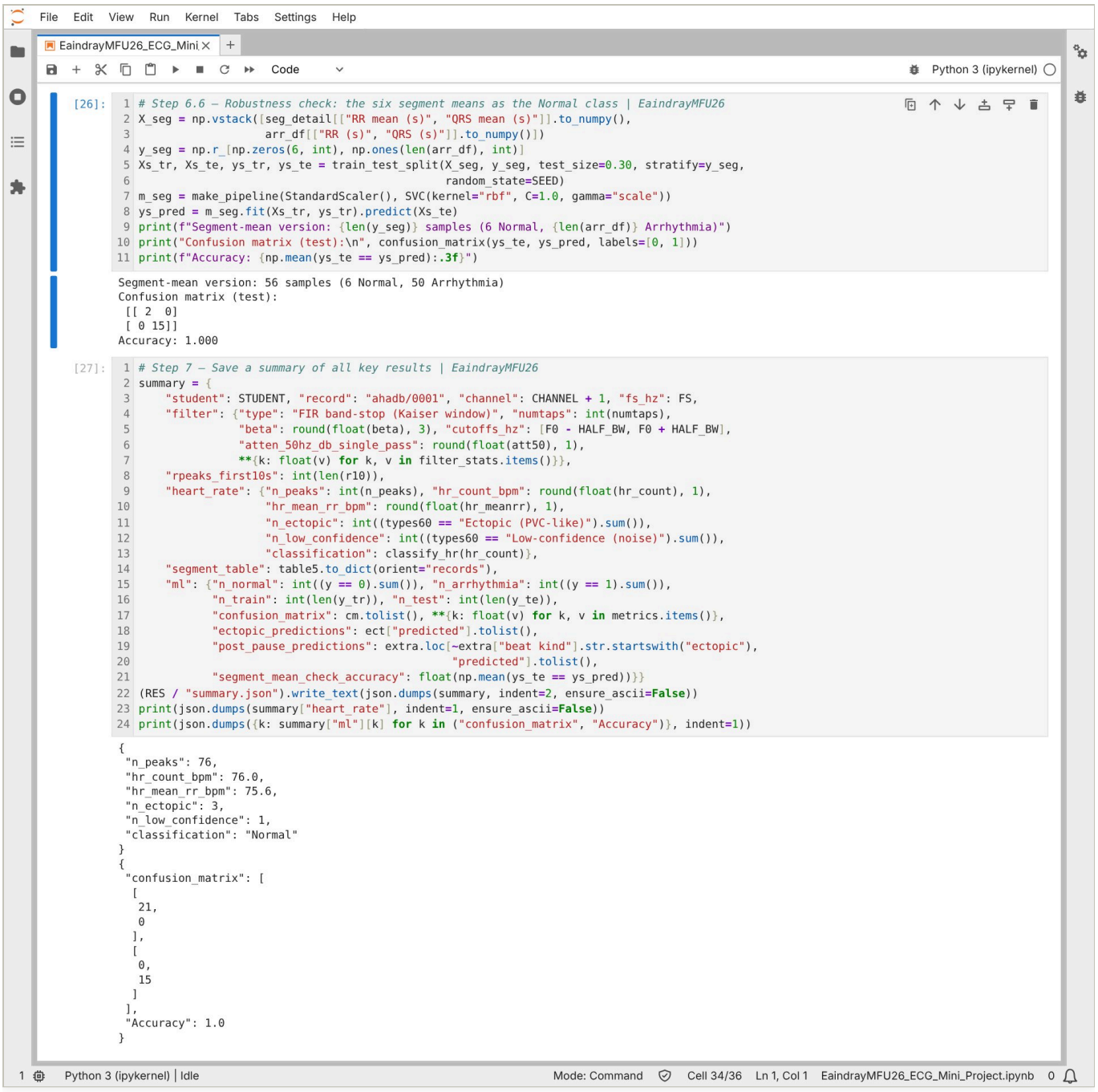
```
Beats excluded from Step 6 (never used for training or testing):
                      beat kind  t_R (s)  RR (s)  QRS (s)   predicted  decision score

beat
5             ectopic (PVC-like)    3.172   0.436    0.112  Arrhythmia            0.70
6     sinus beat after the pause    4.384   1.212    0.072  Arrhythmia            0.24
58            ectopic (PVC-like)   45.280   0.436    0.108  Arrhythmia            0.60
59    sinus beat after the pause   46.428   1.148    0.064      Normal           -0.08
73            ectopic (PVC-like)   57.132   0.456    0.108  Arrhythmia            0.56
74    sinus beat after the pause   58.268   1.136    0.072      Normal           -0.05
```



1 Python 3 (ipykernel) | Idle Mode: Command Cell 33/36 Ln 1, Col 1 EaindrayMFU26_ECG_Mini_Project.ipynb 0

Screenshot 25. Step 6 in JupyterLab — notebook cells 6.5. [ss24_step6.5.jpg]



Screenshot 26. Step 6 in JupyterLab — notebook cells 6.6, 7. [ss25_step6.6.jpg]

Table 8. Combined data set — summary per class.

class	RR (s) count	RR (s) mean	RR (s) std	RR (s) min	RR (s) max	QRS (s) count	QRS (s) mean	QRS (s) std	QRS (s) min	QRS (s) max
Arrhythmia	50	0.836	0.492	0.200	1.593	50	0.160	0.023	0.120	0.197
Normal	67	0.793	0.023	0.744	0.844	67	0.065	0.006	0.048	0.080

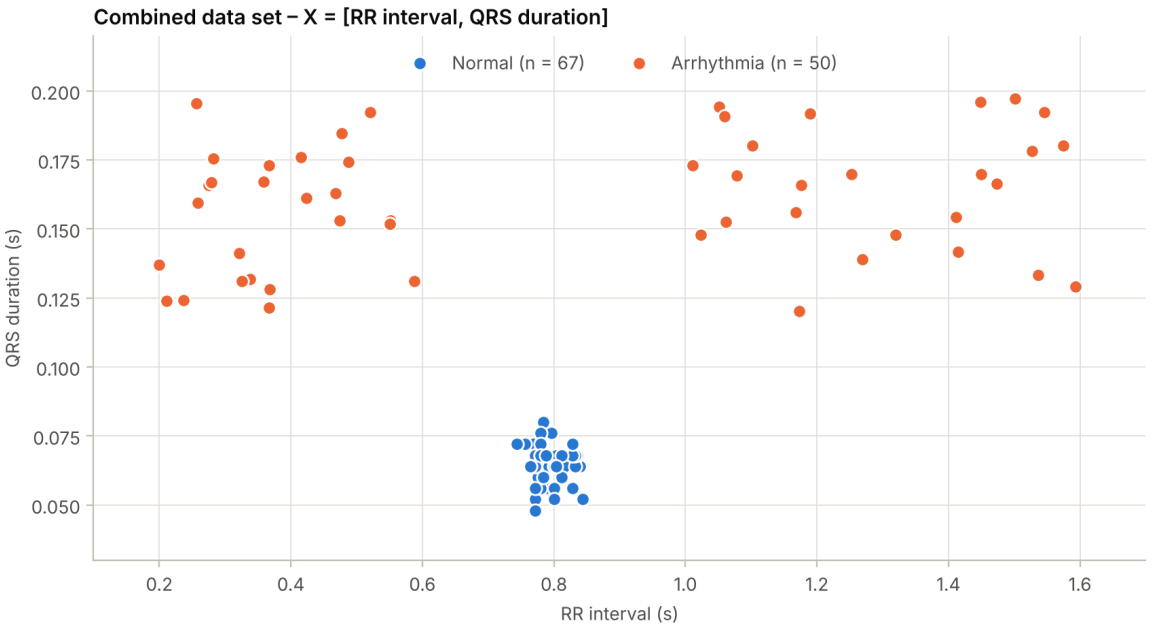


Figure 12. The combined data set in feature space: X = [RR interval, QRS duration].

Table 9. Stratified train/test split (70 % / 30 %).

split	Arrhythmia	Normal
test	15	21
train	35	46

Table 10. Predicted class of the 36 test samples (decision score < 0 → Normal, > 0 → Arrhythmia).

sample	RR (s)	QRS (s)	label	predicted	decision score	correct
A08	0.3382	0.1317	Arrhythmia	Arrhythmia	1.16	True
A10	0.4155	0.1760	Arrhythmia	Arrhythmia	1.26	True
A14	0.5512	0.1531	Arrhythmia	Arrhythmia	1.28	True
A15	0.2110	0.1240	Arrhythmia	Arrhythmia	1.07	True
A16	0.4682	0.1629	Arrhythmia	Arrhythmia	1.33	True
A18	0.4235	0.1612	Arrhythmia	Arrhythmia	1.32	True
A21	0.5203	0.1923	Arrhythmia	Arrhythmia	1.14	True
A23	0.3254	0.1311	Arrhythmia	Arrhythmia	1.16	True
A26	1.5368	0.1332	Arrhythmia	Arrhythmia	1.16	True
A31	1.0590	0.1907	Arrhythmia	Arrhythmia	1.10	True
A34	1.3199	0.1479	Arrhythmia	Arrhythmia	1.34	True
A36	1.1893	0.1917	Arrhythmia	Arrhythmia	1.11	True
A44	1.4736	0.1663	Arrhythmia	Arrhythmia	1.18	True
A48	1.1762	0.1659	Arrhythmia	Arrhythmia	1.29	True
A50	1.0780	0.1694	Arrhythmia	Arrhythmia	1.24	True
N02	0.8160	0.0640	Normal	Normal	-1.11	True
N03	0.8360	0.0680	Normal	Normal	-1.07	True
N11	0.7720	0.0640	Normal	Normal	-1.12	True
N15	0.7840	0.0600	Normal	Normal	-1.12	True
N19	0.7760	0.0720	Normal	Normal	-1.06	True
N21	0.7680	0.0720	Normal	Normal	-1.05	True
N22	0.7720	0.0680	Normal	Normal	-1.09	True
N23	0.7840	0.0640	Normal	Normal	-1.12	True

sample	RR (s)	QRS (s)	label	predicted	decision score	correct
N24	0.7880	0.0560	Normal	Normal	-1.11	True
N30	0.7920	0.0640	Normal	Normal	-1.12	True
N36	0.8160	0.0640	Normal	Normal	-1.11	True
N39	0.8320	0.0640	Normal	Normal	-1.09	True
N40	0.8440	0.0520	Normal	Normal	-1.05	True
N44	0.7720	0.0480	Normal	Normal	-1.04	True
N52	0.7800	0.0560	Normal	Normal	-1.11	True
N53	0.7560	0.0720	Normal	Normal	-1.05	True
N56	0.7720	0.0560	Normal	Normal	-1.11	True
N57	0.7800	0.0720	Normal	Normal	-1.06	True
N58	0.8000	0.0640	Normal	Normal	-1.11	True
N59	0.8280	0.0720	Normal	Normal	-1.04	True
N61	0.8000	0.0640	Normal	Normal	-1.11	True

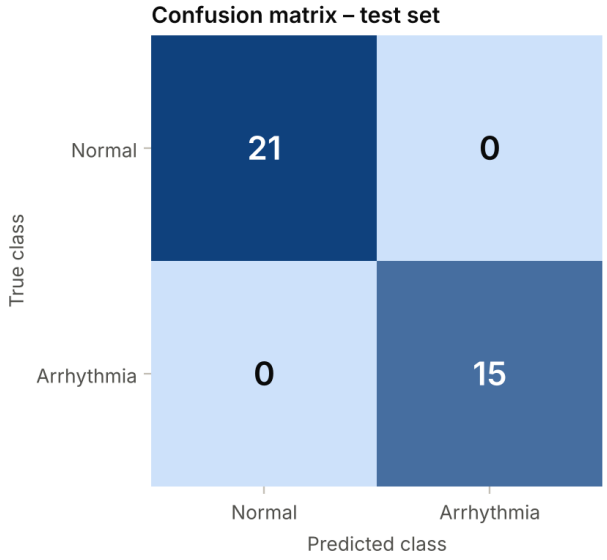


Figure 13. Confusion matrix of the test set (positive class = Arrhythmia).

Table 11. Model performance on the test set (positive class = Arrhythmia) and 5-fold cross-validation on the whole data set.

Quantity	Value
Accuracy	100.0 %
Sensitivity / Recall	100.0 %
Specificity	100.0 %
Precision (PPV)	100.0 %
F1-score	100.0 %
5-fold CV accuracy (mean)	100.0 %
5-fold CV accuracy (SD)	0.000

Accuracy = (TP + TN) / N = (15 + 21) / 36 · Sensitivity = TP / (TP + FN) = 15 / 15 · Specificity = TN / (TN + FP) = 21 / 21 · Precision = TP / (TP + FP) = 15 / 15

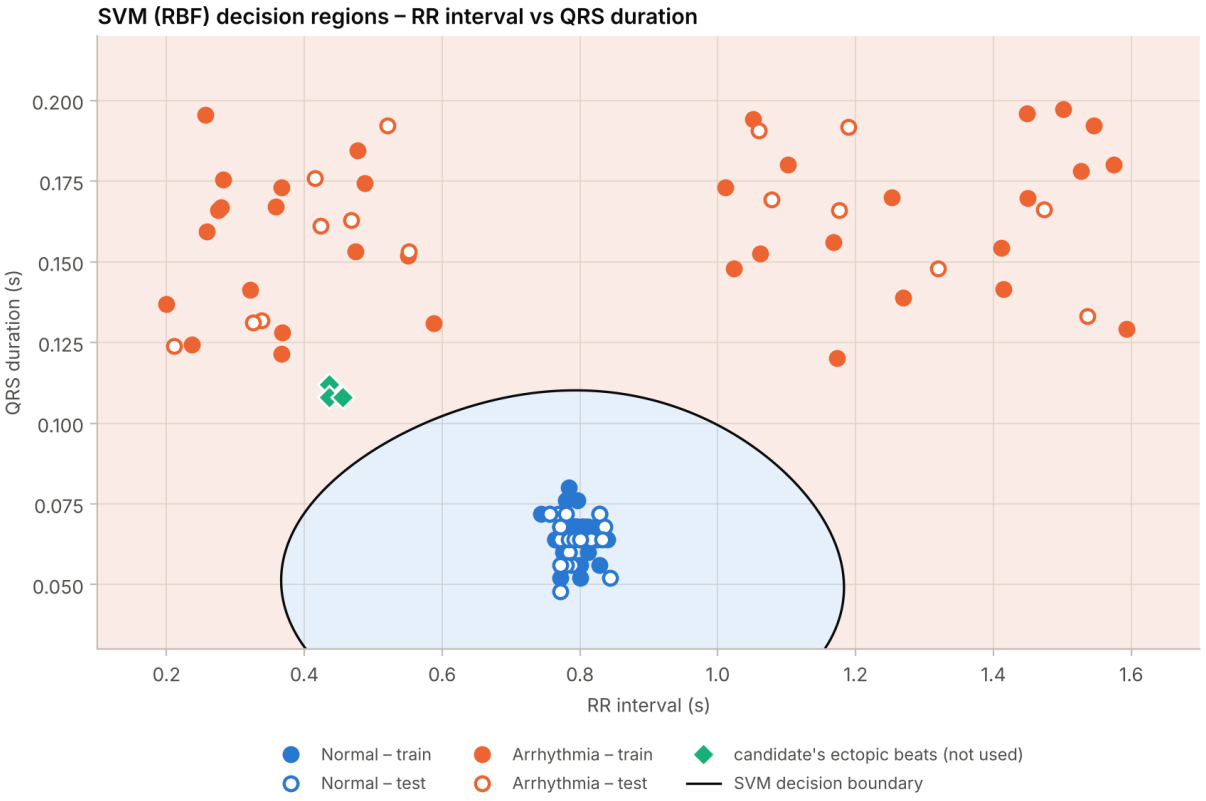


Figure 14. SVM decision regions (blue = Normal, orange = Arrhythmia) with the training (filled) and test (open) samples; green diamonds = the candidate's three ectopic beats, which were not used for training or testing.

Table 12. Model applied to beats that were excluded from the data set: the three ectopic beats and the sinus beats that follow their compensatory pauses.

beat	beat kind	t_R (s)	RR (s)	QRS (s)	predicted	decision score
5	ectopic (PVC-like)	3.172	0.436	0.112	Arrhythmia	0.70
6	sinus beat after the pause	4.384	1.212	0.072	Arrhythmia	0.24
58	ectopic (PVC-like)	45.280	0.436	0.108	Arrhythmia	0.60
59	sinus beat after the pause	46.428	1.148	0.064	Normal	-0.08
73	ectopic (PVC-like)	57.132	0.456	0.108	Arrhythmia	0.56
74	sinus beat after the pause	58.268	1.136	0.072	Normal	-0.05

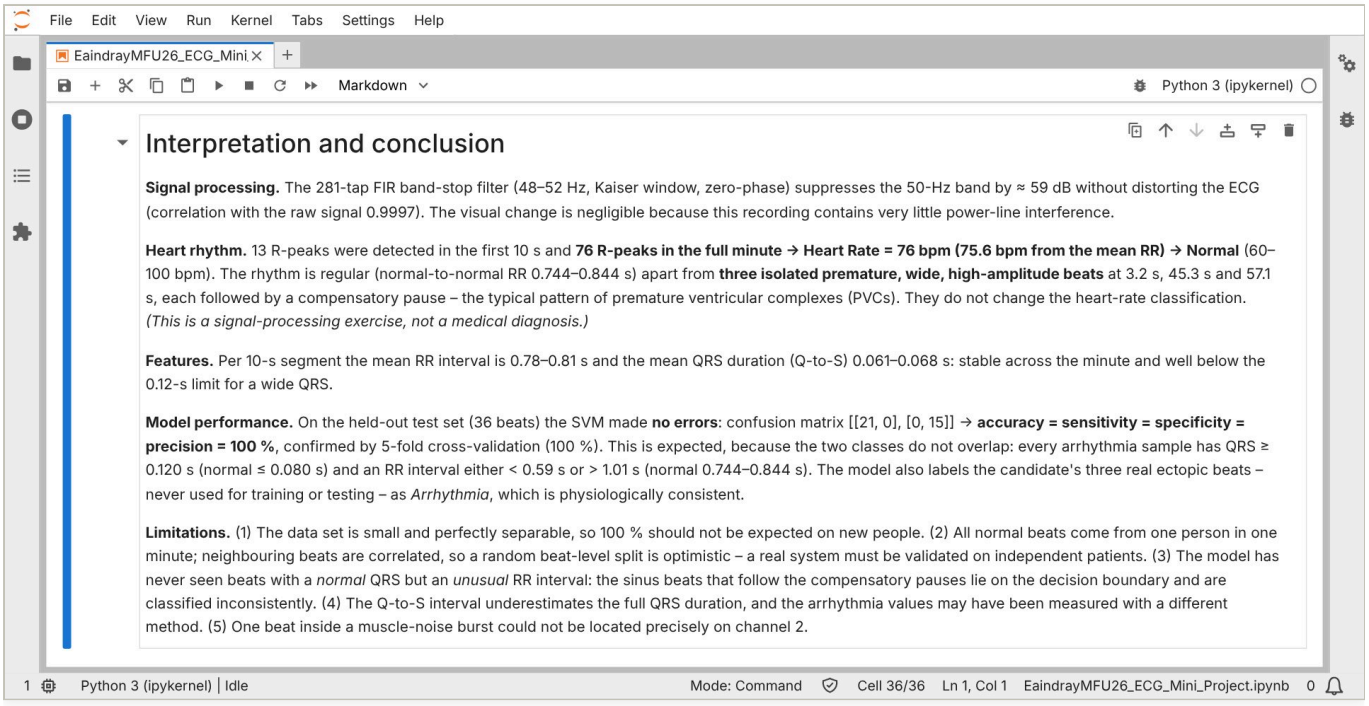
Robustness check. Using only the six segment means as the Normal class (6 Normal + 50 Arrhythmia samples) also gives a test accuracy of 100 %.

Interpretation of the model performance. The SVM classified all 36 test beats correctly: TN = 21, FP = 0, FN = 0, TP = 15. This gives accuracy, sensitivity (recall), specificity and precision of 100 %, and 5-fold cross-validation agrees (100 %). The result is expected because the two classes do not overlap in feature space. Every arrhythmia sample has a wide QRS (≥ 0.120 s, normal ≤ 0.080 s), and its RR interval is either short (< 0.59 s) or long (> 1.01 s), while the normal RR range is 0.744–0.844 s. The SVM learned a closed region around the normal beats. It also labelled the candidate's three real ectopic beats as *Arrhythmia*, which is physiologically consistent, although it never saw them.

The perfect score must be read with care: (1) the data set is small and perfectly separable; (2) all normal beats come from one person in one minute, and neighbouring beats are correlated, so a random beat-level split is optimistic; (3) the model has never seen beats with a normal QRS but an unusual RR, such as the sinus beats after the compensatory pauses, which lie on the decision boundary (scores between −0.1 and +0.25) and are classified inconsistently; (4) the arrhythmia values may have been measured with a different QRS method. A real screening model would need many patients, beat types and inter-patient validation.

Interpretation, conclusion and limitations

- **Filtering.** The 281-tap FIR band-stop filter (48–52 Hz, zero phase) removes the 50-Hz band (≈ −59 dB) without distorting the ECG. The visible improvement is negligible because the recording contains very little power-line interference.
- **R-peaks.** Pan–Tompkins with two plausibility checks detected 13 R-peaks in the first 10 s and 76 in the full minute. 75 of the 76 beats agree with an independent detector on channel 1, and the remaining beat lies in a muscle-noise burst.
- **Heart rhythm.** Heart Rate = 76 bpm (75.6 bpm from the mean RR) → **Normal**. The rhythm is regular apart from three isolated premature, wide, high-amplitude beats with compensatory pauses (premature ventricular complexes). This is a signal-processing exercise, not a medical diagnosis.
- **Features.** Segment means: RR 0.782–0.806 s, QRS 0.061–0.068 s. The features are stable and typical of a normal rhythm with narrow QRS complexes.
- **Machine learning.** The SVM (RBF) reaches 100 % accuracy, sensitivity, specificity and precision on the test set, because the two classes are clearly separated by QRS duration and RR interval. The estimate is optimistic: the data set is small and comes from one subject, and the model is uncertain for normal-QRS beats with unusual RR intervals.



Screenshot 27. The conclusion cell in JupyterLab. [ss26_stepnotes.jpg]

Appendix A — Beat-level feature table (all 76 beats, 1 minute)

Table 13. All detected beats with Q, R, S times, RR interval (ending at the beat), QRS duration, beat type and whether the beat is a normal-to-normal (NN) beat used in Steps 5–6.

beat	segment	t_Q (s)	t_R (s)	t_S (s)	RR (s)	QRS (s)	beat type	NN beat
1	0–10 s	0.212	0.244	0.280	–	0.068	Normal	False
2	0–10 s	1.056	1.084	1.120	0.840	0.064	Normal	True
3	0–10 s	1.872	1.900	1.936	0.816	0.064	Normal	True
4	0–10 s	2.700	2.736	2.768	0.836	0.068	Normal	True
5	0–10 s	3.140	3.172	3.252	0.436	0.112	Ectopic (PVC-like)	False
6	0–10 s	4.344	4.384	4.416	1.212	0.072	Normal	False
7	0–10 s	5.168	5.208	5.236	0.824	0.068	Normal	True
8	0–10 s	5.996	6.032	6.060	0.824	0.064	Normal	True
9	0–10 s	6.800	6.836	6.868	0.804	0.068	Normal	True
10	0–10 s	7.592	7.632	7.660	0.796	0.068	Normal	True
11	0–10 s	8.364	8.404	8.436	0.772	0.072	Normal	True
12	0–10 s	9.156	9.184	9.220	0.780	0.064	Normal	True
13	0–10 s	9.920	9.956	9.992	0.772	0.072	Normal	True
14	10–20 s	10.700	10.728	10.764	0.772	0.064	Normal	True
15	10–20 s	11.448	11.484	11.520	0.756	0.072	Normal	True
16	10–20 s	12.240	12.276	12.308	0.792	0.068	Normal	True
17	10–20 s	13.072	13.100	13.136	0.824	0.064	Normal	True
18	10–20 s	13.856	13.884	13.916	0.784	0.060	Normal	True
19	10–20 s	14.636	14.668	14.700	0.784	0.064	Normal	True
20	10–20 s	15.424	15.464	15.500	0.796	0.076	Normal	True
21	10–20 s	16.204	16.236	16.272	0.772	0.068	Normal	True
22	10–20 s	16.972	17.012	17.044	0.776	0.072	Normal	True
23	10–20 s	17.772	17.804	17.828	0.792	0.056	Normal	True
24	10–20 s	18.532	18.572	18.604	0.768	0.072	Normal	True
25	10–20 s	19.304	19.344	19.372	0.772	0.068	Normal	True
26	20–30 s	20.096	20.128	20.160	0.784	0.064	Normal	True
27	20–30 s	20.884	20.916	20.940	0.788	0.056	Normal	True
28	20–30 s	21.660	21.692	21.724	0.776	0.064	Normal	True
29	20–30 s	22.424	22.464	22.492	0.772	0.068	Normal	True
30	20–30 s	23.204	23.244	23.272	0.780	0.068	Normal	True
31	20–30 s	23.984	24.016	24.036	0.772	0.052	Normal	True
32	20–30 s	24.776	24.812	24.844	0.796	0.068	Normal	True
33	20–30 s	25.568	25.604	25.632	0.792	0.064	Normal	True
34	20–30 s	26.348	26.388	26.428	0.784	0.080	Normal	True
35	20–30 s	27.144	27.180	27.212	0.792	0.068	Normal	True
36	20–30 s	27.920	27.960	27.996	0.780	0.076	Normal	True
37	20–30 s	28.696	28.724	28.760	0.764	0.064	Normal	True
38	20–30 s	29.496	29.532	29.564	0.808	0.068	Normal	True
39	30–40 s	30.320	30.348	30.384	0.816	0.064	Normal	True
40	30–40 s	31.116	31.148	31.172	0.800	0.056	Normal	True
41	30–40 s	31.944	31.976	32.000	0.828	0.056	Normal	True
42	30–40 s	32.772	32.808	32.836	0.832	0.064	Normal	True
43	30–40 s	33.620	33.652	33.672	0.844	0.052	Normal	True
44	30–40 s	34.440	34.480	34.512	0.828	0.072	Normal	True

beat	segment	t_Q (s)	t_R (s)	t_S (s)	RR (s)	QRS (s)	beat type	NN beat
45	30–40 s	34.972	35.012	35.024	0.532	0.052	Low-confidence (noise)	False
46	30–40 s	36.028	36.060	36.092	1.048	0.064	Normal	False
47	30–40 s	36.820	36.852	36.888	0.792	0.068	Normal	True
48	30–40 s	37.600	37.628	37.660	0.776	0.060	Normal	True
49	30–40 s	38.376	38.400	38.424	0.772	0.048	Normal	True
50	30–40 s	39.140	39.172	39.204	0.772	0.064	Normal	True
51	30–40 s	39.924	39.960	39.992	0.788	0.068	Normal	True
52	40–50 s	40.752	40.788	40.820	0.828	0.068	Normal	True
53	40–50 s	41.588	41.620	41.656	0.832	0.068	Normal	True
54	40–50 s	42.416	42.448	42.484	0.828	0.068	Normal	True
55	40–50 s	43.232	43.260	43.292	0.812	0.060	Normal	True
56	40–50 s	44.032	44.064	44.100	0.804	0.068	Normal	True
57	40–50 s	44.820	44.844	44.876	0.780	0.056	Normal	True
58	40–50 s	45.252	45.280	45.360	0.436	0.108	Ectopic (PVC-like)	False
59	40–50 s	46.396	46.428	46.460	1.148	0.064	Normal	False
60	40–50 s	47.144	47.184	47.216	0.756	0.072	Normal	True
61	40–50 s	47.912	47.948	47.976	0.764	0.064	Normal	True
62	40–50 s	48.680	48.712	48.744	0.764	0.064	Normal	True
63	40–50 s	49.456	49.484	49.512	0.772	0.056	Normal	True
64	50–60 s	50.224	50.264	50.296	0.780	0.072	Normal	True
65	50–60 s	51.024	51.064	51.088	0.800	0.064	Normal	True
66	50–60 s	51.852	51.892	51.924	0.828	0.072	Normal	True
67	50–60 s	52.672	52.704	52.740	0.812	0.068	Normal	True
68	50–60 s	53.476	53.504	53.540	0.800	0.064	Normal	True
69	50–60 s	54.252	54.284	54.320	0.780	0.068	Normal	True
70	50–60 s	55.036	55.072	55.104	0.788	0.068	Normal	True
71	50–60 s	55.840	55.872	55.892	0.800	0.052	Normal	True
72	50–60 s	56.648	56.676	56.712	0.804	0.064	Normal	True
73	50–60 s	57.104	57.132	57.212	0.456	0.108	Ectopic (PVC-like)	False
74	50–60 s	58.228	58.268	58.300	1.136	0.072	Normal	False
75	50–60 s	58.976	59.012	59.048	0.744	0.072	Normal	True
76	50–60 s	59.764	59.796	59.824	0.784	0.060	Normal	True

Appendix B — Machine-learning data set (117 samples)

Table 14. Combined data set: N01–N67 = Normal beats from Step 5; A01–A50 = Arrhythmia beats from the assignment table.

sample	RR (s)	QRS (s)	label	split	sample	RR (s)	QRS (s)	label	split
N01	0.8400	0.0640	Normal	train	N60	0.8120	0.0680	Normal	train
N02	0.8160	0.0640	Normal	test	N61	0.8000	0.0640	Normal	test
N03	0.8360	0.0680	Normal	test	N62	0.7800	0.0680	Normal	train
N04	0.8240	0.0680	Normal	train	N63	0.7880	0.0680	Normal	train
N05	0.8240	0.0640	Normal	train	N64	0.8000	0.0520	Normal	train
N06	0.8040	0.0680	Normal	train	N65	0.8040	0.0640	Normal	train
N07	0.7960	0.0680	Normal	train	N66	0.7440	0.0720	Normal	train
N08	0.7720	0.0720	Normal	train	N67	0.7840	0.0600	Normal	train
N09	0.7800	0.0640	Normal	train	A01	0.3668	0.1216	Arrhythmia	train
N10	0.7720	0.0720	Normal	train	A02	0.4881	0.1743	Arrhythmia	train

sample	RR (s)	QRS (s)	label	split
N11	0.7720	0.0640	Normal	test
N12	0.7560	0.0720	Normal	train
N13	0.7920	0.0680	Normal	train
N14	0.8240	0.0640	Normal	train
N15	0.7840	0.0600	Normal	test
N16	0.7840	0.0640	Normal	train
N17	0.7960	0.0760	Normal	train
N18	0.7720	0.0680	Normal	train
N19	0.7760	0.0720	Normal	test
N20	0.7920	0.0560	Normal	train
N21	0.7680	0.0720	Normal	test
N22	0.7720	0.0680	Normal	test
N23	0.7840	0.0640	Normal	test
N24	0.7880	0.0560	Normal	test
N25	0.7760	0.0640	Normal	train
N26	0.7720	0.0680	Normal	train
N27	0.7800	0.0680	Normal	train
N28	0.7720	0.0520	Normal	train
N29	0.7960	0.0680	Normal	train
N30	0.7920	0.0640	Normal	test
N31	0.7840	0.0800	Normal	train
N32	0.7920	0.0680	Normal	train
N33	0.7800	0.0760	Normal	train
N34	0.7640	0.0640	Normal	train
N35	0.8080	0.0680	Normal	train
N36	0.8160	0.0640	Normal	test
N37	0.8000	0.0560	Normal	train
N38	0.8280	0.0560	Normal	train
N39	0.8320	0.0640	Normal	test
N40	0.8440	0.0520	Normal	test
N41	0.8280	0.0720	Normal	train
N42	0.7920	0.0680	Normal	train
N43	0.7760	0.0600	Normal	train
N44	0.7720	0.0480	Normal	test
N45	0.7720	0.0640	Normal	train
N46	0.7880	0.0680	Normal	train
N47	0.8280	0.0680	Normal	train
N48	0.8320	0.0680	Normal	train
N49	0.8280	0.0680	Normal	train
N50	0.8120	0.0600	Normal	train
N51	0.8040	0.0680	Normal	train
N52	0.7800	0.0560	Normal	test
N53	0.7560	0.0720	Normal	test
N54	0.7640	0.0640	Normal	train
N55	0.7640	0.0640	Normal	train
N56	0.7720	0.0560	Normal	test
N57	0.7800	0.0720	Normal	test
N58	0.8000	0.0640	Normal	test

sample	RR (s)	QRS (s)	label	split
A03	0.2001	0.1369	Arrhythmia	train
A04	0.3209	0.1412	Arrhythmia	train
A05	0.2587	0.1593	Arrhythmia	train
A06	0.2369	0.1243	Arrhythmia	train
A07	0.2745	0.1659	Arrhythmia	train
A08	0.3382	0.1317	Arrhythmia	test
A09	0.3587	0.1671	Arrhythmia	train
A10	0.4155	0.1760	Arrhythmia	test
A11	0.3677	0.1282	Arrhythmia	train
A12	0.4741	0.1531	Arrhythmia	train
A13	0.2818	0.1756	Arrhythmia	train
A14	0.5513	0.1531	Arrhythmia	test
A15	0.2110	0.1240	Arrhythmia	test
A16	0.4682	0.1629	Arrhythmia	test
A17	0.3669	0.1731	Arrhythmia	train
A18	0.4235	0.1612	Arrhythmia	test
A19	0.2561	0.1956	Arrhythmia	train
A20	0.2792	0.1669	Arrhythmia	train
A21	0.5203	0.1923	Arrhythmia	test
A22	0.5873	0.1310	Arrhythmia	train
A23	0.3254	0.1311	Arrhythmia	test
A24	0.4769	0.1846	Arrhythmia	train
A25	0.5506	0.1518	Arrhythmia	train
A26	1.5368	0.1332	Arrhythmia	test
A27	1.0510	0.1942	Arrhythmia	train
A28	1.0234	0.1478	Arrhythmia	train
A29	1.1019	0.1801	Arrhythmia	train
A30	1.5269	0.1781	Arrhythmia	train
A31	1.0590	0.1907	Arrhythmia	test
A32	1.2527	0.1699	Arrhythmia	train
A33	1.5747	0.1801	Arrhythmia	train
A34	1.3199	0.1479	Arrhythmia	test
A35	1.4151	0.1416	Arrhythmia	train
A36	1.1893	0.1917	Arrhythmia	test
A37	1.4119	0.1542	Arrhythmia	train
A38	1.5008	0.1972	Arrhythmia	train
A39	1.0110	0.1731	Arrhythmia	train
A40	1.4501	0.1697	Arrhythmia	train
A41	1.5933	0.1292	Arrhythmia	train
A42	1.4489	0.1960	Arrhythmia	train
A43	1.1683	0.1560	Arrhythmia	train
A44	1.4736	0.1663	Arrhythmia	test
A45	1.0619	0.1527	Arrhythmia	train
A46	1.2687	0.1390	Arrhythmia	train
A47	1.5452	0.1923	Arrhythmia	train
A48	1.1762	0.1659	Arrhythmia	test
A49	1.1727	0.1202	Arrhythmia	train
A50	1.0780	0.1694	Arrhythmia	test

sample	RR (s)	QRS (s)	label	split
N59	0.8280	0.0720	Normal	test

Appendix C — Reproducibility

Environment printed by the notebook (Step 0.1):

```
Student (GitHub) : EaindrayMFU26
Project          : FACE IT 2026 - Hands-on Mini Project (ECG, AHA DB record 0001)
Run at           : 2026-10-15 14:33 UTC
Python           : 3.13.16 (Linux x86_64)
Working folder   : /home/user/FACE-IT-2026-ECG-Mini-Project
numpy            : 2.5.3
scipy            : 1.18.1
matplotlib       : 3.11.2
pandas           : 3.0.5
sklearn          : 1.9.1
wfdb             : 4.3.1
```

Notebook	`EaindrayMFU26_ECG_Mini_Project.ipynb` (executed top-to-bottom in JupyterLab, execution counts 1–27)
Data	`data/ahadb/0001.*` (PhysioNet ahadb v1.0.0, SHA-256 verified); `data/arrhythmia_training_data.csv`
Outputs	`figures/` (plots, 200 dpi), `results/` (CSV/JSON tables), `screenshots/` (JupyterLab screenshots)
Re-run	`pip install -r requirements.txt`, then `jupyter lab` → Run ▸ Run All Cells

References

1. Goldberger A. L. et al. PhysioBank, PhysioToolkit, and PhysioNet: components of a new research resource for complex physiologic signals. *Circulation* 101(23):e215–e220, 2000.
2. PhysioNet AHA Database (ahadb), version 1.0.0. https://physionet.org/content/ahadb/1.0.0/
3. Pan J., Tompkins W. J. A real-time QRS detection algorithm. *IEEE Trans. Biomed. Eng.* 32(3):230–236, 1985.
4. Virtanen P. et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods* 17:261–272, 2020.
5. Pedregosa F. et al. Scikit-learn: machine learning in Python. *J. Mach. Learn. Res.* 12:2825–2830, 2011.